Learn Go with Tests

Chapter 1 - Why unit tests?

- The strength of software is that we can change it. Most software will need to change over time in unpredictable ways, but don't try to over-engineer for that, because the future is too hard to predict.
- Instead, we need to keep our software malleable. Software has to be refactored as it evolves, or it will turn into a mess.
- A good test suite lets you refactor faster and with less stress.
- Writing good unit tests is a design problem, so structure your code into meaningful units that you can fit together like Lego bricks.
- TDD can help, and even force, you to design well-factored software iteratively, backed by tests that support future work as it arrives.
- Wrapping up
    - The strength of software is that we can change it. Most software will require change over time in unpredictable ways, but don't try to over-engineer, because it's too hard to predict the future.
    - Instead we need to make it so we can keep our software malleable. In order to change software we have to refactor it as it evolves, or it will turn into a mess.
    - A good test suite can help you refactor quicker and in a less stressful manner.
    - Writing good unit tests is a design problem, so think about structuring your code so you have meaningful units that you can integrate together like Lego bricks.
    - TDD can help and force you to design well-factored software iteratively, backed by tests to help future work as it arrives.

Chapter 2 - Hello, World

- Run `go mod init MODULENAME` once, in the top-level folder, before running commands like `go test` or `go build`. Each chapter folder below it is its own package within that one module. We used `example.com/learn-go-with-tests`. Use a path with a dot in its first element, like a domain. Go reserves dot-free first elements for the standard library and its tools.

- Go finds `go.mod` by searching parent directories, so `go test` works inside a chapter folder, and `go test ./...` at the top runs every chapter's tests.
- Writing tests
  - The test must be in a file with a name like `xxx_test.go`.
  - The test function's name must start with `Test`.
  - The test function takes exactly one argument, `t *testing.T`.
  - To use the `*testing.T` type, you need to import `"testing"`.
- To see the documentation for `fmt`, run `go doc fmt`.
- You can also install pkgsite with `go install golang.org/x/pkgsite/cmd/pkgsite@latest`, then run `pkgsite -open` in your module directory to browse the docs at http://localhost:8080. The home page only links to your module, but the standard library is served too: browse http://localhost:8080/std or search for a package like `fmt`. `pkgsite -cache .` also serves your module cache; the `.` is needed there, since `-cache` alone serves only the cache.
- It is often useful to define one test like `TestHello` for the `Hello` function, then define several subtests within it using `t.Run`.
- Test helper functions should take `t testing.TB`. `testing.TB` is an interface that both `*testing.T` and `*testing.B` satisfy, so the helper works in tests and benchmarks. Inside the helper, we call `t.Helper()` and, when needed, `t.Errorf` or `t.Fail()`. `t.Helper()` makes failure messages report the line in the test that called the helper, instead of a line inside the helper.
- When consecutive parameters have the same type, you can write `(got, want string)` instead of `(got string, want string)`.
- `:=` is Go's short variable declaration (declare and assign). `=` is plain assignment to a variable that already exists.
- In Go, any identifier (function, type, constant, variable, or struct field) that starts with a capital letter is exported, meaning it is visible outside its package. One that starts with a lowercase letter is unexported, meaning it is private to its package.

```go
// 02-hello-world/hello.go
package main

import "fmt"

const (
	spanish = "Spanish"
	french  = "French"

	englishHelloPrefix = "Hello, "
	spanishHelloPrefix = "Hola, "
	frenchHelloPrefix  = "Bonjour, "
)

// Hello returns a personalised greeting in a given language.
func Hello(name string, language string) string {
	if name == "" {
		name = "World"
	}

	return greetingPrefix(language) + name
}

func greetingPrefix(language string) (prefix string) {
	switch language {
	case spanish:
		prefix = spanishHelloPrefix
	case french:
		prefix = frenchHelloPrefix
	default:
		prefix = englishHelloPrefix
	}
	return
}

func main() {
	fmt.Println(Hello("world", "English"))
}
```

```go
// 02-hello-world/hello_test.go
package main

import "testing"

func TestHello(t *testing.T) {
        t.Run("saying hello to people", func(t *testing.T) {
                got := Hello("Chris", "")
                want := "Hello, Chris"
                assertCorrectMessage(t, got, want)
        })
        t.Run("say 'Hello, World' when an empty string is supplied",
                func(t *testing.T) {
                        got := Hello("", "")
                        want := "Hello, World"
                        assertCorrectMessage(t, got, want)
                })
        t.Run("in Spanish", func(t *testing.T) {
                got := Hello("Elodie", "Spanish")
                want := "Hola, Elodie"
                assertCorrectMessage(t, got, want)
        })
        t.Run("in French", func(t *testing.T) {
                got := Hello("Marcella", "French")
                want := "Bonjour, Marcella"
                assertCorrectMessage(t, got, want)
        })
}

func assertCorrectMessage(t testing.TB, got, want string) {
        t.Helper()
        if got != want {
                t.Errorf("got %q want %q", got, want)
        }
}
```

- Wrapping up
    - Some of Go's syntax around:
        - Writing tests
        - Declaring functions, with arguments and return types
        - `if`, `const`, and `switch`
        - Declaring variables and constants

    - The TDD process and why the steps are important
        - Write a failing test and see it fail so we know we have written a relevant test for our requirements, and that it produces an easy-to-understand description of the failure.
        - Write the smallest amount of code to make it pass, so we know we have working software.
        - Then refactor, backed with the safety of our tests to ensure we have well-crafted code that is easy to work with.

Chapter 3 - Integers

- A doc comment is an ordinary `//` comment directly above a top-level package, func, type, const or var declaration, with no blank line in between.
- Example functions begin with `Example` and live in a package's `_test.go` files. They appear in your documentation and are compiled whenever tests run. The `// Output: 6` comment is what makes `go test` run the example and check its output; remove it and the example is compiled but never run. Examples without `Output` are useful for code that can't run as a unit test, such as code that accesses the network, since the compiler still checks that they build.
- `go test -v` lists each test and example as it runs, with its result.
- Run `pkgsite -open` (see the last chapter for installation) and browse to http://localhost:8080/ to see our doc comment and example.
- Aside from running `pkgsite`, you can see Go module documentation at https://pkg.go.dev/.

```go
// 03-integers/adder_test.go
package integers

import (
        "fmt"
        "testing"
)

func TestAdder(t *testing.T) {
        sum := Add(2, 2)
        expected := 4

        if sum != expected {
                t.Errorf("expected '%d' but got '%d'", expected, sum)
        }
}

func ExampleAdd() {
        sum := Add(1, 5)
        fmt.Println(sum)
        // Output: 6
}

// 03-integers/adder.go
package integers

// Add takes two integers and returns the sum of them.
func Add(x, y int) int {
        return x + y
}
```

- Wrapping up
    - We have covered:
        - More practice of the TDD workflow
        - Integers, addition
        - Writing better documentation so users of our code can understand its usage quickly
        - Examples of how to use our code, which are checked as part of our tests

Chapter 4 - Iteration

- To repeat things in Go, you need `for`. There are no `while`, `do`, or `until` keywords.
- In Go there are no parentheses surrounding the three components of the `for` statement. It's just `for i := 0; i < 5; i++ {`.
- You can also do an infinite loop with just `for {`. It's infinite, but can still be exited with `break`, `return`, or `panic`.
- We will see `for` with `range` in the next chapter.
- You can declare a variable `repeated` of `string` type like `var repeated string`. You can declare a constant like `const repeatCount = 5`.
- The `+=` operator is called the Add AND assignment operator, and it adds the right operand to the left operand and assigns the result to the left operand.
- In your test file you can declare benchmarks as `func BenchmarkXxx(b *testing.B) {`. `*testing.B` gives you access to `b.Loop()`, which returns true as long as the benchmark should keep running. To run benchmarks, run `go test -bench=.` (or `go test -bench="."` in PowerShell). By default benchmarks run sequentially.
- The standard library provides `strings.Builder`, which minimizes memory copying of strings. It has a `WriteString` method which is more efficient than repeated uses of `+=`. Switching it to `WriteString` was five times as fast in the benchmark.
- You can run `go test -bench=. -benchmem` to report memory allocations where `B/op` is the number of bytes allocated per iteration and `allocs/op` is the number of allocations per iteration.

```go
// 04-for/repeat_test.go
package iteration

import (
	"fmt"
	"testing"
)

func TestRepeat(t *testing.T) {
	repeated := Repeat("a", 5)
	expected := "aaaaa"

	if repeated != expected {
		t.Errorf("expected %q but got %q", expected, repeated)
	}
}

func BenchmarkRepeat(b *testing.B) {
	//... setup ...
	for b.Loop() {
		//... code to measure ...
		Repeat("a", 5)
	}
	// ... cleanup ...
}

func ExampleRepeat() {
	output := Repeat("a", 5)
	fmt.Println(output)
	// Output: aaaaa
}

// 04-for/repeat.go
package iteration

import "strings"

// Repeat returns character repeated repeatCount times.
func Repeat(character string, repeatCount int) string {
	var repeated strings.Builder
	for i := 0; i < repeatCount; i++ {
		repeated.WriteString(character)
	}
	return repeated.String()
}
```

- Wrapping up
    - More TDD practice
    - Learned `for`
    - Learned how to write benchmarks

Chapter 5 - Arrays and slices

- Arrays allow you to store multiple elements of the same type in a variable, in a particular order. Arrays have a fixed capacity, which you define when you declare the variable. We can initialize an array in two ways:
    - `[N]type{value1, value2, ..., valueN}`, e.g. `numbers := [5]int{1, 2, 3, 4, 5}`
    - `[...]type{value1, value2, ..., valueN}`, e.g. `numbers := [...]int{1, 2, 3, 4, 5}`, where the compiler counts the elements for you.
- You'll notice in our `Errorf` in this test code we're using a different placeholder. `%d` prints an integer in base 10, `%q` prints a double-quoted string with Go escapes (so an empty string shows up as `""`), and `%v` is the "default" format for any value, which works well for arrays (`[1 2 3]`).
- Remember as always to write the minimal amount of code for the test to run. After starting a test for `Sum`, define `Sum` and return a bogus result. You want to get to the point where you get a clear error message and not an `undefined: Sum` error.

```go
// 05-arrays/sum_test.go
package main

import "testing"

func TestSum(t *testing.T) {

        numbers := [5]int{1, 2, 3, 4, 5}

        got := Sum(numbers)
        want := 15

        if got != want {
                t.Errorf("got %d want %d given, %v", got, want, numbers)
        }
}

// 05-arrays/sum.go
package main

func Sum(numbers [5]int) int {
        return 0
}

// at this point `go test` should give:
// --- FAIL: TestSum (0.00s)
//  sum_test.go:13: got 0 want 15 given, [1 2 3 4 5]
```

- You could fix it like this by manually incrementing a counter and accessing the array by index like `array[index]`:

```go
// 05-arrays/sum.go
package main

func Sum(numbers [5]int) int {
        sum := 0
        for i := 0; i < 5; i++ {
                sum += numbers[i]
        }
        return sum
}
```

- But after that gets the tests to pass, we want to refactor to use `range`:

```
// 05-arrays/sum.go
package main

func Sum(numbers [5]int) int {
        sum := 0
        for _, number := range numbers {
                sum += number
        }
        return sum
}
```

- `range` lets you iterate over an array. Each iteration, `range` returns two values - the index and the value. We are choosing to ignore the index using `_` (the blank identifier).
- An interesting property of arrays is that the size is encoded in its type. If you try to pass a `[4]int` to this function that expects a `[5]int`, it won't compile even though our code would handle either now. Go has slices though which work with any size.
- If we write a new test that passes our function a slice, we will get a new error `cannot use numbers (type []int) as type [5]int in argument to Sum`. We have a choice between breaking our existing API by changing the `Sum` argument to be a slice or creating a new function. No one is using our API right now, so we're going to break it rather than have two functions to maintain. For reference, here is our code now that we're using slices and have fixed the compiler messages:

```
// 05-arrays/sum_test.go
package main

import "testing"

func TestSum(t *testing.T) {

        t.Run("collection of 5 numbers", func(t *testing.T) {
                numbers := []int{1, 2, 3, 4, 5}

                got := Sum(numbers)
                want := 15

                if got != want {
                        t.Errorf("got %d want %d given, %v", got, want,
numbers)
                }
        })

        t.Run("collection of any size", func(t *testing.T) {
                numbers := []int{1, 2, 3}

                got := Sum(numbers)
                want := 6

                if got != want {
                        t.Errorf("got %d want %d given, %v", got, want,
numbers)
                }
        })
}

// 05-arrays/sum.go
package main

func Sum(numbers []int) int {
        sum := 0
        for _, number := range numbers {
                sum += number
        }
        return sum
}
```

- You'll find most of the time slices are more flexible than arrays. They're defined like `[]int{1, 2, 3}` for some number of values. Type can vary but we're using `int` here.
- It shouldn't be our goal to have as many tests as possible, but rather to have as much confidence as possible in our code base. **Every test has a cost.** In our case, we have two tests that test passing a slice. Slices of different sizes aren't treated differently, so it's a little redundant.
- Go can analyze your test coverage by running `go test -cover`. If you run it now, you'll see you have 100% coverage. Delete one test, and you still have 100% coverage.
- We are going to define a new function `SumAll` that takes a varying number of slices and returns a new slice containing the totals for each slice passed in. First we'll write a test:

```
// 05-arrays/sum_test.go
// ...
func TestSumAll(t *testing.T) {

        got := SumAll([]int{1, 2}, []int{0, 9})
        want := []int{3, 9}

        if got != want {
                t.Errorf("got %v want %v", got, want)
        }
}
```

- That obviously fails with `undefined: SumAll`, so let's define it:

```
// 05-arrays/sum.go
func SumAll(numbersToSum ...[]int) []int {
        return nil
}
```

- If we try to run our tests now, we get a new error that teaches us something about slices: `invalid operation: got != want (slice can only be compared to nil)`. Go does not let you use equality operators with slices. You could iterate over the slice and compare values, but the `slices` package has been available since Go 1.21 and has a `slices.Equal` function to do a simple shallow compare of two slices of the same type. The function expects the elements to be comparable, so it can't be applied to slices with non-

comparable elements like 2D slices; use `slices.EqualFunc(a, b, slices.Equal[[]Int])` for those.

```
// 05-arrays/sum_test.go
func TestSumAll(t *testing.T) {

        got := SumAll([]int{1, 2}, []int{0, 9})
        want := []int{3, 9}

        if !slices.Equal(got, want) {
                t.Errorf("got %v want %v", got, want)
        }
}
```

- Now we get a normal error message about unexpected values: `sum_test.go:28: got [] want [3 9]`.
- So to fix this, we need to iterate over the varargs, calculate the sum using our existing `Sum` function, then add it to the slice we will return:

```
// 05-arrays/sum.go
// ...
func SumAll(numbersToSum ...[]int) []int {
        lengthOfNumbers := len(numbersToSum)
        sums := make([]int, lengthOfNumbers)

        for i, numbers := range numbersToSum {
                sums[i] = Sum(numbers)
        }

        return sums
}
```

- The book skipped over explaining varargs, but `numbersToSum ...[]int` means we can take any number of slices. Inside the function, `numbersToSum` is a `[][]int`.
- Now there's a new way to create a slice. `make([]int, n)` creates a slice of length `n` with every element set to the zero value. We're setting `n` to the `len` of the `numbersToSum` we need to work through. The length of a slice, `len(mySlice)`, is the number of elements it holds, while the capacity, `cap(mySlice)`, is the number of elements its underlying array has room for. A third argument sets capacity separately: `make([]int, 0, 5)` creates a

slice with length 0 and capacity 5. With only two arguments, as in our code, the capacity equals the length. After making our slice, we can index into it with `mySlice[N]` to get a value out or assign it a new value with `=`. This gets our tests to pass.

- Indexing is limited by a slice's length, not its capacity. If you have a slice of length 2 and try to do `mySlice[10] = 1`, you will get a *runtime error* (a panic). Instead, you can use the `append` function, which takes a slice and new values and returns a slice with the values added, growing the underlying array if needed. We will refactor our `SumAll` to do this so we don't need to manually assign by index:

```
// 05-arrays/sum.go
func SumAll(numbersToSum ...[]int) []int {
        var sums []int
        for _, numbers := range numbersToSum {
                sums = append(sums, Sum(numbers))
        }

        return sums
}
```

- Now we aren't manually managing length or capacity either. We start with an empty (`nil`) slice, `var sums []int`, and just `append` the result of `Sum` to it as we work through the varargs. Now we are going to change our requirements, though. Instead of `SumAll` we want `SumAllTails`, which calculates the total of the "tails" of each slice. The tail of a collection is all items in the collection except the first one (the "head"). I'm actually going to keep `SumAll` and just write and test `SumAllTails` separately.

```
// 05-arrays/sum_test.go
func TestSumAllTails(t *testing.T) {
        got := SumAllTails([]int{1, 2}, []int{0, 9})
        want := []int{2, 9}

        if !slices.Equal(got, want) {
                t.Errorf("got %v want %v", got, want)
        }
}
```

- If we run the test now, we get the expected `undefined` error: `undefined: SumAllTails`. If we copy in the body of `SumAll`, it compiles but we get the wrong values back, so let's use `numbers[1:]` to access the tail:

```
// 05-arrays/sum.go
func SumAllTails(numbersToSum ...[]int) []int {
        var sums []int
        for _, numbers := range numbersToSum {
                tail := numbers[1:]
                sums = append(sums, Sum(tail))
        }
        return sums
}
```

- This works. Slices can be sliced! The syntax is `slice[low:high]`, which includes `low` but not `high`. Omitting `low` means 0 and omitting `high` means `len(slice)`, so `numbers[1:]` says take everything from index 1 to the end. But what would happen if we passed it an empty slice? We can write the test!

```
// 05-arrays/sum_test.go
func TestSumAllTails(t *testing.T) {

        t.Run("make the sums of some tails of slices", func(t *testing.T) {
                got := SumAllTails([]int{1, 2}, []int{0, 9})
                want := []int{2, 9}

                if !slices.Equal(got, want) {
                        t.Errorf("got %v want %v", got, want)
                }
        })

        t.Run("safely sum empty slices", func(t *testing.T) {
                got := SumAllTails([]int{}, []int{3, 4, 5})
                want := []int{0, 9}

                if !slices.Equal(got, want) {
                        t.Errorf("got %v want %v", got, want)
                }
        })
}
```

- So testing with empty slices is generally a good idea, and here it finds a panic: `panic: runtime error: slice bounds out of range [1:0]`. `low` may be at most `len(slice)`, so `[]int{7}[1:]` is an empty slice, but `[]int{}[1:]` panics. What is the tail of an empty slice? We want its sum to be 0:

```
// 05-arrays/sum.go
func SumAllTails(numbersToSum ...[]int) []int {
        var sums []int
        for _, numbers := range numbersToSum {
                if len(numbers) == 0 {
                        sums = append(sums, 0)
                } else {
                        tail := numbers[1:]
                        sums = append(sums, Sum(tail))
                }
        }
        return sums
}
```

- We have repeating code in our assertions again, so we're going to factor that out into a helper:

```
// 05-arrays/sum_test.go
func TestSumAllTails(t *testing.T) {

        checkSums := func(t testing.TB, got, want []int) {
                t.Helper()
                if !slices.Equal(got, want) {
                        t.Errorf("got %v want %v", got, want)
                }
        }

        t.Run("make the sums of some tails of slices", func(t *testing.T) {
                got := SumAllTails([]int{1, 2}, []int{0, 9})
                want := []int{2, 9}
                checkSums(t, got, want)
        })

        t.Run("safely sum empty slices", func(t *testing.T) {
                got := SumAllTails([]int{}, []int{3, 4, 5})
                want := []int{0, 9}
                checkSums(t, got, want)
        })
}
```

- Since we only need the function in one place, we are declaring it and assigning it to a variable. It allows you to reduce the surface area of your API.

- Final version of this chapter's code:

```go
// 05-arrays/sum_test.go
package main

import (
	"slices"
	"testing"
)

func TestSum(t *testing.T) {

	t.Run("collection of any size", func(t *testing.T) {
		numbers := []int{1, 2, 3}

		got := Sum(numbers)
		want := 6

		if got != want {
			t.Errorf("got %d want %d given, %v", got, want,
numbers)
		}
	})
}

func TestSumAll(t *testing.T) {

	got := SumAll([]int{1, 2}, []int{0, 9})
	want := []int{3, 9}

	if !slices.Equal(got, want) {
		t.Errorf("got %v want %v", got, want)
	}
}

func TestSumAllTails(t *testing.T) {

	checkSums := func(t testing.TB, got, want []int) {
		t.Helper()
		if !slices.Equal(got, want) {
			t.Errorf("got %v want %v", got, want)
		}
	}

	t.Run("make the sums of some tails of slices", func(t *testing.T) {
		got := SumAllTails([]int{1, 2}, []int{0, 9})
		want := []int{2, 9}
```

```
                checkSums(t, got, want)
        })

        t.Run("safely sum empty slices", func(t *testing.T) {
                got := SumAllTails([]int{}, []int{3, 4, 5})
                want := []int{0, 9}
                checkSums(t, got, want)
        })
}

// 05-arrays/sum.go
package main

// Sum calculates the total from a slice of numbers.
func Sum(numbers []int) int {
        sum := 0
        for _, number := range numbers {
                sum += number
        }
        return sum
}

// SumAll calculates the respective sums of every slice passed in.
func SumAll(numbersToSum ...[]int) []int {
        var sums []int
        for _, numbers := range numbersToSum {
                sums = append(sums, Sum(numbers))
        }

        return sums
}

// SumAllTails calculates the sums of all but the first number given a
// collection of slices.
func SumAllTails(numbersToSum ...[]int) []int {
        var sums []int
        for _, numbers := range numbersToSum {
                if len(numbers) == 0 {
                        sums = append(sums, 0)
                } else {
                        tail := numbers[1:]
                        sums = append(sums, Sum(tail))
                }
        }
        return sums
}
```

- Wrapping up
  - We covered:
    - Arrays
    - Slices
      - The various ways to make them
      - How they have a fixed capacity but you can create new slices from old ones using `append`
      - How to slice, slices!
    - `len` to get the length of an array or slice
    - Test coverage tool (`go test -cover`)
    - `slices.Equal` and why it is needed instead of regular equality operators

Chapter 6 - Structs, methods & interfaces

- Alright, let's say we want to calculate the perimeter of a rectangle with decimal measurements of height and width. We know we'll need a `Perimeter(width float64, height float64)` function, where `float64` is a 64-bit floating point number. We start with a test, and we use the `%f` format string or more specifically `%.2f` to display our floats:

```
// 06-structs/shapes_test.go
package main

import "testing"

func TestPerimeter(t *testing.T) {
        got := Perimeter(10.0, 10.0)
        want := 40.0

        if got != want {
                t.Errorf("got %.2f want %.2f", got, want)
        }
}
```

- By now we should know how to fix it:

```go
// 06-structs/shapes.go
package main

func Perimeter(width float64, height float64) float64 {
        return 0
}
```

- That will change our error to: `shapes_test.go:10: got 0.00 want 40.00`. So again we write just enough code to make it pass:

```go
// 06-structs/shapes.go
package main

func Perimeter(width float64, height float64) float64 {
        return 2 * (width + height)
}
```

- Next we define `Area` the same way, test first.
- We're comparing floats with `==` and `!=` here, but it's safe because we're using whole numbers where they are exactly representable. If you were using 0.1 or results of division, you could use `cmp.Diff` with `cmpopts.EquateApprox` from `go-cmp`.
- Our `Area` and `Perimeter` functions work, but it isn't obvious they are meant for rectangles. We will create our own type called `Rectangle` to encapsulate the concept using a **struct**. A struct is just a named collection of fields where you can store data.

```go
// 06-structs/shapes.go
type Rectangle struct {
        Width  float64
        Height float64
}

// Then refactor our tests to use it
// 06-structs/shapes_test.go
package main

import "testing"

func TestPerimeter(t *testing.T) {
        rectangle := Rectangle{10.0, 10.0}
        got := Perimeter(rectangle)
        want := 40.0

        if got != want {
                t.Errorf("got %.2f want %.2f", got, want)
        }
}

func TestArea(t *testing.T) {
        rectangle := Rectangle{12.0, 6.0}
        got := Area(rectangle)
        want := 72.0

        if got != want {
                t.Errorf("got %.2f want %.2f", got, want)
        }
}
```

- Now when we run our tests we get an error, since our functions still expect two numbers but we're passing one `Rectangle`: `./shapes_test.go:7:19: not enough arguments in call to Perimeter`. So we will make our functions take rectangles and access their fields with the `myStruct.field` syntax:

```go
// 06-structs/shapes.go
package main

type Rectangle struct {
        Width  float64
        Height float64
}

func Perimeter(rectangle Rectangle) float64 {
        return 2 * (rectangle.Width + rectangle.Height)
}

func Area(rectangle Rectangle) float64 {
        return rectangle.Width * rectangle.Height
}
```

- And everything is green! But wait, we have a new requirement to calculate the area of circles. Of course we start with a test:

```go
// 06-structs/shapes_test.go
func TestArea(t *testing.T) {
        t.Run("rectangles", func(t *testing.T) {
                rectangle := Rectangle{12.0, 6.0}
                got := Area(rectangle)
                want := 72.0

                if got != want {
                        t.Errorf("got %.2f want %.2f", got, want)
                }
        })

        t.Run("circles", func(t *testing.T) {
                circle := Circle{10.0}
                got := Area(circle)
                want := 314.1592653589793

                if got != want {
                        t.Errorf("got %.2f want %.2f", got, want)
                }
        })
}
```

- And we have the error: `./shapes_test.go:27:13: undefined: Circle`. So let's define a `Circle` struct:

```
// 06-structs/shapes.go
type Circle struct {
        Radius float64
}
```

- We change our error message to: `./shapes_test.go:28:15: cannot use circle (variable of struct type Circle) as Rectangle value in argument to Area`.

- Some languages allow you to overload a function name, declaring it again with different parameters, but not Go. We could either declare a function with the same name in a new package, or we could define methods on our newly defined types instead.

- A method is a function with a receiver. A method declaration binds an identifier, the method name, to a method, and associates the method with the receiver's base type. Methods are very similar to functions, but they are called by invoking them on an instance of a particular type. Let's change our tests as if we already had methods defined:

```go
// 06-structs/shapes_test.go
func TestArea(t *testing.T) {
        t.Run("rectangles", func(t *testing.T) {
                rectangle := Rectangle{12.0, 6.0}
                got := rectangle.Area()
                want := 72.0

                if got != want {
                        t.Errorf("got %.2f want %.2f", got, want)
                }
        })

        t.Run("circles", func(t *testing.T) {
                circle := Circle{10.0}
                got := circle.Area()
                want := 314.1592653589793

                if got != want {
                        t.Errorf("got %.2f want %.2f", got, want)
                }
        })
}
```

- Now we have two errors: `./shapes_test.go:18:20: rectangle.Area undefined (type Rectangle has no field or method Area)` and `./shapes_test.go:28:17: circle.Area undefined (type Circle has no field or method Area)`. So let's define these as methods:

```
// 06-structs/shapes.go
type Rectangle struct {
        Width  float64
        Height float64
}

func (r Rectangle) Area() float64 {
        return 0
}

type Circle struct {
        Radius float64
}

func (c Circle) Area() float64 {
        return 0
}
```

- The syntax for declaring methods is almost the same as for functions aside from specifying the receiver: `func (receiverName ReceiverType) MethodName(args)`. When your method is called on a variable of that type, you get your reference to its data via the `receiverName` variable. In many programming languages this is done implicitly and you access the receiver via `this`. It is a convention to make the receiver variable the first letter of the type: `r Rectangle`. Now we just get errors because we're returning the wrong values:

```
--- FAIL: TestArea (0.00s)
    --- FAIL: TestArea/rectangles (0.00s)
        shapes_test.go:22: got 0.00 want 72.00
    --- FAIL: TestArea/circles (0.00s)
        shapes_test.go:32: got 0.00 want 314.16
```

- So we can attach calculations to our methods:

```go
// 06-structs/shapes.go
package main

import "math"

type Rectangle struct {
        Width  float64
        Height float64
}

func (r Rectangle) Area() float64 {
        return r.Width * r.Height
}

type Circle struct {
        Radius float64
}

func (c Circle) Area() float64 {
        return math.Pi * c.Radius * c.Radius
}
```

- We're green again, so it's time to refactor. We'll define a `checkArea` helper:

```
// 06-structs/shapes_test.go
func TestArea(t *testing.T) {

        checkArea := func(t testing.TB, shape Shape, want float64) {
                t.Helper()
                got := shape.Area()
                if got != want {
                        t.Errorf("got %g want %g", got, want)
                }
        }

        t.Run("rectangles", func(t *testing.T) {
                rectangle := Rectangle{12.0, 6.0}
                checkArea(t, rectangle, 72.0)
        })

        t.Run("circles", func(t *testing.T) {
                circle := Circle{10.0}
                checkArea(t, circle, 314.1592653589793)
        })
}
```

- But now we're asking for a `Shape` to be passed in. The error explains the problem: `./shapes_test.go:17:40: undefined: Shape`. How does something become a shape? We just tell Go what a `Shape` is using an interface declaration:

```
// 06-structs/shapes.go
type Shape interface {
        Area() float64
}
```

- We're creating a new type just like we did for `Rectangle` and `Circle`, but this time we're creating an `interface` rather than a `struct`. In Go **interface resolution is implicit**. If the type you pass matches what the interface is asking for (here having an `Area` method that returns a `float64`), it will compile. Our helper does not need to concern itself with whether the shape is a `Rectangle` or a `Circle` or even a `Triangle`. By declaring an interface, the helper is *decoupled* from the concrete types and only has the method it needs to do its job. This kind of approach of using interfaces to declare **only what you need** is very important in software design and will be covered in more detail in later sections.

- Now with everything green again, it's time for more refactoring. We will implement "table driven tests". Table driven tests are useful when you want to build a list of test cases that can be tested in the same manner.

```
// 06-structs/shapes_test.go
func TestArea(t *testing.T) {

        areaTests := []struct {
                shape Shape
                want  float64
        }{
                {Rectangle{12, 6}, 72.0},
                {Circle{10}, 314.1592653589793},
        }

        for _, tt := range areaTests {
                got := tt.shape.Area()
                if got != tt.want {
                        t.Errorf("got %g want %g", got, tt.want)
                }
        }
}
```

- The only new syntax here is creating an "anonymous struct" `areaTests`. We are declaring a slice of structs by using `[]struct` with two fields, the `shape` and the `want`. Then we fill the slice with cases. We then iterate over them just like we do any other slice, using the struct fields to run our tests. It is very easy to add a new shape. In addition, if a bug is found with `Area` it is very easy to add a new test case to exercise it before fixing it.

- Table driven tests can be a great item in your toolbox, but be sure that you have a need for the extra noise in the tests. They are a great fit when you wish to test various implementations of an interface, or if the data being passed in to a function has lots of different requirements that need testing.

- It's only one line to add a test for `Triangle`: `{Triangle{12, 6}, 36.0},`. If we tried to run it now we get `undefined: Triangle`, so let's add that to `shapes.go`.

```
// 06-structs/shapes.go
type Triangle struct {
        Base   float64
        Height float64
}
```

- So now we get a new error. `./shapes_test.go:23:4: cannot use Triangle{…} (value of struct type Triangle) as Shape value in struct literal: Triangle does not implement Shape (missing method Area)`. We can fix that by defining `Area` for `Triangle`:

```
// 06-structs/shapes.go
func (t Triangle) Area() float64 {
        return 0
}
```

- And now we just get `shapes_test.go:29: got 0 want 36`, so we need to define our calculation:

```
// 06-structs/shapes.go
func (t Triangle) Area() float64 {
        return (t.Base * t.Height) * 0.5
}
```

- And we're green again! So we can refactor. Our next pain point to clean up is our list of test cases for `TestArea`. Look at it:

```
                {Rectangle{12, 6}, 72.0},
                {Circle{10}, 314.1592653589793},
                {Triangle{12, 6}, 36.0},
```

- It's not immediately clear what all the numbers represent. So far, we've only seen structs like `MyStruct{val1, val2}` but you can optionally use the names of the fields:

```
                {shape: Rectangle{Width: 12, Height: 6}, want: 72.0},
                {shape: Circle{Radius: 10}, want: 314.1592653589793},
                {shape: Triangle{Base: 12, Height: 6}, want: 36.0},
```

- Our current table driven tests have the problem that it's hard to tell which one broke when something goes wrong. We can make it clearer by giving each case a `name` and passing it to `t.Run`, so it becomes the subtest's name in the output. We can also print the shape itself in the failure message with `%#v`, which formats a value as Go syntax, like `main.Rectangle{Width:12, Height:6}`. `want` is also a little vague, so let's make it `hasArea`:

```
// 06-structs/shapes_test.go
func TestArea(t *testing.T) {

        areaTests := []struct {
                name    string
                shape   Shape
                hasArea float64
        }{
                {name: "Rectangle", shape: Rectangle{Width: 12, Height: 6},
hasArea: 72.0},
                {name: "Circle", shape: Circle{Radius: 10}, hasArea:
314.1592653589793},
                {name: "Triangle", shape: Triangle{Base: 12, Height: 6},
hasArea: 36.0},
        }

        for _, tt := range areaTests {
                // using tt.name from the case to use it as the `t.Run` test
name
                t.Run(tt.name, func(t *testing.T) {
                        got := tt.shape.Area()
                        if got != tt.hasArea {
                                t.Errorf("%#v got %g want %g", tt.shape, got,
tt.hasArea)
                        }
                })
        }
}
```

- Let's look at the final code for this chapter:

```go
// 06-structs/shapes_test.go
package main

import "testing"

func TestPerimeter(t *testing.T) {
	rectangle := Rectangle{10.0, 10.0}
	got := Perimeter(rectangle)
	want := 40.0

	if got != want {
		t.Errorf("got %.2f want %.2f", got, want)
	}
}

func TestArea(t *testing.T) {

	areaTests := []struct {
		name    string
		shape   Shape
		hasArea float64
	}{
		{name: "Rectangle", shape: Rectangle{Width: 12, Height: 6},
hasArea: 72.0},
		{name: "Circle", shape: Circle{Radius: 10}, hasArea:
314.1592653589793},
		{name: "Triangle", shape: Triangle{Base: 12, Height: 6},
hasArea: 36.0},
	}

	for _, tt := range areaTests {
		// using tt.name from the case to use it as the `t.Run` test
name
		t.Run(tt.name, func(t *testing.T) {
			got := tt.shape.Area()
			if got != tt.hasArea {
				t.Errorf("%#v got %g want %g", tt.shape, got,
tt.hasArea)
			}
		})
	}
}

// 06-structs/shapes.go
package main
```

```go
import "math"

// Shape is implemented by anything that can tell us its Area.
type Shape interface {
	Area() float64
}

// Triangle represents the dimensions of a triangle.
type Triangle struct {
	Base   float64
	Height float64
}

// Area returns the area of the triangle.
func (t Triangle) Area() float64 {
	return (t.Base * t.Height) * 0.5
}

// Rectangle has the dimensions of a rectangle.
type Rectangle struct {
	Width  float64
	Height float64
}

// Area returns the area of the rectangle.
func (r Rectangle) Area() float64 {
	return r.Width * r.Height
}

// Circle represents a circle...
type Circle struct {
	Radius float64
}

// Area returns the area of the circle.
func (c Circle) Area() float64 {
	return math.Pi * c.Radius * c.Radius
}

// Perimeter returns the perimeter of a rectangle.
func Perimeter(rectangle Rectangle) float64 {
	return 2 * (rectangle.Width + rectangle.Height)
}
```

- Wrapping up
    - Declaring structs to create your own data types, which lets you bundle related data together and make the intent of your code clearer
    - Declaring interfaces so you can define functions that can be used by different types (ad hoc polymorphism)
    - Adding methods so you can add functionality to your data types and so you can implement interfaces
    - Table driven tests to make your assertions clearer and your test suites easier to extend & maintain

Chapter 7 - Pointers & errors

- We're going to make a `Wallet` struct that lets us deposit `Bitcoin`. The idea with this struct is controlling access to the data via methods.

```
// 07-pointers/wallet_test.go
package main

import "testing"

func TestWallet(t *testing.T) {

        wallet := Wallet{}

        wallet.Deposit(10)

        got := wallet.Balance()
        want := 10

        if got != want {
                t.Errorf("got %d want %d", got, want)
        }
}
```

- If we run the test now, we get `undefined: Wallet`. This is as easy to fix as adding a `type Wallet struct{}` line to `wallet.go`, but then we get `wallet.Deposit` and `wallet.Balance` undefined errors. We want the minimum code, so we'll define those to do nothing and return 0 (remember the receiver!). So now here is `wallet.go`:

```
// 07-pointers/wallet.go
package main

type Wallet struct{}

func (w Wallet) Deposit(amount int) {

}

func (w Wallet) Balance() int {
        return 0
}
```

- And we get an error of `got 0 want 10`. Our tests are executing, but we're not getting the right value back. We could slavishly work just to pass the test and hardcode a return value of 10, then require multiple tests to force us to write working code, but we aren't taking it that far! We'll go ahead and give our Wallet some state.

```
// 07-pointers/wallet.go
package main

type Wallet struct {
        balance int
}

func (w Wallet) Deposit(amount int) {
        w.balance += amount
}

func (w Wallet) Balance() int {
        return w.balance
}
```

- In Go, if an identifier starts with a lowercase letter, it is private to *the package it's defined in*. In this case, we want our methods to manipulate `balance`, but no one else.

- But when we run the test, we still get `got 0 want 10`. This looks like it should work. In Go, when you call a function or a method the arguments are copied, and that includes the receiver, so when calling `func (w Wallet) Deposit(amount int)` the `w` is a copy of whatever we called the method on. So to troubleshoot we'll print out the memory address of `balance` in the test and within the `Deposit` method using `&wallet.balance` and

`&w.balance`, since `&myVal` gives us the address of `myVal`. In our print statements, we can format it with `%p`. The `%p` placeholder prints memory addresses in base 16 notation with leading `0x` and `\n` prints a new line. Like this:

```
// 07-pointers/wallet_test.go
        got := wallet.Balance()
        fmt.Printf("address of balance in test is %p \n", &wallet.balance)
        want := 10

// 07-pointers/wallet.go
func (w Wallet) Deposit(amount int) {
        fmt.Printf("address of balance in Deposit is %p \n", &w.balance)
        w.balance += amount
}
```

- And when we run our test we see:

```
address of balance in Deposit is 0x41018ad90090
address of balance in test is 0x41018ad90088
```

- So when we change the value of the balance in the code, we are changing a copy separate from what is seen by the test. The way we fix this is with *pointers*. Rather than taking a copy of the Wallet, we instead take a pointer to that wallet so we can change the original values within it.

```
// 07-pointers/wallet.go
func (w *Wallet) Deposit(amount int) {
        w.balance += amount
}

func (w *Wallet) Balance() int {
        return w.balance
}
```

- We have changed the receiver type from `Wallet` to `*Wallet`, which you can read as "a pointer to a wallet". When we re-run the tests, they now pass. You may notice that we didn't dereference the pointer, like `(*w).balance`. This is because pointers to structs are called *struct pointers*, and they are automatically dereferenced. Also, technically we didn't need to change the receiver for `Balance`, as we weren't modifying anything and could read from a

copy, but by convention you should keep your method receiver types the same for consistency.

- Now that the tests pass, we want a simple refactor. We are using `int` for money, but we mean `Bitcoin`. `int` is actually fine for this, but we want a more descriptive name. So `type Bitcoin int` lets us create a `Bitcoin` type from `int`. To make `Bitcoin` we simply use `Bitcoin(999)`. By doing this, we can declare methods on our new type. We want to be able to print `Bitcoin`, so we will implement `Stringer`. For reference, here is `Stringer`:

```
type Stringer interface {
        String() string
}
```

- `Stringer` is defined in the `fmt` package and lets you define how your type is printed when used with the `%s` or `%v` format strings, or printed directly with `fmt.Println`.

```
func (b Bitcoin) String() string {
        return fmt.Sprintf("%d BTC", b)
}
```

- The syntax for creating a method on a type declaration is the same as for a struct. We didn't write a test in advance for this `String` method, so we'll add one now. We'll also add a separate test for "withdraw" within `TestWallet`:

```go
// 07-pointers/wallet_test.go
package main

import (
	"testing"
)

func TestWallet(t *testing.T) {

	t.Run("deposit", func(t *testing.T) {
		wallet := Wallet{}

		wallet.Deposit(Bitcoin(10))

		got := wallet.Balance()
		want := Bitcoin(10)

		if got != want {
			t.Errorf("got %s want %s", got, want)
		}
	})

	t.Run("withdraw", func(t *testing.T) {
		wallet := Wallet{balance: Bitcoin(20)}

		wallet.Withdraw(Bitcoin(10))

		got := wallet.Balance()
		want := Bitcoin(10)

		if got != want {
			t.Errorf("got %s want %s", got, want)
		}
	})
}

func TestBitcoin(t *testing.T) {
	t.Run("Bitcoin String", func(t *testing.T) {
		btc := Bitcoin(10)
		got := btc.String()
		want := "10 BTC"

		if got != want {
			t.Errorf("got %s want %s", got, want)
		}
```

```
        })
}
```

- And of course this fails with `wallet.Withdraw undefined`, so we add an empty method and get `got 20 BTC want 10 BTC`, so we specify `w.balance -= amount`. And we're back to green. We take this opportunity to factor out an `assertBalance` helper for both our balance checks.

- So now we need to think about what happens when we try to withdraw more than is in an account. In Go, it is idiomatic for your function to return an `error` for the caller to check and act on. So we'll assume that functionality is there when writing our test:

```
// 07-pointers/wallet_test.go
        t.Run("withdraw insufficient funds", func(t *testing.T) {
                startingBalance := Bitcoin(20)
                wallet := Wallet{startingBalance}
                err := wallet.Withdraw(Bitcoin(100))

                assertBalance(t, wallet, startingBalance)

                if err == nil {
                        t.Error("wanted an error but didn't get one")
                }
        })
```

- This new test fails with `./wallet_test.go:35:10: wallet.Withdraw(Bitcoin(100)) (no value) used as value`, which is a little unclear. It's actually a build failure because `Withdraw` has no return value. If we update `Withdraw`:

```
// 07-pointers/wallet.go
func (w *Wallet) Withdraw(amount Bitcoin) error {
        w.balance -= amount
        return nil
}
```

- Now we get a proper error message:

```
--- FAIL: TestWallet (0.00s)
    --- FAIL: TestWallet/withdraw_insufficient_funds (0.00s)
        wallet_test.go:37: got -80 BTC want 20 BTC
        wallet_test.go:40: wanted an error but didn't get one
```

- So clearly we need to both deal with the situation so we don't overdraw funds and return an `err`. `errors.New` is how you create an error.

```
// 07-pointers/wallet.go
func (w *Wallet) Withdraw(amount Bitcoin) error {

        if amount > w.balance {
                return errors.New("oh no")
        }

        w.balance -= amount
        return nil
}
```

- This gets our test suite green again, but we're going to refactor in two ways. First we want an `assertError` to match our `assertBalance`. Second, we want a better error message than "oh no", and to do that we need a way to test the error string.

```
// 07-pointers/wallet_test.go
func TestWallet(t *testing.T) {
        // ...
        assertError := func(t testing.TB, got error, want string) {
                t.Helper()

                if got == nil {
                        t.Fatal("wanted an error but didn't get one")
                }

                if got.Error() != want {
                        t.Errorf("got %q, want %q", got, want)
                }
        }
        // ...
        t.Run("withdraw insufficient funds", func(t *testing.T) {
                startingBalance := Bitcoin(20)
                wallet := Wallet{startingBalance}
                err := wallet.Withdraw(Bitcoin(100))

                assertError(t, err, "cannot withdraw, insufficient funds")
                assertBalance(t, wallet, startingBalance)
        })
}
```

- As you can see above, by calling the `Error` method on an `error`, we can convert it to a string. First we make sure it is not `nil`, so we don't call `Error` on `nil`. We're also using `t.Fatal`, which will stop the test if it is called. This is because we don't want to make any more assertions on the error returned if there isn't one around. Without this the test could carry on to the next step and panic because of a nil pointer. Now when we run the tests, we get `wallet_test.go:49: got "oh no", want "cannot withdraw, insufficient funds"`, which is as easy to fix as updating our error message in `wallet.go`. When we correct that and go green again, it is time to refactor. The current problem is we have our error message string hardcoded in two places. It would be really annoying for the test to fail if someone reworded the error. We will refactor this out to a `var ErrInsufficientFunds = errors.New("cannot withdraw, insufficient funds")` and rework our test to check for this instead of a string. While we're refactoring, we move the helpers out of the tests and into separate functions, so when someone opens up a file they can start reading assertions first instead of helpers. We also refactor our `assertError` to use `errors.Is` rather than an equality check so it can catch wrapped errors.

- The tests here help us understand the *real* usage of our code so we can make sympathetic code. We can see here that a developer can simply call our code, use `errors.Is` to check for `ErrInsufficientFunds`, and act accordingly.
- We will also install errcheck. To do so, run `go install github.com/kisielk/errcheck@latest`. It is a linter you can run by typing `errcheck .` in your project directory. If we run it now, it will say `wallet_test.go:19:18: wallet.Withdraw(Bitcoin(10))`. It is pointing out that we don't check for the absence of an `error` when we call `Withdraw` in our normal withdraw scenario. We will refactor our test now and define an `assertNoError` helper to check this.
- Also, I mentioned using `errors.Is` earlier, but here the book explains why. Imagine we have a method that calls this code:

```
func ProcessWithdrawal(wallet *Wallet, accountID string,
        amount Bitcoin) error {
        if err := wallet.Withdraw(amount); err != nil {
                return fmt.Errorf("processing withdrawal for account %s: %w",
                        accountID, err)
        }
        return nil
}
```

- The `%w` verb is like `%v` in that it substitutes in `err`'s string, but it also wraps `err` inside the new error that `fmt.Errorf` returns, rather than copying its message into a new, unrelated string. So we have the higher level context of `accountID` but still have the underlying `err` from underneath. Since `ProcessWithdrawal` returns a different error than `ErrInsufficientFunds`, comparing with `==` would fail. `errors.Is` solves this problem - it checks a chain of wrapped errors, not just the outermost one. It works by repeatedly calling `errors.Unwrap` on `err` until it either finds a match or runs out of errors to unwrap. If you need to check for an error of a particular *type* (and get at its fields) rather than compare against a sentinel value like `ErrInsufficientFunds`, then you'll need `errors.As`, which will be explained in the error types chapter later. Rule of thumb: use `errors.Is` to check *which* error it is, and `errors.As` to get *data out of* it.
- Final code:

```go
// 07-pointers/wallet_test.go
package main

import (
	"errors"
	"testing"
)

func TestWallet(t *testing.T) {

	t.Run("deposit", func(t *testing.T) {
		wallet := Wallet{}
		wallet.Deposit(Bitcoin(10))

		assertBalance(t, wallet, Bitcoin(10))
	})

	t.Run("withdraw", func(t *testing.T) {
		wallet := Wallet{balance: Bitcoin(20)}
		err := wallet.Withdraw(Bitcoin(10))

		assertNoError(t, err)
		assertBalance(t, wallet, Bitcoin(10))
	})

	t.Run("withdraw insufficient funds", func(t *testing.T) {
		startingBalance := Bitcoin(20)
		wallet := Wallet{startingBalance}
		err := wallet.Withdraw(Bitcoin(100))

		assertError(t, err, ErrInsufficientFunds)
		assertBalance(t, wallet, startingBalance)
	})
}

func TestBitcoin(t *testing.T) {
	t.Run("Bitcoin String", func(t *testing.T) {
		btc := Bitcoin(10)
		got := btc.String()
		want := "10 BTC"

		if got != want {
			t.Errorf("got %s want %s", got, want)
		}
	})
```

```
}

func assertBalance(t testing.TB, wallet Wallet, want Bitcoin) {
        t.Helper()
        got := wallet.Balance()

        if got != want {
                t.Errorf("got %s want %s", got, want)
        }
}

func assertNoError(t testing.TB, got error) {
        t.Helper()
        if got != nil {
                t.Fatal("got an error but didn't want one")
        }
}

func assertError(t testing.TB, got error, want error) {
        t.Helper()

        if got == nil {
                t.Fatal("wanted an error but didn't get one")
        }

        if !errors.Is(got, want) {
                t.Errorf("got %q, want %q", got, want)
        }
}

// 07-pointers/wallet.go
package main

import (
        "errors"
        "fmt"
)

// ErrInsufficientFunds means a wallet does not have enough Bitcoin to
// perform a withdraw.
var ErrInsufficientFunds = errors.New("cannot withdraw, insufficient funds")

// Bitcoin represents a number of Bitcoins.
type Bitcoin int

func (b Bitcoin) String() string {
```

```
        return fmt.Sprintf("%d BTC", b)
}

// Wallet stores the number of Bitcoin someone owns.
type Wallet struct {
        balance Bitcoin
}

// Deposit will add some Bitcoin to a wallet.
func (w *Wallet) Deposit(amount Bitcoin) {
        w.balance += amount
}

// Withdraw subtracts some Bitcoin from the wallet, returning an error if it
// cannot be performed.
func (w *Wallet) Withdraw(amount Bitcoin) error {

        if amount > w.balance {
                return ErrInsufficientFunds
        }

        w.balance -= amount
        return nil
}

// Balance returns the number of Bitcoin a wallet has.
func (w *Wallet) Balance() Bitcoin {
        return w.balance
}
```

- Wrapping up
    - Pointers
        - Go copies values when you pass them to functions/methods, so if you're writing a function that needs to mutate state you'll need it to take a pointer to the thing you want to change.
        - The fact that Go takes a copy of values is useful a lot of the time, but sometimes you won't want your system to make a copy of something, in which case you need to pass a reference. Examples include referencing very large data structures or things where only one instance is necessary (like database connection pools).
    - nil
        - Pointers can be nil

        - When a function returns a pointer to something, you need to make sure you check if it's nil, or you might get a runtime panic - the compiler won't help you here.
        - Useful for when you want to describe a value that could be missing
    - Errors
        - Errors are the way to signify failure when calling a function/method.
        - By listening to our tests we concluded that checking for a string in an error would result in a flaky test. So we refactored our implementation to use a meaningful value instead, and this resulted in easier to test code and concluded this would be easier for users of our API too.
        - This is not the end of the story with error handling; you can do more sophisticated things, but this is just an intro. Later sections will cover more strategies.
        - Don't just check errors, handle them gracefully
    - Create new types from existing ones
        - Useful for adding more domain specific meaning to values
        - Can let you implement interfaces

Chapter 8 - Maps

- Declaring a map is somewhat similar to an array, except it starts with the `map` keyword and requires two types. The key type is written in `[]` and the value type goes right after the `[]`, as in `map[string]string`. The key must be a comparable type (one that works with `==`), which is explained in depth in the language spec at https://go.dev/ref/spec#Comparison_operators. The value can be any type:

```
// 08-maps/dictionary_test.go
package main

import "testing"

func TestSearch(t *testing.T) {
        dictionary := map[string]string{"test": "this is just a test"}

        got := Search(dictionary, "test")
        want := "this is just a test"

        if got != want {
                t.Errorf("got %q want %q given, %q", got, want, "test")
        }
}

// and after we get "undefined: Search"
// 08-maps/dictionary.go
package main

func Search(dictionary map[string]string, word string) string {
        return ""
}
```

- Now our error is: `dictionary_test.go:12: got "" want "this is just a test" given, "test"`, so we can write some code. Most of the logic is in the map, though; all we need for the body of `Search` is `return dictionary[word]`. Getting a value out of a map looks like indexing an array, but with a key: `myMap[key]`.
- While we're green, we define an `assertStrings` helper to compare the results.
- I feel like we understand TDD well enough now that I'm not going to report every time we fail for something not defined or not returning the right values. Here we added a new test for a word not in the dictionary that required us to return an `error`. Until we modified the function to return two values, it wouldn't compile. And then we had to change our map lookup to check `ok` and return a real `ErrNotFound` when the entry isn't there. `ErrNotFound` is an honest to goodness error defined like we did last chapter. I'll post the whole file here in a minute. After the test passes, we also extract an `assertError` function and pass it our new `ErrNotFound`.
- A map lookup can return 2 values. The second value is a boolean which indicates if the key was found successfully. This lets us differentiate between a word that doesn't exist in the

map and one whose definition is the zero value (an empty string), since a missing key also reads as `""`.

- That's a lot of changes! Let's see where our code stands now that we're back to green and have refactored our error testing:

```go
// 08-maps/dictionary_test.go
package main

import (
	"errors"
	"testing"
)

func TestSearch(t *testing.T) {
	dictionary := Dictionary{"test": "this is just a test"}

	t.Run("known word", func(t *testing.T) {
		got, _ := dictionary.Search("test")
		want := "this is just a test"

		assertStrings(t, got, want)
	})

	t.Run("unknown word", func(t *testing.T) {
		_, got := dictionary.Search("unknown")
		if got == nil {
			t.Fatal("expected to get an error.")
		}
		assertError(t, got, ErrNotFound)
	})
}

func assertStrings(t testing.TB, got, want string) {
	t.Helper()

	if got != want {
		t.Errorf("got %q want %q", got, want)
	}
}

func assertError(t testing.TB, got, want error) {
	t.Helper()

	if !errors.Is(got, want) {
		t.Errorf("got error %q want %q", got, want)
	}
}

// 08-maps/dictionary.go
package main
```

```
import "errors"

type Dictionary map[string]string

var ErrNotFound = errors.New("could not find the word you were looking for")

func (d Dictionary) Search(word string) (string, error) {
        definition, ok := d[word]
        if !ok {
                return "", ErrNotFound
        }
        return definition, nil
}
```

- A dictionary with one word in it isn't very helpful, so we need a way to add words and for that we need a `TestAdd`.

```
// 08-maps/dictionary_test.go
func TestAdd(t *testing.T) {
        dictionary := Dictionary{}
        dictionary.Add("test", "this is just a test")

        want := "this is just a test"
        got, err := dictionary.Search("test")
        if err != nil {
                t.Fatal("should find added word:", err)
        }
        assertStrings(t, got, want)
}
```

- I also learned a little more about gopls. `grd` jumps to definition, then `Ctrl-o` / `Ctrl-i` jump back and forward. `gra` opens the code actions menu; on an undefined function it offers to create a stub for it (check the signature afterward, since gopls often guesses the return types wrong). `grn` is rename symbol. `grd` is one of a series of jumps: `grd` / `grt` / `gri` go to definition/type definition/implementation. `gO` lists the symbols in the current file and `gW` searches symbols project wide. `gW` is a live search, so its list stays empty until you start typing a name. `K` on a method shows its documentation. And `<leader>sd` (space, then `sd`) searches diagnostics. That's a few helpful shortcuts; there are many more. (GOPLS Shortcuts)

- I started running my tests automatically by splitting my tmux window with `C-a %` then switching to my `learning-go-with-tests` directory and running `gotestsum -watch`. I have to restart it when I make a new directory. To workaround this, I now run a zsh alias `gtw` for go test watch.

- The body of our new `Add` function is simply `d[word] = definition`. It takes two string parameters: `word` and `definition`. Adding to a map is similar to assigning to an array element. You just need to specify a key and set it equal to a value. An interesting property of maps is that you can modify them without passing their address (e.g. `&myMap`). A map value is really a pointer to the underlying data structure, so passing a map only copies that pointer.

- A gotcha with maps is that they can be a `nil` value. A `nil` map behaves like an empty map when reading, but attempts to write to a `nil` map will cause a runtime panic. So never declare a map with `var m map[string]string` and then write to it. Instead do `var dictionary = map[string]string{}`, with the `{}` at the end, or `var dictionary = make(map[string]string)`. Both approaches create an empty hash map and point `dictionary` at it, so you will never get a runtime panic.

- We did factor out an `assertDefinition` test helper and use it in `TestAdd`, along with having another test:

```go
// 08-maps/dictionary_test.go
func TestAdd(t *testing.T) {
        t.Run("new word", func(t *testing.T) {
                dictionary := Dictionary{}
                word := "test"
                definition := "this is just a test"

                err := dictionary.Add(word, definition)

                assertError(t, err, nil)
                assertDefinition(t, dictionary, word, definition)
        })

        t.Run("existing word", func(t *testing.T) {
                word := "test"
                definition := "this is just a test"
                dictionary := Dictionary{word: definition}
                err := dictionary.Add(word, "new test")
                assertError(t, err, ErrWordExists)
                assertDefinition(t, dictionary, word, definition)
        })
}

func assertDefinition(t testing.TB, dictionary Dictionary, word, definition
string) {
        t.Helper()
        got, err := dictionary.Search(word)
        if err != nil {
                t.Fatal("should find added word:", err)
        }
        assertStrings(t, got, definition)
}
```

- Our add function doesn't return anything yet and `ErrWordExists` needs definition:

```
// 08-maps/dictionary.go
var (
        ErrNotFound   = errors.New("could not find the word you were looking
for")
        ErrWordExists = errors.New("cannot add word because it already
exists")
)

func (d Dictionary) Add(word, definition string) error {
        _, err := d.Search(word)

        switch err {
        case ErrNotFound:
                d[word] = definition
        case nil:
                return ErrWordExists
        default:
                return err
        }

        return nil
}
```

- By using a `switch` to check the error we have an extra safety net in case `Search` returns an error other than `ErrNotFound`. We're also going to refactor our errors now that we're green again:

```
// 08-maps/dictionary.go
const (
        ErrNotFound   = DictionaryErr("could not find the word you were
looking for")
        ErrWordExists = DictionaryErr("cannot add word because it already
exists")
)

type DictionaryErr string

func (e DictionaryErr) Error() string {
        return string(e)
}
```

- So here we changed our errors to constants; this required us to create our own `DictionaryErr` type, which implements the `error` interface. Note that `T(x)` is Go's type conversion syntax. We're converting a string error message to a `DictionaryErr`, and in the `Error` method converting it back to a `string`. Go lets you convert freely between types that share an underlying type. Note that `string(someInt)` doesn't convert to the string form of the number but instead treats the integer as a Unicode code point (`go vet` warns about it). Use `strconv.Itoa(someInt)` or `fmt.Sprint(someInt)`. But converting between `string` types like we do here is always safe.

- Now we want to add a way to `Update` definitions; test first of course.

```
// 08-maps/dictionary_test.go
func TestUpdate(t *testing.T) {
        word := "test"
        definition := "this is just a test"
        dictionary := Dictionary{word: definition}
        newDefinition := "new definition"

        dictionary.Update(word, newDefinition)

        assertDefinition(t, dictionary, word, newDefinition)
}

// and after we see the test fail
// 08-maps/dictionary.go
func (d Dictionary) Update(word, definition string) {
        d[word] = definition
}
```

- But this has the reverse of the problem with `Add`. If you give it a word not in the dictionary, it adds it. Let's write some tests to catch this behavior:

```go
// 08-maps/dictionary_test.go
func TestUpdate(t *testing.T) {
	t.Run("existing word", func(t *testing.T) {
		word := "test"
		definition := "this is just a test"
		dictionary := Dictionary{word: definition}
		newDefinition := "new definition"

		err := dictionary.Update(word, newDefinition)

		assertError(t, err, nil)
		assertDefinition(t, dictionary, word, newDefinition)
	})

	t.Run("new word", func(t *testing.T) {
		word := "test"
		definition := "this is just a test"
		dictionary := Dictionary{}

		err := dictionary.Update(word, definition)

		assertError(t, err, ErrWordDoesNotExist)
	})
}

// 08-maps/dictionary.go
const (
	ErrNotFound         = DictionaryErr("could not find the word you were
looking for")
	ErrWordExists       = DictionaryErr("cannot add word because it
already exists")
	ErrWordDoesNotExist = DictionaryErr("cannot perform operation on word
because it does not exist")
)

func (d Dictionary) Update(word, definition string) error {
	_, err := d.Search(word)

	switch err {
	case ErrNotFound:
		return ErrWordDoesNotExist
	case nil:
		d[word] = definition
	default:
		return err
```

```
        }

        return nil
}
```

- This looks almost identical to `Add`, but the two cases are swapped: we update the `dictionary` on `nil` and return an error on `ErrNotFound`.
- We could have reused `ErrNotFound` instead of declaring a new error, but it is often better to have a precise error for when an update fails. Specific errors give you more information about what went wrong. In a webapp, for example, we might want to redirect on `ErrNotFound` and display an error on `ErrWordDoesNotExist`.
- Now we're going to create a `Delete` function:

```
// 08-maps/dictionary_test.go
func TestDelete(t *testing.T) {
        word := "test"
        dictionary := Dictionary{word: "test definition"}

        dictionary.Delete(word)

        _, err := dictionary.Search(word)
        assertError(t, err, ErrNotFound)
}

// 08-maps/dictionary.go
func (d Dictionary) Delete(word string) {
        delete(d, word)
}
```

- The `delete` command takes a map and a key and deletes the key from the map.
- This is green, but we might as well refactor to handle cases where the word doesn't exist. We should recognize the pattern by now:

```go
// 08-maps/dictionary_test.go
func TestDelete(t *testing.T) {
	t.Run("existing word", func(t *testing.T) {
		word := "test"
		dictionary := Dictionary{word: "test definition"}

		err := dictionary.Delete(word)

		assertError(t, err, nil)
		_, err = dictionary.Search(word)
		assertError(t, err, ErrNotFound)
	})

	t.Run("non-existing word", func(t *testing.T) {
		word := "test"
		dictionary := Dictionary{}

		err := dictionary.Delete(word)
		assertError(t, err, ErrWordDoesNotExist)
	})
}

// 08-maps/dictionary.go
func (d Dictionary) Delete(word string) error {
	_, err := d.Search(word)

	switch err {
	case ErrNotFound:
		return ErrWordDoesNotExist
	case nil:
		delete(d, word)
	default:
		return err
	}

	return nil
}
```

- Final code:

```go
// 08-maps/dictionary_test.go
package main

import (
	"errors"
	"testing"
)

func TestSearch(t *testing.T) {
	dictionary := Dictionary{"test": "this is just a test"}

	t.Run("known word", func(t *testing.T) {
		got, _ := dictionary.Search("test")
		want := "this is just a test"

		assertStrings(t, got, want)
	})

	t.Run("unknown word", func(t *testing.T) {
		_, got := dictionary.Search("unknown")
		if got == nil {
			t.Fatal("expected to get an error.")
		}
		assertError(t, got, ErrNotFound)
	})
}

func TestAdd(t *testing.T) {
	t.Run("new word", func(t *testing.T) {
		dictionary := Dictionary{}
		word := "test"
		definition := "this is just a test"

		err := dictionary.Add(word, definition)

		assertError(t, err, nil)
		assertDefinition(t, dictionary, word, definition)
	})

	t.Run("existing word", func(t *testing.T) {
		word := "test"
		definition := "this is just a test"
		dictionary := Dictionary{word: definition}
		err := dictionary.Add(word, "new test")
		assertError(t, err, ErrWordExists)
```

```
                assertDefinition(t, dictionary, word, definition)
        })
}

func TestUpdate(t *testing.T) {
        t.Run("existing word", func(t *testing.T) {
                word := "test"
                definition := "this is just a test"
                dictionary := Dictionary{word: definition}
                newDefinition := "new definition"

                err := dictionary.Update(word, newDefinition)

                assertError(t, err, nil)
                assertDefinition(t, dictionary, word, newDefinition)
        })

        t.Run("new word", func(t *testing.T) {
                word := "test"
                definition := "this is just a test"
                dictionary := Dictionary{}

                err := dictionary.Update(word, definition)

                assertError(t, err, ErrWordDoesNotExist)
        })
}

func TestDelete(t *testing.T) {
        t.Run("existing word", func(t *testing.T) {
                word := "test"
                dictionary := Dictionary{word: "test definition"}

                err := dictionary.Delete(word)

                assertError(t, err, nil)
                _, err = dictionary.Search(word)
                assertError(t, err, ErrNotFound)
        })

        t.Run("non-existing word", func(t *testing.T) {
                word := "test"
                dictionary := Dictionary{}

                err := dictionary.Delete(word)
                assertError(t, err, ErrWordDoesNotExist)
```

```
        })
}

func assertStrings(t testing.TB, got, want string) {
        t.Helper()

        if got != want {
                t.Errorf("got %q want %q", got, want)
        }
}

func assertError(t testing.TB, got, want error) {
        t.Helper()

        if !errors.Is(got, want) {
                t.Errorf("got error %q want %q", got, want)
        }
}

func assertDefinition(t testing.TB, dictionary Dictionary, word, definition
string) {
        t.Helper()
        got, err := dictionary.Search(word)
        if err != nil {
                t.Fatal("should find added word:", err)
        }
        assertStrings(t, got, definition)
}

// 08-maps/dictionary.go
package main

const (
        // ErrNotFound means the definition could not be found for the given
word
        ErrNotFound = DictionaryErr("could not find the word you were looking
for")

        // ErrWordExists means you are trying to add a word that is already
known
        ErrWordExists = DictionaryErr("cannot add word because it already
exists")

        // ErrWordDoesNotExist occurs when trying to perform an operation on
a word
        // not in the dictionary
```

```
        ErrWordDoesNotExist = DictionaryErr("cannot perform operation on word
because it does not exist")
)

// Dictionary store definitions to words.
type Dictionary map[string]string

// DictionaryErr are errors that can happen when interacting with the
// dictionary.
type DictionaryErr string

func (e DictionaryErr) Error() string {
        return string(e)
}

// Add inserts a word and definition into the dictionary.
func (d Dictionary) Add(word, definition string) error {
        _, err := d.Search(word)

        switch err {
        case ErrNotFound:
                d[word] = definition
        case nil:
                return ErrWordExists
        default:
                return err
        }

        return nil
}

// Delete removes a word from the dictionary.
func (d Dictionary) Delete(word string) error {
        _, err := d.Search(word)

        switch err {
        case ErrNotFound:
                return ErrWordDoesNotExist
        case nil:
                delete(d, word)
        default:
                return err
        }

        return nil
}
```

```go
// Update changes the definition of a given word.
func (d Dictionary) Update(word, definition string) error {
	_, err := d.Search(word)

	switch err {
	case ErrNotFound:
		return ErrWordDoesNotExist
	case nil:
		d[word] = definition
	default:
		return err
	}

	return nil
}

// Search find a word in the dictionary.
func (d Dictionary) Search(word string) (string, error) {
	definition, ok := d[word]
	if !ok {
		return "", ErrNotFound
	}
	return definition, nil
}
```

- Wrapping up
  - In this section we made a full CRUD (Create, Read, Update and Delete) API for our dictionary and throughout the process learned how to:
    - Create maps
    - Search for items in maps
    - Add new items to maps
    - Update items in maps
    - Delete items from a map
    - Learned more about errors
      - How to create errors that are constants
      - Writing error wrappers

Chapter 9 - Dependency Injection

- The benefits of dependency injection include:
  - You don't need a framework
  - It does not overcomplicate your design
  - It facilitates testing
  - It allows you to write great, general-purpose functions.
- Imagine we have the following function:

```
func Greet(name string) {
        fmt.Printf("Hello, %s", name)
}
```

- We want to test it, but it prints to stdout. What we need to do is **inject** the dependency of printing. Our function doesn't need to care *where* or *how* the printing happens, so we should accept an *interface* rather than a concrete type. If we do that, we can then change the implementation to print to something we control so we can test it. In "real life" we would inject something that writes to stdout. You can see in the source code for `fmt.Printf` there is already a seam for us to hook in:

```
// It returns the number of bytes written and any write error encountered
func Printf(format string, a ...interface{}) (n int, err error) {
        return Fprintf(os.Stdout, format, a...)
}
```

- But what exactly is an `os.Stdout`? We can see from the definition of `Fprintf`:

```
func Fprintf(w io.Writer, format string, a ...interface{}) (n int, err error)
{
        p := newPrinter()
        p.doPrintf(format, a)
        n, err = w.Write(p.buf)
        p.free()
        return
}
```

- An `io.Writer`:

```
type Writer interface {
        Write(p []byte) (n int, err error)
}
```

- So we see that `os.Stdout` implements `io.Writer`; `Printf` passes `os.Stdout` to `Fprintf` which expects an `io.Writer`. As you write more Go code, you'll see this interface popping up a lot because it's a great general purpose interface for "put this data somewhere". So we know under the covers we're ultimately using `Writer` to send our greeting somewhere and we will use this existing abstraction to make our code testable and more reusable.

```
// 09-di/di_test.go
package main

import (
        "bytes"
        "testing"
)

func TestGreet(t *testing.T) {
        buffer := bytes.Buffer{}
        Greet(&buffer, "Chris")

        got := buffer.String()
        want := "Hello, Chris"

        if got != want {
                t.Errorf("got %q want %q", got, want)
        }
}
```

- The `Buffer` type from the `bytes` package implements the `Writer` interface, because it has the method `Write(p []byte) (n int, err error)`. We start with an implementation that ignores the `Buffer` and does `fmt.Printf`.

```
// 09-di/di.go
package main

import (
        "bytes"
        "fmt"
)

func Greet(writer *bytes.Buffer, name string) {
        fmt.Printf("Hello, %s", name)
}
```

- That gets us to the `got "" want "Hello, Chris"` error message. We can even see the output when the test runs. Remember `fmt.Fprintf` is like `fmt.Printf` but takes a `Writer`, whereas `fmt.Printf` defaults to stdout.

```
// 09-di/di.go
func Greet(writer *bytes.Buffer, name string) {
        fmt.Fprintf(writer, "Hello, %s", name)
}
```

- We have made our new function too specific now. It only takes a `*bytes.Buffer` and not `os.Stdout`. If we change this, we can use `Greet` in both tests and `main`:

```
// 09-di/di.go
package main

import (
        "fmt"
        "io"
        "os"
)

func Greet(writer io.Writer, name string) {
        fmt.Fprintf(writer, "Hello, %s", name)
}

func main() {
        Greet(os.Stdout, "Elodie")
}
```

- Now that we take `io.Writer` we have options for where to send our greeting. We could pass in an `http.ResponseWriter` and define a `MyGreeterHandler`. `http.ResponseWriter` also implements `io.Writer` allowing us to re-use `Greet`. We do this in the final code for the chapter below:

```go
// 09-di/di_test.go
package main

import (
        "bytes"
        "testing"
)

func TestGreet(t *testing.T) {
        buffer := bytes.Buffer{}
        Greet(&buffer, "Chris")

        got := buffer.String()
        want := "Hello, Chris"

        if got != want {
                t.Errorf("got %q want %q", got, want)
        }
}

// 09-di/di.go
package main

import (
        "fmt"
        "io"
        "log"
        "net/http"
)

// Greet sends a personalised greeting to writer.
func Greet(writer io.Writer, name string) {
        fmt.Fprintf(writer, "Hello, %s", name)
}

// MyGreeterHandler says Hello, world over HTTP.
func MyGreeterHandler(w http.ResponseWriter, r *http.Request) {
        Greet(w, "world")
}

func main() {
        log.Fatal(http.ListenAndServe(":5001",
http.HandlerFunc(MyGreeterHandler)))
}
```

- Our first round of code was not easy to test because it wrote data to somewhere we couldn't control. Motivated by our tests we refactored the code so we could control where the data was written by **injecting a dependency**, which allowed us to:
    - **Test our code.** If you can't test a function easily, it's usually because of dependencies hard-wired into a function or global state. If you have a global database connection pool, for instance, that is used by some kind of service layer, it is likely going to be difficult to test and slow to run. DI will motivate you to inject in a database dependency (via an interface) which you can then mock out with something you can control in your tests.
    - **Separate our concerns**, decoupling *where the data goes* from *how to generate it*. If you ever feel like a method/function has too many responsibilities (generating data and writing to a db? handling HTTP requests and doing domain level logic?) DI is probably going to be the tool you need.
    - **Allow our code to be re-used in different contexts.** The first "new" context our code can be used in is inside tests. But further on, if someone wants to try something new with your function, they can inject their own dependencies.
- You use mocking to replace real things you inject with a pretend version that you can control and inspect in your tests. We'll discuss it more next chapter. In this example, the standard library had something ready for us to use.

- By having some familiarity with the `io.Writer` interface we are able to use `bytes.Buffer` in our test as our `Writer`, and then we can use other `Writer`s from the standard library to use our function in a command line app or in a web server. The more familiar you are with the standard library, the more you'll see these general purpose interfaces, which you can then re-use in your own code to make your software reusable in a number of contexts.

Chapter 10 - Mocking

- We want a program that counts down from 3, printing each number on a new line with a 1-second pause, and when it reaches zero it will print "Go!" and exit. We want to design this simple application through an iterative, test-driven approach. It's an important skill to be able to slice up requirements as small as you can so you can have working software. So we're breaking this into three steps:
    - Print 3
    - Print 3, 2, 1 and Go!
    - Wait a second between each line

- We just learned last chapter how to test a program that prints to stdout.

```go
// 10-mocking/countdown_test.go
package main

import (
        "bytes"
        "testing"
)

func TestCountdown(t *testing.T) {
        buffer := &bytes.Buffer{}

        Countdown(buffer)

        got := buffer.String()
        want := "3"

        if got != want {
                t.Errorf("got %q want %q", got, want)
        }
}

// then we see Countdown undefined, then we need to take arguments, then we
need
// to print 3!
// 10-mocking/main.go
package main

import (
        "fmt"
        "io"
        "os"
)

func main() {
        Countdown(os.Stdout)
}

func Countdown(out io.Writer) {
        fmt.Fprint(out, "3")
}
```

- Now we have completed step 1. We print 3 to stdout in main and print it to a buffer for testing. It seems trivial, but this is the approach the book author recommends. **Take a thin slice of functionality and make it work end-to-end, backed by tests.**
- Now let's make it print 2, 1, and then "Go!". First update the test. Note you can declare a *raw string literal* surrounded by backticks (`` `string` ``), which can include things like newlines:

```
// 10-mocking/countdown_test.go
        got := buffer.String()
        want := `3
2
1
Go!`

// And update the code to print the new output
// 10-mocking/main.go
func Countdown(out io.Writer) {
        for i := 3; i > 0; i-- {
                fmt.Fprintln(out, i)
        }
        fmt.Fprint(out, "Go!")
}
```

- After pulling out some constants and adding the pause, here is our final code:

```
// 10-mocking/main.go
package main

import (
        "fmt"
        "io"
        "os"
        "time"
)

const finalWord = "Go!"
const countdownStart = 3

func main() {
        Countdown(os.Stdout)
}

func Countdown(out io.Writer) {
        for i := countdownStart; i > 0; i-- {
                fmt.Fprintln(out, i)
                time.Sleep(1 * time.Second)
        }
        fmt.Fprint(out, finalWord)
}
```

- This works well, with a few problems. Now our test takes 3 seconds to run. Slow tests ruin developer productivity. What if we had to add functionality and wait for these tests with every change? We also haven't tested our pause, which is an important part of the application requirements. So we need to extract our dependency on `Sleep`ing. If we *mock* `time.Sleep`, we can use *dependency injection* to use it instead of a "real" `time.Sleep`, and then we can **spy on the calls** to make assertions on them. We will start with the tests:

```
// 10-mocking/countdown_test.go
package main

import (
        "bytes"
        "testing"
)

func TestCountdown(t *testing.T) {
        buffer := &bytes.Buffer{}
        spySleeper := &SpySleeper{}

        Countdown(buffer, spySleeper)

        got := buffer.String()
        want := `3
2
1
Go!`

        if got != want {
                t.Errorf("got %q want %q", got, want)
        }

        if spySleeper.Calls != 3 {
                t.Errorf("not enough calls to sleeper, want 3 got %d",
spySleeper.Calls)
        }
}

type SpySleeper struct {
        Calls int
}

func (s *SpySleeper) Sleep() {
        s.Calls++
}

// and after we get our compiler errors, we fix our main.go
// 10-mocking/main.go
package main

import (
        "fmt"
        "io"
```

```
        "os"
        "time"
)

const finalWord = "Go!"
const countdownStart = 3

type Sleeper interface {
        Sleep()
}

type DefaultSleeper struct{}

func (d *DefaultSleeper) Sleep() {
        time.Sleep(1 * time.Second)
}

func main() {
        sleeper := &DefaultSleeper{}
        Countdown(os.Stdout, sleeper)
}

func Countdown(out io.Writer, sleeper Sleeper) {
        for i := countdownStart; i > 0; i-- {
                fmt.Fprintln(out, i)
                sleeper.Sleep()
        }
        fmt.Fprint(out, finalWord)
}
```

- *Spies* are a kind of *mock* that can record how a dependency is used. They can record the arguments sent in, how many times it has been called, etc. In our case, we're keeping track of how many times `Sleep()` is called so we can check it in our test.
- So our latest test is a marked improvement. It no longer takes 3 seconds to run and it verified `Sleep` was called 3 times. But there's another important property we haven't tested. We are supposed to go Print, Sleep, Print, Sleep, etc. For all we know, it might be sleeping 3 seconds then printing everything at once. When writing tests, if you're not confident that your tests are testing everything, just break it! (after committing your work to source control, of course). If we change it to sleep 3 seconds and print everything at once, we will see our tests still pass. We will use spying again with a new test to check this. We have two different dependencies and we want to record all of their operations into one list, so we'll create one spy for them both. Our spy needs to implement both the `Sleeper` and

`io.Writer` interfaces. For this test, we're only concerned about the order of operations, so just recording them as a list of named operations is sufficient.

```go
// 10-mocking/countdown_test.go
package main

import (
	"bytes"
	"reflect"
	"testing"
)

func TestCountdown(t *testing.T) {
	t.Run("prints 3 to Go!", func(t *testing.T) {
		buffer := &bytes.Buffer{}
		Countdown(buffer, &SpyCountdownOperations{})

		got := buffer.String()
		want := `3
2
1
Go!`

		if got != want {
			t.Errorf("got %q want %q", got, want)
		}

	})

	t.Run("sleep before every print", func(t *testing.T) {
		spySleepPrinter := &SpyCountdownOperations{}
		Countdown(spySleepPrinter, spySleepPrinter)

		want := []string{
			write,
			sleep,
			write,
			sleep,
			write,
			sleep,
			write,
		}

		if !reflect.DeepEqual(want, spySleepPrinter.Calls) {
			t.Errorf("wanted calls %v got %v", want,
spySleepPrinter.Calls)
		}
	})
```

```
}

type SpyCountdownOperations struct {
        Calls []string
}

func (s *SpyCountdownOperations) Sleep() {
        s.Calls = append(s.Calls, sleep)
}

func (s *SpyCountdownOperations) Write(p []byte) (n int, err error) {
        s.Calls = append(s.Calls, write)
        return
}

const sleep = "sleep"
const write = "write"
```

- Our new test fails if we sleep 3 seconds and rush to print, and succeeds when we revert `Countdown` back to the way it is supposed to work. It would be nice for our `Sleeper` to be configurable in case requirements change. We're going to give the `sleep` field the same function type as `time.Sleep`, `func(time.Duration)`, so we can easily assign it.

```
// 10-mocking/main.go
type ConfigurableSleeper struct {
        duration time.Duration
        sleep    func(time.Duration)
}

func (c *ConfigurableSleeper) Sleep() {
        c.sleep(c.duration)
}

func main() {
        sleeper := &ConfigurableSleeper{1 * time.Second, time.Sleep}
        Countdown(os.Stdout, sleeper)
}

// 10-mocking/countdown_test.go
func TestConfigurableSleeper(t *testing.T) {
        sleepTime := 5 * time.Second

        spyTime := &SpyTime{}
        sleeper := ConfigurableSleeper{sleepTime, spyTime.SetDurationSlept}
        sleeper.Sleep()

        if spyTime.durationSlept != sleepTime {
                t.Errorf("should have slept for %v but slept for %v",
sleepTime, spyTime.durationSlept)
        }
}

type SpyTime struct {
        durationSlept time.Duration
}

func (s *SpyTime) SetDurationSlept(duration time.Duration) {
        s.durationSlept = duration
}
```

- When we get this done, we can delete the `DefaultSleeper` implementation.
- Some people consider mocking evil. This is because people normally get into a bad state when they don't *listen to their tests* and are *not respecting the refactoring stage*. If your mocking code is becoming complicated or you are having to mock out lots of things to test something, you should *listen* to that bad feeling and think about your code. It is usually a sign of:

    - The thing you are testing is having to do too many things because it has too many dependencies to mock
        - Break the module apart so it does less
    - Its dependencies are too fine-grained
        - Think about how you can consolidate some of these dependencies into one meaningful module
    - Your test is too concerned with implementation details
        - Favour testing expected behaviour rather than the implementation
- Normally a lot of mocking points to *bad abstraction* in your code. Well-designed code is easy to test.

- If you make changes and break lots of tests, this usually is a sign that you are testing too much *implementation detail*. Try to make it so your tests are testing *useful behaviour* unless the implementation is really important to how the system runs.

- It is sometimes hard to know *what level* to test exactly but here are some thought processes and rules the author follows:

    - **The definition of refactoring is that the code changes but the behaviour stays the same.** If you have decided to do some refactoring, in theory you should be able to make the commit without any test changes. So when writing a test, ask yourself:
        - Am I testing the behaviour I want, or the implementation details?
        - If I were to refactor this code, would I have to make lots of changes to the tests?
    - Although Go lets you test private functions, I would avoid it, as private functions are implementation detail to support public behaviour. Test the public behaviour. Sandi Metz describes private functions as being "less stable" and you don't want to couple your tests to them.
    - I feel like if a test is working with **more than 3 mocks then it is a red flag** - time for a rethink on the design
    - Use spies with caution. Spies let you see the insides of the algorithms you are writing, which can be very useful, but that means a tighter coupling between your test code and the implementation. **Be sure you actually care about these details if you're going to spy on them.**
- Final code

```go
// 10-mocking/countdown_test.go
package main

import (
	"bytes"
	"reflect"
	"testing"
	"time"
)

func TestCountdown(t *testing.T) {
	t.Run("prints 3 to Go!", func(t *testing.T) {
		buffer := &bytes.Buffer{}
		Countdown(buffer, &SpyCountdownOperations{})

		got := buffer.String()
		want := `3
2
1
Go!`

		if got != want {
			t.Errorf("got %q want %q", got, want)
		}

	})

	t.Run("sleep before every print", func(t *testing.T) {
		spySleepPrinter := &SpyCountdownOperations{}
		Countdown(spySleepPrinter, spySleepPrinter)

		want := []string{
			write,
			sleep,
			write,
			sleep,
			write,
			sleep,
			write,
		}

		if !reflect.DeepEqual(want, spySleepPrinter.Calls) {
			t.Errorf("wanted calls %v got %v", want,
spySleepPrinter.Calls)
		}
```

```
        })
}

func TestConfigurableSleeper(t *testing.T) {
        sleepTime := 5 * time.Second

        spyTime := &SpyTime{}
        sleeper := ConfigurableSleeper{sleepTime, spyTime.SetDurationSlept}
        sleeper.Sleep()

        if spyTime.durationSlept != sleepTime {
                t.Errorf("should have slept for %v but slept for %v",
sleepTime, spyTime.durationSlept)
        }
}

type SpyCountdownOperations struct {
        Calls []string
}

func (s *SpyCountdownOperations) Sleep() {
        s.Calls = append(s.Calls, sleep)
}

func (s *SpyCountdownOperations) Write(p []byte) (n int, err error) {
        s.Calls = append(s.Calls, write)
        return
}

const sleep = "sleep"
const write = "write"

type SpyTime struct {
        durationSlept time.Duration
}

func (s *SpyTime) SetDurationSlept(duration time.Duration) {
        s.durationSlept = duration
}

// 10-mocking/main.go
package main

import (
        "fmt"
        "io"
```

```
        "os"
        "time"
)

const finalWord = "Go!"
const countdownStart = 3

// Sleeper allows you to put delays.
type Sleeper interface {
        Sleep()
}

// ConfigurableSleeper is an implementation of Sleeper with a defined delay.
type ConfigurableSleeper struct {
        duration time.Duration
        sleep    func(time.Duration)
}

// Sleep will pause execution for the defined Duration.
func (c *ConfigurableSleeper) Sleep() {
        c.sleep(c.duration)
}

func main() {
        sleeper := &ConfigurableSleeper{1 * time.Second, time.Sleep}
        Countdown(os.Stdout, sleeper)
}

// Countdown prints a countdown from 3 to out with a delay between count
// provided by Sleeper.
func Countdown(out io.Writer, sleeper Sleeper) {
        for i := countdownStart; i > 0; i-- {
                fmt.Fprintln(out, i)
                sleeper.Sleep()
        }
        fmt.Fprint(out, finalWord)
}
```

- Wrapping up
    - More on TDD approach
        - When faced with less trivial examples, break the problem down into "thin vertical slices". Try to get to a point where you have *working software backed by tests* as soon as you can, to avoid getting in rabbit holes and taking a "big bang" approach.

        - Once you have some working software it should be easier to *iterate with small steps* until you arrive at the software you need.
        - "When to use iterative development? You should use iterative development only on projects that you want to succeed." --Martin Fowler
    - Mocking
        - **Without mocking important areas of your code will be untested.** In our case we would not be able to test that our code paused between each print, but there are countless other examples. Calling a service that *can* fail? Wanting to test your system in a particular state? It is very hard to test these scenarios without mocking.
        - Without mocks you may have to set up databases and other third party things just to test simple business rules. You're likely to have slow tests, resulting in **slow feedback loops**.
        - By having to spin up a database or a webservice to test something you're likely to have **fragile tests** due to the unreliability of such services.
    - Once a developer learns about mocking it becomes very easy to over-test every single facet of a system in terms of the *way it works* rather than *what it does*. Always be mindful about **the value of your tests** and what impact they would have in future refactoring.
    - We have only covered **Spies**, which are a type of *test double*. Test double is a generic term for any case where you replace a production object for testing purposes. Under test doubles, there are various types like stubs, spies, and indeed mocks!

Chapter 11 - Concurrency

- To tell Go to start a new goroutine we turn a function call into a `go` statement by putting the keyword `go` in front of it: `go doSomething()`. Because the only way to start a goroutine is to put `go` in front of a function call, we often use *anonymous functions* when we want to start a goroutine. An anonymous function literal looks just the same as a normal function declaration, but without a name. An anonymous function can be called when declared by putting `()` after its closing brace. They maintain the lexical scope in which they are defined - all the variables that are available at the point when you declare the anonymous function are also available in the body of the function.

```go
// 11-concurrency/check_websites.go
package concurrency

type WebsiteChecker func(string) bool

func CheckWebsites(wc WebsiteChecker, urls []string) map[string]bool {
	results := make(map[string]bool)

	for _, url := range urls {
		go func() {
			results[url] = wc(url)
		}()
	}

	return results
}

// 11-concurrency/check_websites_test.go
package concurrency

import (
	"reflect"
	"testing"
)

func mockWebsiteChecker(url string) bool {
	return url != "waat://furhurterwe.geds"
}

func TestCheckWebsites(t *testing.T) {
	websites := []string{
		"http://google.com",
		"http://blog.gypsydave5.com",
		"waat://furhurterwe.geds",
	}

	want := map[string]bool{
		"http://google.com":          true,
		"http://blog.gypsydave5.com": true,
		"waat://furhurterwe.geds":    false,
	}

	got := CheckWebsites(mockWebsiteChecker, websites)

	if !reflect.DeepEqual(want, got) {
```

```
                t.Fatalf("wanted %v, got %v", want, got)
        }
}

// 11-concurrency/check_websites_benchmark_test.go
package concurrency

import (
        "testing"
        "time"
)

func slowStubWebsiteChecker(_ string) bool {
        time.Sleep(20 * time.Millisecond)
        return true
}

func BenchmarkCheckWebsites(b *testing.B) {
        urls := make([]string, 100)
        for i := 0; i < len(urls); i++ {
                urls[i] = "a url"
        }

        for b.Loop() {
                CheckWebsites(slowStubWebsiteChecker, urls)
        }
}
```

- Go maps are not safe for concurrent writes, so sometimes our tests crash and our benchmark always crashes. This is a *data race*. You can use Go's race detector by running `go test -race`. We can solve this by coordinating using channels.
- *Channels* are a Go data structure that can both receive and send values. They allow communication between different goroutines. In this case, we want to communicate between the parent goroutine and each of the goroutines that it makes to do the work of running the `WebsiteChecker` function with the url.
- Note if your project's `go.mod` declares a Go version *older* than 1.22, you will need to explicitly pass `url string` into your anonymous func and call it with `(url)` instead of `()`, because before 1.22 every iteration shared one `url` variable.
- We made the fields of our `result` struct anonymous in the fixed code below.

```go
// 11-concurrency/check_websites.go
package concurrency

// WebsiteChecker checks a url, returning a bool.
type WebsiteChecker func(string) bool
type result struct {
        string
        bool
}

// CheckWebsites takes a WebsiteChecker and a slice of urls and returns a map
// of urls to the result of checking each url with the WebsiteChecker
// function.
func CheckWebsites(wc WebsiteChecker, urls []string) map[string]bool {
        results := make(map[string]bool)
        resultChannel := make(chan result)

        for _, url := range urls {
                go func() {
                        resultChannel <- result{url, wc(url)}
                }()
        }

        for i := 0; i < len(urls); i++ {
                r := <-resultChannel
                results[r.string] = r.bool
        }

        return results
}
```

- Instead of writing the results directly into the map, we are using a *send statement* to send a `result` struct for each call to `wc` on the `resultChannel`.

```go
// Send statement
resultChannel <- result{url, wc(url)}
```

- This uses the `<-` operator, taking a channel on the left and a value on the right. The next `for` loop iterates once for each of the urls. Inside we're using a *receive expression*, which assigns a value received from a channel to a variable, which we then use to update our initial map. This also uses the `<-` operator, but with the two operands reversed: the channel is now on the right and the variable we are assigning to is on the left:

```
// Receive expression
r := <-resultChannel
```

- By using the channel we can control the timing of each write into the `results` map, ensuring that it happens one at a time.
- Wrapping up
  - We have done one large refactor of the `CheckWebsites` function; the inputs and outputs never changed; it just got faster. The tests we had in place allowed us to maintain confidence the software was still working and the benchmark displayed it had gotten about 100x faster.
  - In making it faster we learned about
    - *goroutines*, the basic unit of concurrency in Go, which let us manage more than one website check request.
    - *anonymous functions*, which we used to start each of the concurrent processes that check websites.
    - *channels*, to help organize and control the communication between the different processes, allowing us to avoid a *race condition* bug.
    - *the race detector* which helped us debug problems with concurrent code
  - Make it work, make it right, make it fast
    - Where 'work' is making the tests pass, 'right' is refactoring the code, and 'fast' is optimizing the code to make it run quickly. We can only 'make it fast' once we've made it work and made it right. We should never try to 'make it fast' before the other two steps have been performed.

Chapter 12 - Select

- We have been asked to make a function called `WebsiteRacer` which takes two URLs and "races" them by hitting them with an HTTP GET request and returning the URL which returned first. If neither of them returns within 10 seconds then it should return an `error`. For this we will be using:
  - `net/http` to make the HTTP calls.
  - `net/http/httptest` to help us test them.
  - goroutines
  - `select` to synchronize goroutines.

```go
// 12-select/racer_test.go
package racer

import "testing"

func TestRacer(t *testing.T) {
	slowURL := "http://www.facebook.com"
	fastURL := "http://www.quii.dev"

	want := fastURL
	got := Racer(slowURL, fastURL)

	if got != want {
		t.Errorf("got %q, want %q", got, want)
	}
}

// 12-select/racer.go
package racer

import (
	"net/http"
	"time"
)

func Racer(a, b string) (winner string) {
	startA := time.Now()
	resp, err := http.Get(a)
	if err == nil {
		resp.Body.Close()
	}
	aDuration := time.Since(startA)

	startB := time.Now()
	resp, err = http.Get(b)
	if err == nil {
		resp.Body.Close()
	}
	bDuration := time.Since(startB)

	if aDuration < bDuration {
		return a
	}

	return b
```

```
}
```

- For each URL:
  - `time.Now()` records the time just before we try to get the URL.
  - We use `http.Get` to try to perform an HTTP GET request against the URL. It returns an `http.Response` and an `error`. We don't care about either right now, but if there's no error we close the response body (see below).
  - `time.Since` takes the start time and returns a `time.Duration` of the difference.
- The test may or may not pass right now. We're at the mercy of real websites and real HTTP requests. But the standard library includes `net/http/httptest` which lets us easily create a mock HTTP server.

```go
// 12-select/racer_test.go
package racer

import (
        "net/http"
        "net/http/httptest"
        "testing"
        "time"
)

func TestRacer(t *testing.T) {
        slowServer := httptest.NewServer(
                http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
{
                        time.Sleep(20 * time.Millisecond)
                        w.WriteHeader(http.StatusOK)
                }))
        fastServer := httptest.NewServer(
                http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
{
                        w.WriteHeader(http.StatusOK)
                }))

        slowURL := slowServer.URL
        fastURL := fastServer.URL

        want := fastURL
        got := Racer(slowURL, fastURL)

        if got != want {
                t.Errorf("got %q, want %q", got, want)
        }

        slowServer.Close()
        fastServer.Close()
}
```

- `httptest.NewServer` takes an `http.HandlerFunc`, which we are defining as an anonymous function. `http.HandlerFunc` is a type that looks like this: `type HandlerFunc func(ResponseWriter, *Request)`. This is how you would write a real HTTP server in Go as well; there is no magic. The only difference is that it is wrapped in `httptest.NewServer`, which makes it easier to use with testing as it finds an open port to listen on and then you can close it when you're done.

- We did a bit of refactoring to keep our code DRY:

```go
// 12-select/racer_test.go
package racer

import (
	"net/http"
	"net/http/httptest"
	"testing"
	"time"
)

func TestRacer(t *testing.T) {

	slowServer := makeDelayedServer(20 * time.Millisecond)
	fastServer := makeDelayedServer(0 * time.Millisecond)

	defer slowServer.Close()
	defer fastServer.Close()

	slowURL := slowServer.URL
	fastURL := fastServer.URL

	want := fastURL
	got := Racer(slowURL, fastURL)

	if got != want {
		t.Errorf("got %q, want %q", got, want)
	}
}

func makeDelayedServer(delay time.Duration) *httptest.Server {
	return httptest.NewServer(
		http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
{
			time.Sleep(delay)
			w.WriteHeader(http.StatusOK)
		}))
}

// 12-select/racer.go
package racer

import (
	"net/http"
	"time"
)
```

```
func Racer(a, b string) (winner string) {
        aDuration := measureResponseTime(a)
        bDuration := measureResponseTime(b)

        if aDuration < bDuration {
                return a
        }

        return b
}

func measureResponseTime(url string) time.Duration {
        start := time.Now()
        resp, err := http.Get(url)
        if err == nil {
                resp.Body.Close()
        }
        return time.Since(start)
}
```

- We also introduce a new keyword: `defer`. By prefixing a function call with `defer` it will now call that function at the end of the containing function. It's great for cleaning up resources, such as closing a file or in our case closing a server. You want to execute it at the end of the function but keep the instruction near where you created the server for the benefit of readers.
- We just learned concurrency, so why are we testing sites one after the other? Also, we don't really care about the exact response times, just which is faster. But we're going to do more than the concurrency we learned last chapter. We will use `select`, which helps us synchronize goroutines really easily and clearly.

```go
// 12-select/racer.go
package racer

import "net/http"

func Racer(a, b string) (winner string) {
        select {
        case <-ping(a):
                return a
        case <-ping(b):
                return b
        }
}

func ping(url string) chan struct{} {
        ch := make(chan struct{})
        go func() {
                resp, err := http.Get(url)
                if err == nil {
                        resp.Body.Close()
                }
                close(ch)
        }()
        return ch
}
```

- And we've completely transformed the program! We defined `ping` which creates a `chan struct{}` and returns it. In our case, we don't care what type is sent to the channel, *we just want to signal we are done* and closing the channel works perfectly. Why `struct{}` and not another type like `bool`? Well, a `chan struct{}` is the smallest data type available from a memory perspective, so we get no allocation versus a `bool`. Since we are closing and not sending anything, why allocate anything? Inside the same function we start a goroutine which will signal on that channel once we have completed `http.Get(url)`. My version also closes the response body straight away, since we only care that the request finished, and leaving it open would leak connections. (My copy of the book calls a bare `http.Get(url)` and never closes the body; the book repo's current text and code close it.)

- Always `make` channels. When you use `var` the variable will be initialized to the "zero" value of the type: `""` for strings, `0` for ints, etc. For channels the zero value is `nil`, and sending to or receiving from a `nil` channel with `<-` blocks forever. So `make` channels.

- When you wait for a value on a channel with `myVar := <-ch`, it is a blocking call as you wait for a value. `select` allows you to wait on multiple channels. The first one to send a value (or be closed) "wins" and the code underneath its `case` is executed. `ping` sets up a channel for each of our URLs. Whichever is closed first will have its code executed in the `select`, which results in its URL being returned.
- Our final requirement is to time out if both sites take longer than 10 seconds.

```
// 12-select/racer_test.go
        t.Run("returns an error if a server doesn't respond within 10s",
                func(t *testing.T) {
                        serverA := makeDelayedServer(11 * time.Second)
                        serverB := makeDelayedServer(12 * time.Second)

                        defer serverA.Close()
                        defer serverB.Close()

                        _, err := Racer(serverA.URL, serverB.URL)

                        if err == nil {
                                t.Error("expected an error but didn't get
one")
                        }
                })
}

// 12-select/racer.go
func Racer(a, b string) (winner string, err error) {
        select {
        case <-ping(a):
                return a, nil
        case <-ping(b):
                return b, nil
        }
}
```

- Now our test fails, so we actually need to return an error.

```
// 12-select/racer.go
func Racer(a, b string) (winner string, err error) {
        select {
        case <-ping(a):
                return a, nil
        case <-ping(b):
                return b, nil
        case <-time.After(10 * time.Second):
                return "", fmt.Errorf("timed out waiting for %s and %s", a,
b)
        }
}
```

- `time.After` is very handy when using `select` so you don't block forever on channels. It returns a `chan` and sends a signal down it after the amount of time you define. The main problem now is the test takes 10 seconds to run. We can make this better by making the timeout configurable. That way in our test we can have a short timeout but in the real world it can be 10 seconds. What we really want is a `Racer` with a 10 second timeout, and a `ConfigurableRacer` we can pass a timeout to for testing.
- Now our tests are fast and complete. Final code:

```
// 12-select/racer_test.go
package racer

import (
        "net/http"
        "net/http/httptest"
        "testing"
        "time"
)

func TestRacer(t *testing.T) {

        t.Run("compares speeds of servers, returning the url of the fastest
one", func(t *testing.T) {

                slowServer := makeDelayedServer(20 * time.Millisecond)
                fastServer := makeDelayedServer(0 * time.Millisecond)

                defer slowServer.Close()
                defer fastServer.Close()

                slowURL := slowServer.URL
                fastURL := fastServer.URL

                want := fastURL
                got, err := Racer(slowURL, fastURL)

                if err != nil {
                        t.Fatalf("did not expect an error but got one %v",
err)
                }

                if got != want {
                        t.Errorf("got %q, want %q", got, want)
                }
        })

        t.Run("returns an error if a server doesn't respond within the
specified time",
                func(t *testing.T) {
                        server := makeDelayedServer(25 * time.Millisecond)

                        defer server.Close()
```

```
                        _, err := ConfigurableRacer(server.URL, server.URL,
20*time.Millisecond)

                        if err == nil {
                                t.Error("expected an error but didn't get
one")
                        }
                })
}

func makeDelayedServer(delay time.Duration) *httptest.Server {
        return httptest.NewServer(
                http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
{
                        time.Sleep(delay)
                        w.WriteHeader(http.StatusOK)
                }))
}

// 12-select/racer.go
package racer

import (
        "fmt"
        "net/http"
        "time"
)

var tenSecondTimeout = 10 * time.Second

// Racer compares the response times of a and b, returning the fastest one,
// timing out after 10s.
func Racer(a, b string) (winner string, err error) {
        return ConfigurableRacer(a, b, tenSecondTimeout)
}

// ConfigurableRacer compares the response times of a and b, returning the
// fastest one.
func ConfigurableRacer(a, b string, timeout time.Duration) (winner string,
err error) {
        select {
        case <-ping(a):
                return a, nil
        case <-ping(b):
                return b, nil
        case <-time.After(timeout):
```

```
                return "", fmt.Errorf("timed out waiting for %s and %s", a,
b)
        }
}

func ping(url string) chan struct{} {
        ch := make(chan struct{})
        go func() {
                resp, err := http.Get(url)
                if err == nil {
                        resp.Body.Close()
                }
                close(ch)
        }()
        return ch
}
```

- Wrapping up
  - `select`
    - Helps you wait on multiple channels.
    - Sometimes you'll want to include `time.After` in one of your cases to prevent your system blocking forever.
  - `httptest`
    - A convenient way of creating test servers so you can have reliable and controllable tests.
    - Uses the same interfaces as the "real" `net/http` servers which is consistent and less for you to learn.

Chapter 13 - Reflection

- Golang challenge: write a function `walk(x interface{}, fn func(string))` which takes a struct `x` and calls `fn` for all string fields found inside. Difficulty level: recursively.
- To do this we will need to use *reflection*.
- *Reflection* in computing is the ability of a program to examine its own structure, particularly through types; it's a form of metaprogramming. It's also a great source of confusion.
- What is `interface{}`?

    - The type `interface{}` lets us get around needing to know a type at compile time. You can think of it as just *any* type. In fact, in Go `any` is an alias for `interface{}`. So `walk(x interface{}, fn func(string))` will accept any value for `x`.
    - So why not use `interface{}` for everything and have flexible functions?
        - As a user of a function that takes `interface{}` you lose type safety. What if you meant to pass `Herd.species` of type `string` into a function but instead did `Herd.count` which is an `int`? The compiler won't be able to inform you of your mistake. You also have no idea *what* you're allowed to pass to a function. Knowing that a function takes a `UserService` for instance is very useful.
        - As a writer of such a function, you have to be able to inspect *anything* that has been passed to you and try to figure out what the type is and what you can do with it. This is done using *reflection*. This can be quite clumsy and difficult to read and is generally less performant as you have to do checks at runtime.
- In short, only use reflection if you really need to. If you want polymorphic functions, consider designing them around an interface so that users can use your function with multiple types if they implement the methods you need for your function to work.

```
// 13-reflection/reflection_test.go
package reflection

import "testing"

func TestWalk(t *testing.T) {

        expected := "Chris"
        var got []string

        x := struct {
                Name string
        }{expected}

        walk(x, func(input string) {
                got = append(got, input)
        })

        if len(got) != 1 {
                t.Errorf("wrong number of function calls, got %d want %d",
len(got), 1)
        }

        if got[0] != expected {
                t.Errorf("got %q, want %q", got[0], expected)
        }
}

// 13-reflection/reflection.go
package reflection

func walk(x interface{}, fn func(input string)) {
        fn("dummy val!")
}
```

- We want to store a slice of strings (`got`) which stores which strings were passed into `fn` by walk. Often in previous chapters, we have made dedicated types for this to spy on function/method invocations but in this case, we can just pass in an anonymous function for `fn` that closes over `got`.
- We use an anonymous `struct` with a `Name` field of type string to go for the simplest "happy" path.

- Finally, call `walk` with `x` and the spy and for now check the length of `got` and after we get something basic working test the value passed in.

```
// 13-reflection/reflection.go
package reflection

import "reflect"

func walk(x interface{}, fn func(input string)) {
        val := reflect.ValueOf(x)
        field := val.Field(0)
        fn(field.String())
}
```

- This code is very unsafe and very naive, but when we are failing tests we want to write the smallest amount of code possible to pass. Then we write more tests to address our concerns. The `reflect` package has a function `ValueOf` which returns a `Value` of a given variable. This has ways for us to inspect a value, including its fields, which we use on the next line. We then make some optimistic assumptions about the value passed in:
    - We look at the first and only field. However if there were no fields at all this would cause a panic.
    - We then call `String()`, which returns the underlying value as a string. This would be wrong if the field was something other than a string.
- We plan to pass a variety of values into our function to show its shortcomings, so we will refactor our test into a table-driven test.

```go
// 13-reflection/reflection_test.go
package reflection

import (
        "reflect"
        "testing"
)

func TestWalk(t *testing.T) {

        cases := []struct {
                Name          string
                Input         interface{}
                ExpectedCalls []string
        }{
                {
                        "struct with one string field",
                        struct {
                                Name string
                        }{"Chris"},
                        []string{"Chris"},
                },
        }

        for _, test := range cases {
                t.Run(test.Name, func(t *testing.T) {
                        var got []string
                        walk(test.Input, func(input string) {
                                got = append(got, input)
                        })

                        if !reflect.DeepEqual(got, test.ExpectedCalls) {
                                t.Errorf("got %v, want %v", got,
test.ExpectedCalls)
                        }
                })
        }
}
```

- So our one passing test is now in a table. Let's add another and go red.

```
// 13-reflection/reflection_test.go
                {
                        "struct with two string fields",
                        struct {
                                Name string
                                City string
                        }{"Chris", "London"},
                        []string{"Chris", "London"},
                },

// 13-reflection/reflection.go
func walk(x interface{}, fn func(input string)) {
        val := reflect.ValueOf(x)

        for i := 0; i < val.NumField(); i++ {
                field := val.Field(i)
                fn(field.String())
        }
}
```

- `val` has a method `NumField` which returns the number of fields in the value.

```
// 13-reflection/reflection_test.go
                {
                        "struct with non string fields",
                        struct {
                                Name string
                                City int
                        }{"Chris", 33},
                        []string{"Chris"},
                },

// 13-reflection/reflection.go
        for i := 0; i < val.NumField(); i++ {
                field := val.Field(i)

                if field.Kind() == reflect.String {
                        fn(field.String())
                }
        }
```

- We only want to pass strings to `fn`, so we can check `field.Kind()`.

```
// 13-reflection/reflection_test.go
                {
                        "nested fields",
                        Person{
                                "Chris",
                                Profile{33, "London"},
                        },
                        []string{"Chris", "London"},
                },
// ...
type Person struct {
        Name    string
        Profile Profile
}

type Profile struct {
        Age  int
        City string
}

// 13-reflection/reflection.go
// ...
        for i := 0; i < val.NumField(); i++ {
                field := val.Field(i)

                if field.Kind() == reflect.String {
                        fn(field.String())
                }

                if field.Kind() == reflect.Struct {
                        walk(field.Interface(), fn)
                }
        }
```

- Ok, so now we check if `field.Kind() == reflect.Struct` and then recurse. But we're starting to need to work with a variety of `Kind`, so let's make it a `switch`. When you're doing a comparison on the same value more than once *generally* refactoring into a `switch` will improve readability and make your code easier to extend:

```
// 13-reflection/reflection.go
        for i := 0; i < val.NumField(); i++ {
                field := val.Field(i)

                switch field.Kind() {
                case reflect.String:
                        fn(field.String())
                case reflect.Struct:
                        walk(field.Interface(), fn)
                }
        }
```

- Another test case to handle: pointers to things:

```
// 13-reflection/reflection_test.go
                {
                        "pointers to things",
                        &Person{
                                "Chris",
                                Profile{33, "London"},
                        },
                        []string{"Chris", "London"},
                },
```

- You can't use `NumField` on a pointer `Value`; you need to extract the underlying value using `Elem()` before you can do that.

```go
// 13-reflection/reflection.go
func walk(x interface{}, fn func(input string)) {
        val := getValue(x)

        for i := 0; i < val.NumField(); i++ {
                field := val.Field(i)

                switch field.Kind() {
                case reflect.String:
                        fn(field.String())
                case reflect.Struct:
                        walk(field.Interface(), fn)
                }
        }
}

func getValue(x interface{}) reflect.Value {
        val := reflect.ValueOf(x)

        if val.Kind() == reflect.Pointer {
                val = val.Elem()
        }

        return val
}
```

- Now we've extracted getting the `reflect.Value` into its own function, separate from iterating over the fields and doing whatever needs to be done depending on type.

```
// 13-reflection/reflection_test.go
                {
                        "slices",
                        []Profile{
                                Profile{33, "London"},
                                Profile{34, "Reykjavik"},
                        },
                        []string{"London", "Reykjavik"},
                },

// 13-reflection/reflection.go
func walk(x interface{}, fn func(input string)) {
        val := getValue(x)

        if val.Kind() == reflect.Slice {
                for i := 0; i < val.Len(); i++ {
                        walk(val.Index(i).Interface(), fn)
                }
                return
        }
// ...
```

- This gets our tests passing but it's ugly. We have to check if our thing is a slice at the top and recurse then return if so. Let's refactor while our tests are green. If you think a little abstractly, we want to call `walk` on either
  - Each field in a struct
  - Each *thing* in a slice
- Let's rework the code to check the type first and then do our work

```
// 13-reflection/reflection.go
func walk(x interface{}, fn func(input string)) {
        val := getValue(x)

        numberOfValues := 0
        var getField func(int) reflect.Value

        switch val.Kind() {
        case reflect.String:
                fn(val.String())
        case reflect.Struct:
                numberOfValues = val.NumField()
                getField = val.Field
        case reflect.Slice:
                numberOfValues = val.Len()
                getField = val.Index
        }

        for i := 0; i < numberOfValues; i++ {
                walk(getField(i).Interface(), fn)
        }
}
```

- If the `value` is a `reflect.String` then we just call `fn`. Otherwise, our `switch` will extract out two things depending on the type:
    - How many fields there are
    - How to extract the `Value` (`Field` or `Index`)
- Once we've determined those things we can iterate through `numberOfValues` calling `walk` with the result of the `getField` function.

```
// 13-reflection/reflection_test.go
                {
                        "arrays",
                        [2]Profile{
                                Profile{33, "London"},
                                Profile{34, "Reykjavik"},
                        },
                        []string{"London", "Reykjavik"},
                },

// 13-reflection/reflection.go
        case reflect.Slice, reflect.Array:
                numberOfValues = val.Len()
                getField = val.Index
```

- A `case` can match multiple values if you separate them by `,`, and we can handle an `Array` the same as a `Slice`.

```
// 13-reflection/reflection_test.go
                {
                        "map",
                        map[string]string{
                                "Cow":   "Moo",
                                "Sheep": "Baa",
                        },
                        []string{"Moo", "Baa"},
                },

// 13-reflection/reflection.go
        case reflect.Map:
                for _, key := range val.MapKeys() {
                        walk(val.MapIndex(key).Interface(), fn)
                }
```

- A map is similar to a struct but you cannot get values out by index. We're going to partially undo our last refactor since this abstraction isn't as useful as we thought it would be:

```go
// 13-reflection/reflection.go
func walk(x interface{}, fn func(input string)) {
	val := getValue(x)

	walkValue := func(value reflect.Value) {
		walk(value.Interface(), fn)
	}

	switch val.Kind() {
	case reflect.String:
		fn(val.String())
	case reflect.Struct:
		for i := 0; i < val.NumField(); i++ {
			walkValue(val.Field(i))
		}
	case reflect.Slice, reflect.Array:
		for i := 0; i < val.Len(); i++ {
			walkValue(val.Index(i))
		}
	case reflect.Map:
		for _, key := range val.MapKeys() {
			walk(val.MapIndex(key).Interface(), fn)
		}
	}
}
```

- We have one final problem. Maps do not guarantee order, so our test might fail. We're going to make a separate test for maps that doesn't expect order.

```
// 13-reflection/reflection_test.go
        t.Run("with maps", func(t *testing.T) {
                aMap := map[string]string{
                        "Cow":   "Moo",
                        "Sheep": "Baa",
                }

                var got []string
                walk(aMap, func(input string) {
                        got = append(got, input)
                })

                assertLength(t, got, len(aMap))
                assertContains(t, got, "Moo")
                assertContains(t, got, "Baa")
        })
}

func assertContains(t testing.TB, haystack []string, needle string) {
        t.Helper()
        contains := false
        for _, x := range haystack {
                if x == needle {
                        contains = true
                }
        }
        if !contains {
                t.Errorf("expected %v to contain %q but it didn't", haystack,
needle)
        }
}

func assertLength(t testing.TB, got []string, want int) {
        t.Helper()
        if len(got) != want {
                t.Errorf("got %d values but expected %d", len(got), want)
        }
}
```

- And channels:

```go
// 13-reflection/reflection_test.go
        t.Run("with channels", func(t *testing.T) {
                aChannel := make(chan Profile)

                go func() {
                        aChannel <- Profile{33, "Berlin"}
                        aChannel <- Profile{34, "Katowice"}
                        close(aChannel)
                }()

                var got []string
                want := []string{"Berlin", "Katowice"}

                walk(aChannel, func(input string) {
                        got = append(got, input)
                })

                if !reflect.DeepEqual(got, want) {
                        t.Errorf("got %v, want %v", got, want)
                }
        })

// 13-reflection/reflection.go
        case reflect.Chan:
                for {
                        if v, ok := val.Recv(); ok {
                                walkValue(v)
                        } else {
                                break
                        }
                }
        }
```

- And last but not least, functions!

```
// 13-reflection/reflection_test.go
        t.Run("with function", func(t *testing.T) {
                aFunction := func() (Profile, Profile) {
                        return Profile{33, "Berlin"}, Profile{34, "Katowice"}
                }

                var got []string
                want := []string{"Berlin", "Katowice"}

                walk(aFunction, func(input string) {
                        got = append(got, input)
                })

                if !reflect.DeepEqual(got, want) {
                        t.Errorf("got %v, want %v", got, want)
                }
        })

// Functions that take arguments don't make much sense in this scenario, but
// we should allow for arbitrary return values and `walk` them:
// 13-reflection/reflection.go
// We also tightened this `Chan` `for` loop
        case reflect.Chan:
                for v, ok := val.Recv(); ok; v, ok = val.Recv() {
                        walkValue(v)
                }
        case reflect.Func:
                valFnResult := val.Call(nil)
                for _, res := range valFnResult {
                        walkValue(res)
                }
```

- And that was it. `func` was last, and now we've solved the original problem.
- Final code:

```go
// 13-reflection/reflection_test.go
package reflection

import (
	"reflect"
	"testing"
)

func TestWalk(t *testing.T) {

	cases := []struct {
		Name          string
		Input         interface{}
		ExpectedCalls []string
	}{
		{
			"struct with one string field",
			struct {
				Name string
			}{"Chris"},
			[]string{"Chris"},
		},
		{
			"struct with two string fields",
			struct {
				Name string
				City string
			}{"Chris", "London"},
			[]string{"Chris", "London"},
		},
		{
			"struct with non string fields",
			struct {
				Name string
				City int
			}{"Chris", 33},
			[]string{"Chris"},
		},
		{
			"nested fields",
			Person{
				"Chris",
				Profile{33, "London"},
			},
			[]string{"Chris", "London"},
```

```
                },
                {
                        "pointers to things",
                        &Person{
                                "Chris",
                                Profile{33, "London"},
                        },
                        []string{"Chris", "London"},
                },
                {
                        "slices",
                        []Profile{
                                Profile{33, "London"},
                                Profile{34, "Reykjavik"},
                        },
                        []string{"London", "Reykjavik"},
                },
                {
                        "arrays",
                        [2]Profile{
                                Profile{33, "London"},
                                Profile{34, "Reykjavik"},
                        },
                        []string{"London", "Reykjavik"},
                },
        }

        for _, test := range cases {
                t.Run(test.Name, func(t *testing.T) {
                        var got []string
                        walk(test.Input, func(input string) {
                                got = append(got, input)
                        })

                        if !reflect.DeepEqual(got, test.ExpectedCalls) {
                                t.Errorf("got %v, want %v", got,
test.ExpectedCalls)
                        }
                })
        }

        t.Run("with maps", func(t *testing.T) {
                aMap := map[string]string{
                        "Cow":   "Moo",
                        "Sheep": "Baa",
                }
```

```
		var got []string
		walk(aMap, func(input string) {
			got = append(got, input)
		})

		assertLength(t, got, len(aMap))
		assertContains(t, got, "Moo")
		assertContains(t, got, "Baa")
	})

	t.Run("with channels", func(t *testing.T) {
		aChannel := make(chan Profile)

		go func() {
			aChannel <- Profile{33, "Berlin"}
			aChannel <- Profile{34, "Katowice"}
			close(aChannel)
		}()

		var got []string
		want := []string{"Berlin", "Katowice"}

		walk(aChannel, func(input string) {
			got = append(got, input)
		})

		if !reflect.DeepEqual(got, want) {
			t.Errorf("got %v, want %v", got, want)
		}
	})

	t.Run("with function", func(t *testing.T) {
		aFunction := func() (Profile, Profile) {
			return Profile{33, "Berlin"}, Profile{34, "Katowice"}
		}

		var got []string
		want := []string{"Berlin", "Katowice"}

		walk(aFunction, func(input string) {
			got = append(got, input)
		})

		if !reflect.DeepEqual(got, want) {
			t.Errorf("got %v, want %v", got, want)
```

```
                }
        })
}

func assertContains(t testing.TB, haystack []string, needle string) {
        t.Helper()
        contains := false
        for _, x := range haystack {
                if x == needle {
                        contains = true
                }
        }
        if !contains {
                t.Errorf("expected %v to contain %q but it didn't", haystack,
needle)
        }
}

func assertLength(t testing.TB, got []string, want int) {
        t.Helper()
        if len(got) != want {
                t.Errorf("got %d values but expected %d", len(got), want)
        }
}

type Person struct {
        Name    string
        Profile Profile
}

type Profile struct {
        Age  int
        City string
}

// 13-reflection/reflection.go
package reflection

import "reflect"

func walk(x interface{}, fn func(input string)) {
        val := getValue(x)

        walkValue := func(value reflect.Value) {
                walk(value.Interface(), fn)
        }
```

```
        switch val.Kind() {
        case reflect.String:
                fn(val.String())
        case reflect.Struct:
                for i := 0; i < val.NumField(); i++ {
                        walkValue(val.Field(i))
                }
        case reflect.Slice, reflect.Array:
                for i := 0; i < val.Len(); i++ {
                        walkValue(val.Index(i))
                }
        case reflect.Map:
                for _, key := range val.MapKeys() {
                        walk(val.MapIndex(key).Interface(), fn)
                }
        case reflect.Chan:
                for v, ok := val.Recv(); ok; v, ok = val.Recv() {
                        walkValue(v)
                }
        case reflect.Func:
                valFnResult := val.Call(nil)
                for _, res := range valFnResult {
                        walkValue(res)
                }
        }
}

func getValue(x interface{}) reflect.Value {
        val := reflect.ValueOf(x)

        if val.Kind() == reflect.Pointer {
                val = val.Elem()
        }

        return val
}
```

- Wrapping up
    - Introduced some concepts from the `reflect` package.
    - Used recursion to traverse arbitrary data structures.
    - Did an in retrospect bad refactor but didn't get too upset about it. By working iteratively with tests it's not such a big deal.

    - This only covered a small aspect of reflection. For more, consider: https://go.dev/blog/laws-of-reflection.
    - Now that you know about reflection, do your best to avoid using it.

Chapter 14 - Sync

- We want to make a counter that is safe to use concurrently. We'll start with an unsafe counter and verify its behaviour works in a single-threaded environment. Then we'll demonstrate its unsafeness in a test and fix it.

```
// 14-sync/sync_test.go
package sync

import "testing"

func TestCounter(t *testing.T) {
        t.Run("incrementing the counter 3 times leaves it at 3", func(t
*testing.T) {
                counter := Counter{}
                counter.Inc()
                counter.Inc()
                counter.Inc()

                if counter.Value() != 3 {
                        t.Errorf("got %d, want %d", counter.Value(), 3)
                }
        })
}

// 14-sync/sync.go
package sync

type Counter struct {
        value int
}

func (c *Counter) Inc() {
        c.value++
}

func (c *Counter) Value() int {
        return c.value
}
```

- When we're green, we refactor out an `assertCounter` helper since we plan on doing multiple `counter.Value()` assertions.

```
// 14-sync/sync_test.go
func assertCounter(t testing.TB, got Counter, want int) {
        t.Helper()
        if got.Value() != want {
                t.Errorf("got %d, want %d", got.Value(), want)
        }
}
```

- Now we need to show the problem we want to fix within a test:

```
// 14-sync/sync_test.go
        t.Run("it runs safely concurrently", func(t *testing.T) {
                wantedCount := 1000
                counter := Counter{}

                var wg sync.WaitGroup
                wg.Add(wantedCount)

                for i := 0; i < wantedCount; i++ {
                        go func() {
                                counter.Inc()
                                wg.Done()
                        }()
                }
                wg.Wait()

                assertCounter(t, counter, wantedCount)
        })
```

- This loops through our `wantedCount` and fires off goroutines to call `counter.Inc()`. We are using `sync.WaitGroup`, which is a convenient way to synchronize goroutines.
- A `WaitGroup` waits for a collection of goroutines to finish. The main goroutine calls `Add` to set the number of goroutines to wait for. Then each of the goroutines runs and calls `Done` when finished. At the same time, `Wait` can be used to block until all the goroutines have finished.
- By waiting with `wg.Wait()` before making our assertions, we can be sure all of our goroutines have attempted to `Inc` the `Counter`.
- Our test fails despite waiting on the goroutines to finish. `c.value++` looks like a single, indivisible operation, but it isn't. It is shorthand for something like:

```
tmp := c.value // 1. read
tmp = tmp + 1  // 2. increment
c.value = tmp  // 3. write
```

- Each of those three steps is a separate operation, and the Go runtime is free to switch between goroutines at any point between them. If two goroutines call `Inc` around the same time their steps can interleave:

```
goroutine A: reads c.value (0)
goroutine B: reads c.value (0)
goroutine A: increments its copy to 1
goroutine B: increments its copy to 1
goroutine A: writes c.value = 1
goroutine B: writes c.value = 1
```

- Both goroutines call `Inc` once each so we would want `c.value` to end up as 2, but instead it ends up as 1. One increment was silently lost because both goroutines read the same starting value before either had written the result back. This is called a *race condition*, and with 1000 goroutines all racing to read, increment and write at once, it's no surprise some of the increments got lost.

- A simple solution is to add a lock to our `Counter`, ensuring only one goroutine can increment the counter at a time. A `Mutex` provides such a lock:

```go
// 14-sync/sync.go
package sync

import "sync"

type Counter struct {
        mu    sync.Mutex
        value int
}

func (c *Counter) Inc() {
        c.mu.Lock()
        defer c.mu.Unlock()
        c.value++
}

func (c *Counter) Value() int {
        return c.value
}
```

- This means any goroutine calling `Inc` will acquire the lock on `Counter`, so our test will pass with the expected value.
- But `go vet` now warns that `assertCounter` *passes lock by value*. A `Mutex` must not be copied after first use, and passing a `Counter` by value copies it. So `assertCounter` takes a `*Counter`, and we add a `NewCounter` constructor that returns a pointer so callers never copy a `Counter` (see the final code).
- When to use `&` and `*`: use a pointer when the callee must change the original, when the value must not be copied (like anything holding a `sync.Mutex`), or when "nothing" (`nil`) is a valid value. Otherwise pass values; copying is usually cheap.
    - Maps, channels, and funcs already share their data when passed. Slices share their *elements* but not their length, so a function that `append`s must return the new slice (`s = append(s, x)`).
    - Constructors like `NewCounter` conventionally return `*T` via `&T{}`. Returning the address of a local is safe in Go: escape analysis moves it to the heap.
    - Go auto-dereferences for field access and method calls (`p.value`, not `(*p).value`), and takes `&` for you when calling a pointer-receiver method on a variable, which is why the book rarely writes `&` explicitly.

    - If any method needs a pointer receiver, give all of the type's methods pointer receivers for consistency.
- This book hasn't explained embedding, but as you might recall, you can put a struct within a struct without a field name, and your parent struct gains the methods of the embedded struct directly. We could do this for our `Mutex`:

```
type Counter struct {
        sync.Mutex
        value int
}

func (c *Counter) Inc() {
        c.Lock()
        defer c.Unlock()
        c.value++
}
```

- This looks a bit nicer but is a bad practice. Embedding types means the methods of that type become *part of the public interface*. The moment we make something public is the moment other code can couple itself to it. Exposing `Lock` and `Unlock` is at best confusing but at worst potentially harmful to your software if callers of your type start calling these methods.

- `sync/atomic` also provides types like `atomic.Int64`. Instead of manually locking with a `Mutex`, we could have used an `atomic.Int64` in our `struct` and our code would be threadsafe, but we would need to call `c.value.Add(1)` instead of `c.value++` and `c.value.Load()` to return the value. This is often faster than a `Mutex` for simple use cases. Reach for `sync/atomic`'s types when you're protecting a single value; reach for `Mutex` once you need to keep multiple fields or some invariant between them consistent together.

- Final code, using `atomic.Int64`. The book's snippet has `Value` return the `int64` from `Load()`, but then `assertCounter` (which takes `want int`) won't compile, so this version converts with `int(...)` and keeps the API the same:

```go
// 14-sync/sync_test.go
package sync

import (
	"sync"
	"testing"
)

func TestCounter(t *testing.T) {
	t.Run("incrementing the counter 3 times leaves it at 3", func(t
*testing.T) {
		counter := NewCounter()
		counter.Inc()
		counter.Inc()
		counter.Inc()

		assertCounter(t, counter, 3)
	})

	t.Run("it runs safely concurrently", func(t *testing.T) {
		wantedCount := 1000
		counter := NewCounter()

		var wg sync.WaitGroup
		wg.Add(wantedCount)

		for i := 0; i < wantedCount; i++ {
			go func() {
				counter.Inc()
				wg.Done()
			}()
		}
		wg.Wait()

		assertCounter(t, counter, wantedCount)
	})
}

func assertCounter(t testing.TB, got *Counter, want int) {
	t.Helper()
	if got.Value() != want {
		t.Errorf("got %d, want %d", got.Value(), want)
	}
}
```

```go
// 14-sync/sync.go
package sync

import "sync/atomic"

// Counter will increment a number.
type Counter struct {
        value atomic.Int64
}

// NewCounter returns a new Counter.
func NewCounter() *Counter {
        return &Counter{}
}

// Inc the count.
func (c *Counter) Inc() {
        c.value.Add(1)
}

// Value returns the current count.
func (c *Counter) Value() int {
        return int(c.value.Load())
}
```

- Wrapping up
  - We've covered a few things from the `sync` package
    - `Mutex` allows us to add locks to our data
    - `WaitGroup` is a means of waiting for goroutines to finish jobs
    - `sync/atomic` offers safe, lock-free access to single values, and is worth reaching for when a `Mutex` would be overkill
  - Channels vs locks
    - Use channels when passing ownership of data
    - Use mutexes for managing state
  - Remember to use `go vet` in your build scripts as it can alert you to some subtle bugs in your code before they hit your poor users.
  - Don't use embedding because it's convenient
    - Think about the effect embedding has on your public API.
    - Do you really want to expose these methods and have people coupling their own code to them?

        - With respect to mutexes, this could be potentially disastrous in very unpredictable and weird ways. Imagine some nefarious code unlocking a mutex when it shouldn't be; this would cause some very strange bugs that will be hard to track down.

Chapter 15 - Context

- Alright, basic server, first test for "returns data from store" passes, second test fails with "store was not told to cancel":

```go
// 15-context/context_test.go
package context

import (
        "context"
        "net/http"
        "net/http/httptest"
        "testing"
        "time"
)

type SpyStore struct {
        response  string
        cancelled bool
}

func (s *SpyStore) Fetch() string {
        time.Sleep(100 * time.Millisecond)
        return s.response
}

func (s *SpyStore) Cancel() {
        s.cancelled = true
}

func TestServer(t *testing.T) {
        data := "hello, world"

        t.Run("returns data from store", func(t *testing.T) {
                svr := Server(&SpyStore{response: data})

                request := httptest.NewRequest(http.MethodGet, "/", nil)
                response := httptest.NewRecorder()

                svr.ServeHTTP(response, request)

                if response.Body.String() != data {
                        t.Errorf(`got "%s", want "%s"`,
response.Body.String(), data)
                }
        })

        t.Run("tells store to cancel work if request is cancelled", func(t
*testing.T) {
                store := &SpyStore{response: data}
```

```
                svr := Server(store)

                request := httptest.NewRequest(http.MethodGet, "/", nil)

                cancellingCtx, cancel :=
context.WithCancel(request.Context())
                time.AfterFunc(5*time.Millisecond, cancel)
                request = request.WithContext(cancellingCtx)

                response := httptest.NewRecorder()

                svr.ServeHTTP(response, request)

                if !store.cancelled {
                        t.Error("store was not told to cancel")
                }
        })
}

// 15-context/context.go
package context

import (
        "fmt"
        "net/http"
)

func Server(store Store) http.HandlerFunc {
        return func(w http.ResponseWriter, r *http.Request) {
                fmt.Fprint(w, store.Fetch())
        }
}

type Store interface {
        Fetch() string
        Cancel()
}
```

- The `context` package provides functions to derive new Context values from existing ones. These values form a tree: when a Context is canceled, all Contexts derived from it are also canceled.
- It is important that you derive your contexts so that cancellations are propagated throughout the call stack for a given request. What we do here is derive a new

`cancellingCtx` from our `request` which returns us a `cancel` function. We then schedule that function to be called in 5 milliseconds by using `time.AfterFunc`. Finally we use this new context in our request by calling `request.WithContext`.

- It's as easy to get this test to pass as doing this:

```
// 15-context/context.go
func Server(store Store) http.HandlerFunc {
        return func(w http.ResponseWriter, r *http.Request) {
                store.Cancel()
                fmt.Fprint(w, store.Fetch())
        }
}
```

- But surely we don't want to cancel before making any request! So let's modify our "returns data from store" test to make sure it isn't cancelled.

```
// 15-context/context_test.go
        t.Run("returns data from store", func(t *testing.T) {
                store := &SpyStore{response: data}
                svr := Server(store)

                request := httptest.NewRequest(http.MethodGet, "/", nil)
                response := httptest.NewRecorder()

                svr.ServeHTTP(response, request)

                if response.Body.String() != data {
                        t.Errorf(`got "%s", want "%s"`,
response.Body.String(), data)
                }

                if store.cancelled {
                        t.Error("it should not have cancelled the store")
                }
        })
```

- Now we have a failing test to force a proper implementation.

```
// 15-context/context.go
func Server(store Store) http.HandlerFunc {
        return func(w http.ResponseWriter, r *http.Request) {
                ctx := r.Context()

                data := make(chan string, 1)

                go func() {
                        data <- store.Fetch()
                }()

                select {
                case d := <-data:
                        fmt.Fprint(w, d)
                case <-ctx.Done():
                        store.Cancel()
                }
        }
}
```

- A `Context` has a method `Done()` that returns a channel which is closed when the context is "done" or "cancelled". We want to listen to that signal and call `store.Cancel` if we get it, but we want to ignore it if our `Store` manages to `Fetch` before it. To manage this, we run `Fetch` in a goroutine where it will write the result into a new channel `data`. We then use `select` to effectively race the two asynchronous operations, and then we either write a response or `Cancel`.
- While the tests are passing, we refactor our spy to take a `*testing.T` and make its own assertions. We also modify the tests to initialize the spy with `t`, `store := &SpyStore{response: data, t: t}`, and use the new assert methods:

```
// 15-context/context_test.go
type SpyStore struct {
        response  string
        cancelled bool
        t         *testing.T
}

func (s *SpyStore) assertWasCancelled() {
        s.t.Helper()
        if !s.cancelled {
                s.t.Error("store was not told to cancel")
        }
}

func (s *SpyStore) assertWasNotCancelled() {
        s.t.Helper()
        if s.cancelled {
                s.t.Error("store was told to cancel")
        }
}
```

- So we're making the spy be in control of checking for cancellation. This is because cancellation is primarily the concern of our `Store`, which it mocks.
- Incoming requests to a server should create a Context, and outgoing calls to servers should accept a Context. The chain of function calls between them must propagate the Context, optionally replacing it with a derived Context created using `WithCancel`, `WithDeadline`, `WithTimeout`, or `WithValue`. When a Context is canceled, all Contexts derived from it are also canceled. -- from the Go docs
- From the Go blog: "At Google, we require that Go programmers pass a Context parameter as the first argument to every function on the call path between incoming and outgoing requests. This allows Go code developed by many different teams to interoperate well. It provides simple control over timeouts and cancellation and ensures that critical values like security credentials transit Go programs properly."
- So the idea is that the `Store` is responsible for handling the `Context` and cancellation. If it has any dependents, it will be responsible for passing the `Context` to them so they too can be responsible for stopping themselves.
- This is a pretty big refactor! The only thing our handler is responsible for now is making sure it sends a context through to the downstream `Store` and that it handles the error that

will come from the `Store` when it is canceled. We also remove the spy's `cancelled` field, its assert methods, and `Cancel` from the `Store` interface, and `Fetch` now takes a `Context` and returns an `error` when canceled. Along the way, we need to define a new `SpyResponseWriter` that implements `http.ResponseWriter`, since `httptest.ResponseRecorder` doesn't have a way to verify that we do not get any kind of response in the error case.

- In the final code you can see the server code has become simpler, as it is no longer explicitly responsible for cancellation; it simply passes through the `context` and relies on the downstream functions to respect any cancellations that may occur. You can see in our new spy too that we use two `select` statements, as we are simulating a process that builds a `response` slowly, one character at a time. The inner `select` logs and `return`s as soon as the context is done, so the spy stops working once it's cancelled.

- Final code:

```go
// 15-context/context_test.go
package context

import (
        "context"
        "errors"
        "log"
        "net/http"
        "net/http/httptest"
        "testing"
        "time"
)

// SpyStore allows you to simulate a store and see how it's used.
type SpyStore struct {
        response string
}

// Fetch returns response after a short delay.
func (s *SpyStore) Fetch(ctx context.Context) (string, error) {
        data := make(chan string, 1)

        go func() {
                var result string
                for _, c := range s.response {
                        select {
                        case <-ctx.Done():
                                log.Println("spy store got cancelled")
                                return
                        default:
                                time.Sleep(10 * time.Millisecond)
                                result += string(c)
                        }
                }
                data <- result
        }()

        select {
        case <-ctx.Done():
                return "", ctx.Err()
        case res := <-data:
                return res, nil
        }
}
```

```
// SpyResponseWriter checks whether a response has been written.
type SpyResponseWriter struct {
        written bool
}

// Header will mark written to true.
func (s *SpyResponseWriter) Header() http.Header {
        s.written = true
        return nil
}

// Write will mark written to true.
func (s *SpyResponseWriter) Write([]byte) (int, error) {
        s.written = true
        return 0, errors.New("not implemented")
}

// WriteHeader will mark written to true.
func (s *SpyResponseWriter) WriteHeader(statusCode int) {
        s.written = true
}

func TestServer(t *testing.T) {
        data := "hello, world"

        t.Run("returns data from store", func(t *testing.T) {
                store := &SpyStore{response: data}
                svr := Server(store)

                request := httptest.NewRequest(http.MethodGet, "/", nil)
                response := httptest.NewRecorder()

                svr.ServeHTTP(response, request)

                if response.Body.String() != data {
                        t.Errorf(`got "%s", want "%s"`,
response.Body.String(), data)
                }
        })

        t.Run("tells store to cancel work if request is cancelled", func(t
*testing.T) {
                store := &SpyStore{response: data}
                svr := Server(store)

                request := httptest.NewRequest(http.MethodGet, "/", nil)
```

```
                cancellingCtx, cancel :=
context.WithCancel(request.Context())
                time.AfterFunc(5*time.Millisecond, cancel)
                request = request.WithContext(cancellingCtx)

                response := &SpyResponseWriter{}

                svr.ServeHTTP(response, request)

                if response.written {
                        t.Error("a response should not have been written")
                }
        })

}

// 15-context/context.go
package context

import (
        "context"
        "fmt"
        "net/http"
)

// Server returns a handler for calling Store.
func Server(store Store) http.HandlerFunc {
        return func(w http.ResponseWriter, r *http.Request) {
                data, err := store.Fetch(r.Context())

                if err != nil {
                        return // todo: log error however you like
                }

                fmt.Fprint(w, data)
        }
}

// Store fetches data.
type Store interface {
        Fetch(ctx context.Context) (string, error)
}
```

- Wrapping up

- We've covered
  - How to test a HTTP handler that has had the request cancelled by the client.
  - How to use context to manage cancellation.
  - How to write a function that accepts `context` and uses it to cancel itself by using goroutines, `select`, and channels.
  - Follow Google's guidelines as to how to manage cancellation by propagating request-scoped context through your call stack.
  - How to roll your own spy or `http.ResponseWriter` if you need it.
- Never use `context.Value`. If a function needs some values, put them as typed parameters rather than trying to fetch them from `context.Value`. This makes it statically checked and documented for everyone to see. The content of `context.Value` is for maintainers not users. It should never be required input for documented or expected results.

Chapter 16 - Roman Numerals

- We're going to do the Roman Numeral Kata, but we're doing it TDD-style. We're starting with 1.

```
// 16-roman-numerals/numeral_test.go
package numerals

import "testing"

func TestRomanNumerals(t *testing.T) {
        got := ConvertToRoman(1)
        want := "I"

        if got != want {
                t.Errorf("got %q, want %q", got, want)
        }
}

// 16-roman-numerals/numeral.go
package numerals

func ConvertToRoman(i int) string {
        return ""
}
```

- Like usual, make sure you get the error you expect from your failing test. And it is indeed `got "", want "I"`. We can just hardcode that until the tests make us be more flexible. But we can't do that for 1 and 2! Remember, we're using tests to explore the solution space, so we're going a step at a time. A test that 2 is II is simple enough that I'm not posting it in my notes, but it did force our first refactor. We are going to be doing a lot of testing "given input X, we expect Y", and that is perfect for table-driven testing. The first "interesting" changes are a loop when we get to 3, and then changing it to count down from the argument value when we get to 4. After 5, we realize we're really subtracting from the number passed to us when we translate it. The book points out that repeated code within loops is usually a sign of an abstraction waiting to be called out. We also are using a `switch` now and looping while `arabic > 0`:

```
// 16-roman-numerals/numeral.go
package numerals

import "strings"

func ConvertToRoman(arabic int) string {

	var result strings.Builder

	for arabic > 0 {
		switch {
		case arabic > 4:
			result.WriteString("V")
			arabic -= 5
		case arabic > 3:
			result.WriteString("IV")
			arabic -= 4
		default:
			result.WriteString("I")
			arabic--
		}
	}

	return result.String()
}
```

- This strategy works through 8, but then we need another case to catch 9 and 10. You can make it work to 39 with a few lines for 9 and 10. You might be suspicious of `switch` from OO programming. Usually you are capturing a concept or data inside some imperative code

when in fact it could be captured in a class structure instead. We don't have classes, but we do have structs. So let's refactor:

```
// 16-roman-numerals/numeral.go
package numerals

import "strings"

type RomanNumeral struct {
        Value  int
        Symbol string
}

var allRomanNumerals = []RomanNumeral{
        {10, "X"},
        {9, "IX"},
        {5, "V"},
        {4, "IV"},
        {1, "I"},
}

func ConvertToRoman(arabic int) string {

        var result strings.Builder

        for _, numeral := range allRomanNumerals {
                for arabic >= numeral.Value {
                        result.WriteString(numeral.Symbol)
                        arabic -= numeral.Value
                }
        }

        return result.String()
}
```

- This approach extends out to handling large numbers quite well, as you'll see in the final code. We also decided to use the data to describe the test case and eliminate the name field.
- We aren't done having fun with Roman numerals yet though. We're going to convert the other way. We can use the same test cases even!

```
// 16-roman-numerals/numeral_test.go
func TestConvertingToArabic(t *testing.T) {
        for _, tt := range cases[:1] {
                t.Run(fmt.Sprintf("%q gets converted to %d", tt.Roman,
tt.Arabic),
                        func(t *testing.T) {
                                got := ConvertToArabic(tt.Roman)
                                if got != tt.Arabic {
                                        t.Errorf("got %d, want %d", got,
tt.Arabic)
                                }
                        })
        }
}

// 16-roman-numerals/numeral.go
func ConvertToArabic(roman string) int {
        arabic := 0
        for _, numeral := range allRomanNumerals {
                for strings.HasPrefix(roman, numeral.Symbol) {
                        arabic += numeral.Value
                        roman = strings.TrimPrefix(roman, numeral.Symbol)
                }
        }
        return arabic
}
```

- There are several rules in the domain of Roman Numerals we have worked with in this chapter:
    - Can't have more than 3 consecutive symbols
    - Only I (1), X (10), and C (100) can be "subtractors"
    - Taking the result of `ConvertToRoman(N)` and passing it to `ConvertToArabic` should return us N
- The tests we have written so far can be described as "example" based tests where we provide *examples* for the tooling to verify. Property based tests help you verify rules by throwing random data at your code and making sure the rules always hold true. The real challenge about property based tests is having a good understanding of your domain so you can write these properties.

```
// 16-roman-numerals/numeral_test.go
func TestPropertiesOfConversion(t *testing.T) {
        assertion := func(arabic int) bool {
                roman := ConvertToRoman(arabic)
                fromRoman := ConvertToArabic(roman)
                return fromRoman == arabic
        }

        if err := quick.Check(assertion, nil); err != nil {
                t.Error("failed checks", err)
        }
}
```

- The `testing/quick` package from the standard library supports property based tests. We provide `quick.Check` a function that it will run against a number of random inputs; if the function returns `false` it will be seen as failing the check. Our `assertion` function takes a random number and runs our functions to test the property.

- We immediately find two problems with our code. One, it cannot deal with negative numbers. You can't represent negatives in Roman numerals, so we really need to restrict the input domain. We also have problems with numbers greater than 3999 because we don't use more than 3 of any symbol and only have symbols up to 1000. So we change all our `int` to `uint16` : unsigned so we don't have to deal with negatives, and 16 bits so the numbers can't be larger than 65535. The book's assertion then returns `true` early for anything over 3999.

- `quick.Check` runs 100 times by default, but only about 6% of random `uint16` values are 3999 or below, so the early return skips ~94% of the runs. The book raises the count with `&quick.Config{MaxCount: 1000}` . I instead used the config's `Values` function to generate only valid numbers (1-3999), so every run checks a real conversion:

```
config := &quick.Config{
        Values: func(args []reflect.Value, r *rand.Rand) {
                args[0] = reflect.ValueOf(uint16(r.Intn(3999) + 1)) //
1..3999
        },
        MaxCount: 1000,
}
```

    - `args[0]` is the assertion's first parameter, and the `reflect.Value` must hold exactly its type (`uint16`, not `int`), or `quick` panics at runtime.
    - `Values` takes the old `math/rand` `*rand.Rand`, not `math/rand/v2`.
    - `testing/quick` is frozen. Go's native fuzzing (`func FuzzXxx(f *testing.F)` with `go test -fuzz`) is the modern alternative.
- Final code:

```go
// 16-roman-numerals/numeral_test.go
package numerals

import (
	"fmt"
	"math/rand"
	"reflect"
	"testing"
	"testing/quick"
)

var cases = []struct {
	Arabic uint16
	Roman  string
}{
	{Arabic: 1, Roman: "I"},
	{Arabic: 2, Roman: "II"},
	{Arabic: 3, Roman: "III"},
	{Arabic: 4, Roman: "IV"},
	{Arabic: 5, Roman: "V"},
	{Arabic: 6, Roman: "VI"},
	{Arabic: 7, Roman: "VII"},
	{Arabic: 8, Roman: "VIII"},
	{Arabic: 9, Roman: "IX"},
	{Arabic: 10, Roman: "X"},
	{Arabic: 14, Roman: "XIV"},
	{Arabic: 18, Roman: "XVIII"},
	{Arabic: 20, Roman: "XX"},
	{Arabic: 39, Roman: "XXXIX"},
	{Arabic: 40, Roman: "XL"},
	{Arabic: 47, Roman: "XLVII"},
	{Arabic: 49, Roman: "XLIX"},
	{Arabic: 50, Roman: "L"},
	{Arabic: 90, Roman: "XC"},
	{Arabic: 100, Roman: "C"},
	{Arabic: 400, Roman: "CD"},
	{Arabic: 500, Roman: "D"},
	{Arabic: 798, Roman: "DCCXCVIII"},
	{Arabic: 900, Roman: "CM"},
	{Arabic: 1006, Roman: "MVI"},
	{Arabic: 1000, Roman: "M"},
	{Arabic: 1984, Roman: "MCMLXXXIV"},
	{Arabic: 2014, Roman: "MMXIV"},
	{Arabic: 3999, Roman: "MMMCMXCIX"},
}
```

```
func TestRomanNumerals(t *testing.T) {
	for _, tt := range cases {
		t.Run(fmt.Sprintf("%d gets converted to %q", tt.Arabic,
tt.Roman),
			func(t *testing.T) {
				got := ConvertToRoman(tt.Arabic)
				if got != tt.Roman {
					t.Errorf("got %q, want %q", got,
tt.Roman)
				}
			})
	}
}

func TestConvertingToArabic(t *testing.T) {
	for _, tt := range cases {
		t.Run(fmt.Sprintf("%q gets converted to %d", tt.Roman,
tt.Arabic),
			func(t *testing.T) {
				got := ConvertToArabic(tt.Roman)
				if got != tt.Arabic {
					t.Errorf("got %d, want %d", got,
tt.Arabic)
				}
			})
	}
}

func TestPropertiesOfConversion(t *testing.T) {
	assertion := func(arabic uint16) bool {
		roman := ConvertToRoman(arabic)
		fromRoman := ConvertToArabic(roman)
		return fromRoman == arabic
	}

	config := &quick.Config{
		Values: func(args []reflect.Value, r *rand.Rand) {
			args[0] = reflect.ValueOf(uint16(r.Intn(3999) + 1))
// 1..3999
		},
		MaxCount: 1000,
	}
	if err := quick.Check(assertion, config); err != nil {
		t.Error("failed checks", err)
	}
```

```
}

// 16-roman-numerals/numeral.go
package numerals

import "strings"

type RomanNumeral struct {
        Value  uint16
        Symbol string
}

var allRomanNumerals = []RomanNumeral{
        {1000, "M"},
        {900, "CM"},
        {500, "D"},
        {400, "CD"},
        {100, "C"},
        {90, "XC"},
        {50, "L"},
        {40, "XL"},
        {10, "X"},
        {9, "IX"},
        {5, "V"},
        {4, "IV"},
        {1, "I"},
}

// ConvertToRoman converts an Arabic number to a Roman Numeral.
func ConvertToRoman(arabic uint16) string {

        var result strings.Builder

        for _, numeral := range allRomanNumerals {
                for arabic >= numeral.Value {
                        result.WriteString(numeral.Symbol)
                        arabic -= numeral.Value
                }
        }

        return result.String()
}

// ConvertToArabic converts a Roman Numeral to an Arabic number.
func ConvertToArabic(roman string) uint16 {
        var arabic uint16
```

```
        for _, numeral := range allRomanNumerals {
                for strings.HasPrefix(roman, numeral.Symbol) {
                        arabic += numeral.Value
                        roman = strings.TrimPrefix(roman, numeral.Symbol)
                }
        }
        return arabic
}
```

- Wrapping up
    - This looked complicated at first, but the trick is to get started with something simple and take small steps. At no point in this process did we make any large leaps, do any huge refactorings, or get in a mess. The skill is knowing how to split work up, and that comes with practice and with some lovely TDD to help you on your way.
    - Property based tests
        - Built into the standard library
        - If you can think of ways to describe your domain rules in code, they are an excellent tool for giving you more confidence
        - Force you to think about your domain deeply
        - Potentially a nice complement for your test suite

Chapter 17 - Maths

- For this chapter, we want to make an SVG of an analog clock. Our goal is to design a nice function that takes a `Time` from the `time` package and spits out an SVG of a clock with all the hands - hour, minute, and second - pointing in the right direction. First we'll need an SVG of a clock. SVGs are a fantastic image format to manipulate programmatically because they're written as a series of shapes described in XML.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
"http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg xmlns="http://www.w3.org/2000/svg"
     width="300"
     height="300"
     viewBox="0 0 300 300"
     version="2.0">

  <!-- bezel -->
  <circle cx="150" cy="150" r="100" style="fill:#fff;stroke:#000;stroke-
width:5px;"/>

  <!-- hour hand -->
  <line x1="150" y1="150" x2="114.150000" y2="132.260000"
        style="fill:none;stroke:#000;stroke-width:7px;"/>

  <!-- minute hand -->
  <line x1="150" y1="150" x2="101.290000" y2="99.730000"
        style="fill:none;stroke:#000;stroke-width:7px;"/>

  <!-- second hand -->
  <line x1="150" y1="150" x2="77.190000" y2="202.900000"
        style="fill:none;stroke:#f00;stroke-width:3px;"/>
</svg>
```

- That's a circle with three lines, each of the lines starting in the middle of the circle (x=150, y=150), and ending some distance away. So we are going to somehow reconstruct the above but change the lines so they point in the appropriate directions for a given time.
- For this project, we are going to start with an acceptance test. Sometimes it is nice to write a test that tells you when you've finished writing an entire usable feature; it tells you that the whole thing you're trying to achieve - the "feature" - is complete. These types of tests can be called "acceptance tests" or "feature tests". The idea is that you write a high level test to describe what you are trying to achieve. When we've written that test, we can then write more tests - unit tests - that build towards a working system that will pass the acceptance tests.
- Looking at the example, we can say that:
    - every clock has a center of (150, 150)
    - the hour hand is 50 long

    - the minute hand is 80 long
    - the second hand is 90 long
- The example SVG doesn't actually match: its hour and minute hands are 40 and 70 long (the second hand is 90). You can work them out with the distance formula, `sqrtdx^2 + dy^2)` from the center to `(x2, y2)`. We'll use the book's 50, 80, and 90 going forward.
- A thing to note about SVGs: the origin - point (0, 0) - is at the top left hand corner, not the bottom left as we might expect. This is important to remember when we're working out what numbers to plug into our lines.
- We're also not deciding how we're going to write the SVG yet.
- And we're declaring our test in a different package from our code, so we're forced to test through the public API. That means we'll need to put our code in the imports.

```go
// 17-math/clockface/clockface_test.go
package clockface_test

import (
        "testing"
        "time"

        "example.com/learn-go-with-tests/17-math/clockface"
)

func TestSecondHandAtMidnight(t *testing.T) {
        tm := time.Date(1337, time.January, 1, 0, 0, 0, 0, time.UTC)

        want := clockface.Point{X: 150, Y: 150 - 90}
        got := clockface.SecondHand(tm)

        if got != want {
                t.Errorf("got %v, want %v", got, want)
        }
}

// And a first draft to make it compile and pass
// 17-math/clockface/clockface.go
package clockface

import "time"

// A Point represents a two-dimensional Cartesian coordinate
type Point struct {
        X float64
        Y float64
}

// SecondHand is the unit vector of the second hand of an analogue clock at
time `t`
// represented as a Point.
func SecondHand(t time.Time) Point {
        return Point{150, 60}
}
```

- So we could have tested more, but this gets us started. To place the second hand at midnight we expect that it hasn't moved from the center of the clockface on the X axis - still 150 - and the Y axis is the length of the hand "up" from the center; 150 - 90, where 90 is the length of the hand.

- `time.Date(year, month, day, hour, min, sec, nsec, loc)` builds a `time.Time`, so `time.Date(1337, time.January, 1, 0, 0, 0, 0, time.UTC)` is midnight on January 1, 1337, UTC. A clock face only cares about the time of day, so the date is arbitrary; the author picks fun ones (1337 is "leet", and the later `simpleTime` helper's October 28, 312 is the Battle of the Milvian Bridge).

- Note we aren't writing acceptance tests yet. This tests an important function, but it still relies on implementation details like the second hand endpoint being represented by a `Point`. We'll write a real "acceptance test" where we check SVG output later.

```
// 17-math/clockface/clockface_test.go
func TestSecondHandAt30Seconds(t *testing.T) {
        tm := time.Date(1337, time.January, 1, 0, 0, 30, 0, time.UTC)

        want := clockface.Point{X: 150, Y: 150 + 90}
        got := clockface.SecondHand(tm)

        if got != want {
                t.Errorf("got %v, want %v", got, want)
        }
}
```

- Alright, now this test really forces our hand (HAHA!). It's the same idea but now the second hand is pointing downwards so we add the length to the Y axis (remember, top left is 0, 0).

- So each minute the second hand goes through 60 positions. Say we wonder where it is pointing at 37 seconds. We would want the angle between 12 o'clock and 37/60ths around the circle. In degrees this is (360 / 60) * 37 = 222, but it's easier to remember that it is 37/60 of a complete rotation. But we need to know the X and Y coordinates that the tip is at, not the angle.

- Think back to math and normal origins where 0, 0 is the center of the unit circle (circle with radius 1). The x and y coordinates of each point on this circle form a triangle with hypotenuse 1. Trigonometry will let us work out the X and Y lengths for each triangle if we know the angle they make with the origin. The X coordinate will be cos(a) and the Y coordinate sin(a), where a is the angle made between the line and the (positive) x axis. The book had lovely pictures, but if you need a visual: https://en.wikipedia.org/wiki/Unit_circle. However, since we want to measure the angle from 12 o'clock rather than from the X axis (3 o'clock), we need to swap the axes around so x = sin(a) and y = cos(a).

- Go's `math` library contains `Sin` and `Cos` but there's a trick. They work in radians. In degrees a full turn of the circle is 360 degrees, but in radians it is 2pi radians.
- We moved our old "acceptance tests" (but not really) in the `clockface_test` package to `clockface_acceptance_test.go` and started a new `clockface_test.go` for package `clockface` to unit test our code. We don't plan to export our radian conversion functions, but we still want to test them.

```
// 17-math/clockface/clockface_test.go
func TestSecondsInRadians(t *testing.T) {
        thirtySeconds := time.Date(312, time.October, 28, 0, 0, 30, 0,
time.UTC)
        want := math.Pi
        got := secondsInRadians(thirtySeconds)
        if want != got {
                t.Fatalf("Wanted %v radians, but got %v", want, got)
        }
}
```

- If a full turn around a circle is 2pi radians, then a half turn is pi. After we see the new test fails properly when we return 0, we hardcode `math.Pi` to pass and then refactor this test as a table to force different values.

```
// 17-math/clockface/clockface_test.go
func TestSecondsInRadians(t *testing.T) {
        cases := []struct {
                time  time.Time
                angle float64
        }{
                {simpleTime(0, 0, 30), math.Pi},
                {simpleTime(0, 0, 0), 0},
                {simpleTime(0, 0, 45), (math.Pi / 2) * 3},
                {simpleTime(0, 0, 7), (math.Pi / 30) * 7},
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
                        got := secondsInRadians(tt.time)
                        if got != tt.angle {
                                t.Fatalf("Wanted %v radians, but got %v",
tt.angle, got)
                        }
                })
        }
}

func simpleTime(hours, minutes, seconds int) time.Time {
        return time.Date(312, time.October, 28, hours, minutes, seconds, 0,
time.UTC)
}

func testName(t time.Time) string {
        return t.Format("15:04:05")
}
```

- Now we need to calculate a value based on input! 0, 30, and 45 are all easy cases, and then 7 requires us to do 7/60ths of a circle (7/30ths of pi).

```
// 17-math/clockface/clockface.go
func secondsInRadians(t time.Time) float64 {
        return float64(t.Second()) * (math.Pi / 30)
}
```

- But not all our tests pass. We get an error: `Wanted 3.141592653589793 radians, but got 3.1415926535897936`, because floats are horrible. We could establish a "near enough" test

since being off by such a small amount won't hurt our clock, but we know what is happening here is because we are dividing by 30 then multiplying by 30 again, so we are going to rearrange our math so instead of `numberOfSeconds * pi / 30` we calculate `pi / (30 / numberOfSeconds)`, which is mathematically equivalent but rounds differently.

```
// 17-math/clockface/clockface.go
func secondsInRadians(t time.Time) float64 {
        return (math.Pi / (30 / (float64(t.Second()))))
}
```

- We do have a lurking problem when `t.Second()` is 0: we are dividing by zero. Go floats follow IEEE 754, so dividing a non-zero float by zero returns `+Inf` instead of panicking (and `0.0 / 0.0` is `NaN`). Only division by zero between *constants* is a compile error, and dividing *integer* variables by zero panics at runtime. So at 0 seconds we get `30 / 0.0 = +Inf`, then `pi / +Inf = 0`, which happens to be the right answer. We'll switch to comparing with a tolerance later, so it was kind of confusing to introduce a calculation that goes through infinity, aside from demonstrating how floats process division by zero.
- We're going to make a simplifying assumption for now in the math that we only have to deal with the unit circle. We'll scale our results by hand length later.

```
// 17-math/clockface/clockface_test.go
func TestSecondHandPoint(t *testing.T) {
        cases := []struct {
                time  time.Time
                point Point
        }{
                {simpleTime(0, 0, 30), Point{0, -1}},
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
                        got := secondHandPoint(tt.time)
                        if got != tt.point {
                                t.Fatalf("Wanted %v Point, but got %v",
tt.point, got)
                        }
                })
        }
}
```

- So we implement a hardcoded return then modify our test to also test 45 is {-1, 0}. Then we need real math in our function. But floating point math strikes again! Both tests fail with `Wanted {0 -1} Point, but got {1.2246467991473515e-16 -1}` and `Wanted {-1 0} Point, but got {-1 -1.8369701987210272e-16}`. We have the option of using the `Rat` type from `math/big`, but as our goal is to draw a clock and not land on the moon, close enough is fine. These numbers are only off in the sixteenth decimal place.

```
// 17-math/clockface/clockface.go
func secondHandPoint(t time.Time) Point {
        angle := secondsInRadians(t)
        x := math.Sin(angle)
        y := math.Cos(angle)

        return Point{x, y}
}

// 17-math/clockface/clockface_test.go
func TestSecondHandPoint(t *testing.T) {
        cases := []struct {
                time  time.Time
                point Point
        }{
                {simpleTime(0, 0, 30), Point{0, -1}},
                {simpleTime(0, 0, 45), Point{-1, 0}},
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
                        got := secondHandPoint(tt.time)
                        if !roughlyEqualPoint(got, tt.point) {
                                t.Fatalf("Wanted %v Point, but got %v",
tt.point, got)
                        }
                })
        }
}

func roughlyEqualFloat64(a, b float64) bool {
        const equalityThreshold = 1e-7
        return math.Abs(a-b) < equalityThreshold
}

func roughlyEqualPoint(a, b Point) bool {
        return roughlyEqualFloat64(a.X, b.X) &&
                roughlyEqualFloat64(a.Y, b.Y)
}
```

- So now we can circle back around and get our "acceptance test" working. We need to do three things to convert our unit vector into a point on the SVG:

    1. Scale it to the length of the hand

2. Flip it over the X axis to account for the SVG having an origin in the top left hand corner
3. Translate it to the right positions so that it's coming from an origin of (150, 150)

- Here's our second attempt at the new function that passes, after we factored out a few magic numbers into constants:

```
// 17-math/clockface/clockface.go
const secondHandLength = 90
const clockCenterX = 150
const clockCenterY = 150

// SecondHand is the unit vector of the second hand of an analogue clock at
time `t`
// represented as a Point.
func SecondHand(t time.Time) Point {
        p := secondHandPoint(t)
        p = Point{p.X * secondHandLength, p.Y * secondHandLength} // scale
        p = Point{p.X, -p.Y}                                      // flip
        p = Point{p.X + clockCenterX, p.Y + clockCenterY}         //
translate
        return p
}
```

- We're ready to draw a clock; or at least the second hand of one. So we will start a new `clockface/main.go` underneath our `clockface` directory. It will have a `main` package that creates the binary that will build an SVG. If you build and run `./clockface > clock.svg` (or `go run ./clockface > clock.svg` from the library's directory), you'll have a perfectly good image of a clock with only a second hand:

```
// 17-math/clockface/clockface/main.go
package main

import (
        "fmt"
        "io"
        "os"
        "time"

        "example.com/learn-go-with-tests/17-math/clockface"
)

func main() {
        t := time.Now()
        sh := clockface.SecondHand(t)
        io.WriteString(os.Stdout, svgStart)
        io.WriteString(os.Stdout, bezel)
        io.WriteString(os.Stdout, secondHandTag(sh))
        io.WriteString(os.Stdout, svgEnd)
}

func secondHandTag(p clockface.Point) string {
        return fmt.Sprintf(`<line x1="150" y1="150" x2="%f" y2="%f"
style="fill:none;stroke:#f00;stroke-width:3px;"/>`, p.X, p.Y)
}

const svgStart = `<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
"http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg xmlns="http://www.w3.org/2000/svg"
     width="100%"
     height="100%"
     viewBox="0 0 300 300"
     version="2.0">`

const bezel = `<circle cx="150" cy="150" r="100"
style="fill:#fff;stroke:#000;stroke-width:5px;"/>`

const svgEnd = `</svg>`
```

- This both works and spits out a perfectly fine looking SVG, but it's a bit ugly, the `secondHandTag` function doesn't even mention SVGs, and we're not testing any of the SVG code. We could add a test that checks the output for containing the line by passing a buffer instead of `os.Stdout`, but that could pass even if we didn't produce a valid SVG and would

fail for the smallest change to the output (say, adding a space). The biggest smell is that we would be testing a data structure, XML, by looking at its representation as a series of characters, a string. That is never a good idea as it tends to produce tests that are both too fragile and not sensitive enough, because you're testing the wrong thing. So the solution is to test the output as XML, and to do that we need to parse it.

- `encoding/xml` is the Go package that handles simple XML parsing. `xml.Unmarshal` takes a `[]byte` of XML data and a pointer to a struct for it to get unmarshalled into. So we'll need a struct to unmarshal our XML into. We could do this manually, but it is quite tedious and there is a program called `zek` that will automate all of that hard work for us. There's even an online version at https://xml-to-go.github.io/. Just paste the SVG from the file into one box and out pops a `struct`. We're going to put this in `clockface_acceptance_test.go` to produce a "real" acceptance test. We also require fairly massive refactorings to `clockface.go` and `main.go`, and we need our new `svg_writer.go` to output floats with 3 decimals and test to that degree of accuracy in the test. Here are the most recent files now that we're back to green.

- The book doesn't spell out all of that refactoring; I had to compare against the repo's `math/v7b` to find it:

  1. Delete the old point-based acceptance tests. They call `SecondHand`, which isn't exported any more once the acceptance tests check the SVG.
  2. Move the SVG constants and helpers out of `clockface.go` into `svg_writer.go`, and delete them from `clockface.go`. Go won't allow two declarations of the same name in one package, even in different files.
  3. Change `main.go` to just call `clockface.SVGWriter(os.Stdout, t)`.

```
// 17-math/clockface/clockface_acceptance_test.go
package clockface_test

import (
        "bytes"
        "encoding/xml"
        "testing"
        "time"

        "example.com/learn-go-with-tests/17-math/clockface"
)

func TestSVGWriterAtMidnight(t *testing.T) {
        tm := time.Date(1337, time.January, 1, 0, 0, 0, 0, time.UTC)

        b := bytes.Buffer{}
        clockface.SVGWriter(&b, tm)

        svg := SVG{}
        xml.Unmarshal(b.Bytes(), &svg)

        x2 := "150.000"
        y2 := "60.000"

        for _, line := range svg.Line {
                if line.X2 == x2 && line.Y2 == y2 {
                        return
                }
        }

        t.Errorf("Expected to find the second hand with x2 of %+v and y2 of
%+v, in the SVG output %v", x2, y2, b.String())
}

type SVG struct {
        XMLName xml.Name `xml:"svg"`
        Text    string   `xml:",chardata"`
        Xmlns   string   `xml:"xmlns,attr"`
        Width   string   `xml:"width,attr"`
        Height  string   `xml:"height,attr"`
        ViewBox string   `xml:"viewBox,attr"`
        Version string   `xml:"version,attr"`
        Circle  struct {
                Text  string `xml:",chardata"`
                Cx    string `xml:"cx,attr"`
```

```
                Cy    string `xml:"cy,attr"`
                R     string `xml:"r,attr"`
                Style string `xml:"style,attr"`
        } `xml:"circle"`
        Line []struct {
                Text  string `xml:",chardata"`
                X1    string `xml:"x1,attr"`
                Y1    string `xml:"y1,attr"`
                X2    string `xml:"x2,attr"`
                Y2    string `xml:"y2,attr"`
                Style string `xml:"style,attr"`
        } `xml:"line"`
}

// 17-math/clockface/svg_writer.go
package clockface

import (
        "fmt"
        "io"
        "time"
)

const (
        secondHandLength = 90
        clockCenterX     = 150
        clockCenterY     = 150
)

// SVGWriter writes an SVG representation of an analogue clock, showing the
// time t, to the writer w
func SVGWriter(w io.Writer, t time.Time) {
        io.WriteString(w, svgStart)
        io.WriteString(w, bezel)
        secondHand(w, t)
        io.WriteString(w, svgEnd)
}

func secondHand(w io.Writer, t time.Time) {
        p := secondHandPoint(t)
        p = Point{p.X * secondHandLength, p.Y * secondHandLength} // scale
        p = Point{p.X, -p.Y}                                      // flip
        p = Point{p.X + clockCenterX, p.Y + clockCenterY}         //
translate
        fmt.Fprintf(w, `<line x1="150" y1="150" x2="%.3f" y2="%.3f"
style="fill:none;stroke:#f00;stroke-width:3px;"/>`, p.X, p.Y)
```

```
}

const svgStart = `<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
"http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg xmlns="http://www.w3.org/2000/svg"
     width="100%"
     height="100%"
     viewBox="0 0 300 300"
     version="2.0">`

const bezel = `<circle cx="150" cy="150" r="100"
style="fill:#fff;stroke:#000;stroke-width:5px;"/>`

const svgEnd = `</svg>`

// 17-math/clockface/clockface/main.go
package main

import (
        "os"
        "time"

        "example.com/learn-go-with-tests/17-math/clockface"
)

func main() {
        t := time.Now()
        clockface.SVGWriter(os.Stdout, t)
}

// 17-math/clockface/clockface.go
package clockface

import (
        "math"
        "time"
)

// A Point represents a two-dimensional Cartesian coordinate
type Point struct {
        X float64
        Y float64
}

func secondsInRadians(t time.Time) float64 {
```

```
        return (math.Pi / (30 / (float64(t.Second()))))
}

func secondHandPoint(t time.Time) Point {
        angle := secondsInRadians(t)
        x := math.Sin(angle)
        y := math.Cos(angle)

        return Point{x, y}
}
```

- First thing we're going to do is clean up our XML struct a bit. It had some `Text` and `Style` fields we don't care about. We also made `Circle` and `Line` proper named types (careful: `Line` is a plain `struct`; the `[]` stays on the `SVG.Line` field), and we specified our coordinates are `float64`:

```
// 17-math/clockface/clockface_acceptance_test.go
type SVG struct {
        XMLName xml.Name `xml:"svg"`
        Xmlns   string   `xml:"xmlns,attr"`
        Width   string   `xml:"width,attr"`
        Height  string   `xml:"height,attr"`
        ViewBox string   `xml:"viewBox,attr"`
        Version string   `xml:"version,attr"`
        Circle  Circle   `xml:"circle"`
        Line    []Line   `xml:"line"`
}

type Circle struct {
        Cx float64 `xml:"cx,attr"`
        Cy float64 `xml:"cy,attr"`
        R  float64 `xml:"r,attr"`
}

type Line struct {
        X1 float64 `xml:"x1,attr"`
        Y1 float64 `xml:"y1,attr"`
        X2 float64 `xml:"x2,attr"`
        Y2 float64 `xml:"y2,attr"`
}

// and also in the same file we make another test
// this one replaces `TestSVGWriterAtMidnight` since it has the same test
case
func TestSVGWriterSecondHand(t *testing.T) {
        cases := []struct {
                time time.Time
                line Line
        }{
                {
                        simpleTime(0, 0, 0),
                        Line{150, 150, 150, 60},
                },
                {
                        simpleTime(0, 0, 30),
                        Line{150, 150, 150, 240},
                },
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
```

```
                        b := bytes.Buffer{}
                        clockface.SVGWriter(&b, tt.time)

                        svg := SVG{}
                        xml.Unmarshal(b.Bytes(), &svg)

                        if !containsLine(tt.line, svg.Line) {
                                t.Errorf("Expected to find the second hand
line %+v, in the SVG lines %+v", tt.line, svg.Line)
                        }
                })
        }
}

// and further down the same file some helpers
func containsLine(l Line, ls []Line) bool {
        for _, line := range ls {
                if line == l {
                        return true
                }
        }
        return false
}

func simpleTime(hours, minutes, seconds int) time.Time {
        return time.Date(312, time.October, 28, hours, minutes, seconds, 0,
time.UTC)
}

func testName(t time.Time) string {
        return t.Format("15:04:05")
}
```

- We now have the same helpers defined in two places, but that's because we have different packages and don't export test helpers. They're small at least. Now we add a suspiciously similar minute hand test, and we're back to red.

```
// 17-math/clockface/clockface_acceptance_test.go
func TestSVGWriterMinuteHand(t *testing.T) {
        cases := []struct {
                time time.Time
                line Line
        }{
                {
                        simpleTime(0, 0, 0),
                        Line{150, 150, 150, 70},
                },
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
                        b := bytes.Buffer{}
                        clockface.SVGWriter(&b, tt.time)

                        svg := SVG{}
                        xml.Unmarshal(b.Bytes(), &svg)

                        if !containsLine(tt.line, svg.Line) {
                                t.Errorf("Expected to find the minute hand
line %+v, in the SVG lines %+v", tt.line, svg.Line)
                        }
                })
        }
}
```

- So we do the minute hand in roughly the same way we did the second hand. We write `TestMinutesInRadians`, then `minutesInRadians`, but for the minute hand we want to handle things nicer than jumping to the next minute on the minute, and we can calculate using `secondsInRadians` to help us do that. For every second, the minute hand will move 1/60th of the angle the second hand moves. Then we need a `TestMinuteHandPoint` and a `minuteHandPoint`. But at that "point" (HAHA!) we notice that we have duplication in our `XHandPoint` functions, so we factor out a more general `angleToPoint` function and call it from each. Then finally we define `minuteHand` and add a call to it from our `SVGWriter`. When we're green again, we do a final refactor to remove the duplication of the two `XHand` functions by factoring out a common `makeHand` that takes a point and length. The code looks quite similar to what we've already seen, and I'll post the final version at the end of the chapter. It is really rather nice being able to do all these refactorings while staying green after writing it the straightforward way on first pass.

- And we need to do the same for the hour hand, starting with an acceptance test. `hoursInRadians` uses `t.Hour() % 12`, since the clock only has 12 hours, and adds the minute hand's angle divided by 12, since one full turn of the minute hand moves the hour hand 1/12th of a turn. Last, the magic numbers become constants (`secondsInHalfClock`, `minutesInClock`, `hoursInClock`, etc.), named for the reasoning: the minute hand's contribution is divided by `hoursInClock` (12) and the second hand's by `minutesInClock` (60).
- Final code:

```
// 17-math/clockface/clockface_acceptance_test.go
package clockface_test

import (
        "bytes"
        "encoding/xml"
        "testing"
        "time"

        "example.com/learn-go-with-tests/17-math/clockface"
)

func TestSVGWriterSecondHand(t *testing.T) {
        cases := []struct {
                time time.Time
                line Line
        }{
                {
                        simpleTime(0, 0, 0),
                        Line{150, 150, 150, 60},
                },
                {
                        simpleTime(0, 0, 30),
                        Line{150, 150, 150, 240},
                },
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
                        b := bytes.Buffer{}
                        clockface.SVGWriter(&b, tt.time)

                        svg := SVG{}
                        xml.Unmarshal(b.Bytes(), &svg)

                        if !containsLine(tt.line, svg.Line) {
                                t.Errorf("Expected to find the second hand
line %+v, in the SVG lines %+v", tt.line, svg.Line)
                        }
                })
        }
}

func TestSVGWriterMinuteHand(t *testing.T) {
        cases := []struct {
```

```
                time time.Time
                line Line
        }{
                {
                        simpleTime(0, 0, 0),
                        Line{150, 150, 150, 70},
                },
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
                        b := bytes.Buffer{}
                        clockface.SVGWriter(&b, tt.time)

                        svg := SVG{}
                        xml.Unmarshal(b.Bytes(), &svg)

                        if !containsLine(tt.line, svg.Line) {
                                t.Errorf("Expected to find the minute hand
line %+v, in the SVG lines %+v", tt.line, svg.Line)
                        }
                })
        }
}

func TestSVGWriterHourHand(t *testing.T) {
        cases := []struct {
                time time.Time
                line Line
        }{
                {
                        simpleTime(6, 0, 0),
                        Line{150, 150, 150, 200},
                },
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
                        b := bytes.Buffer{}
                        clockface.SVGWriter(&b, tt.time)

                        svg := SVG{}
                        xml.Unmarshal(b.Bytes(), &svg)

                        if !containsLine(tt.line, svg.Line) {
```

```
                                t.Errorf("Expected to find the hour hand line
%+v, in the SVG lines %+v", tt.line, svg.Line)
                        }
                })
        }
}

type SVG struct {
        XMLName xml.Name `xml:"svg"`
        Xmlns   string   `xml:"xmlns,attr"`
        Width   string   `xml:"width,attr"`
        Height  string   `xml:"height,attr"`
        ViewBox string   `xml:"viewBox,attr"`
        Version string   `xml:"version,attr"`
        Circle  Circle   `xml:"circle"`
        Line    []Line   `xml:"line"`
}

type Circle struct {
        Cx float64 `xml:"cx,attr"`
        Cy float64 `xml:"cy,attr"`
        R  float64 `xml:"r,attr"`
}

type Line struct {
        X1 float64 `xml:"x1,attr"`
        Y1 float64 `xml:"y1,attr"`
        X2 float64 `xml:"x2,attr"`
        Y2 float64 `xml:"y2,attr"`
}

func containsLine(l Line, ls []Line) bool {
        for _, line := range ls {
                if line == l {
                        return true
                }
        }
        return false
}

func simpleTime(hours, minutes, seconds int) time.Time {
        return time.Date(312, time.October, 28, hours, minutes, seconds, 0,
time.UTC)
}

func testName(t time.Time) string {
```

```
        return t.Format("15:04:05")
}

// 17-math/clockface/clockface_test.go
package clockface

import (
        "math"
        "testing"
        "time"
)

func TestSecondHandPoint(t *testing.T) {
        cases := []struct {
                time  time.Time
                point Point
        }{
                {simpleTime(0, 0, 30), Point{0, -1}},
                {simpleTime(0, 0, 45), Point{-1, 0}},
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
                        got := secondHandPoint(tt.time)
                        if !roughlyEqualPoint(got, tt.point) {
                                t.Fatalf("Wanted %v Point, but got %v",
tt.point, got)
                        }
                })
        }
}

func TestSecondsInRadians(t *testing.T) {
        cases := []struct {
                time  time.Time
                angle float64
        }{
                {simpleTime(0, 0, 30), math.Pi},
                {simpleTime(0, 0, 0), 0},
                {simpleTime(0, 0, 45), (math.Pi / 2) * 3},
                {simpleTime(0, 0, 7), (math.Pi / 30) * 7},
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
                        got := secondsInRadians(tt.time)
```

```
                        if !roughlyEqualFloat64(got, tt.angle) {
                                t.Fatalf("Wanted %v radians, but got %v",
tt.angle, got)
                        }
                })
        }
}

func TestMinuteHandPoint(t *testing.T) {
        cases := []struct {
                time  time.Time
                point Point
        }{
                {simpleTime(0, 30, 0), Point{0, -1}},
                {simpleTime(0, 45, 0), Point{-1, 0}},
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
                        got := minuteHandPoint(tt.time)
                        if !roughlyEqualPoint(got, tt.point) {
                                t.Fatalf("Wanted %v Point, but got %v",
tt.point, got)
                        }
                })
        }
}

func TestMinutesInRadians(t *testing.T) {
        cases := []struct {
                time  time.Time
                angle float64
        }{
                {simpleTime(0, 30, 0), math.Pi},
                {simpleTime(0, 0, 7), 7 * (math.Pi / (30 * 60))},
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
                        got := minutesInRadians(tt.time)
                        if !roughlyEqualFloat64(got, tt.angle) {
                                t.Fatalf("Wanted %v radians, but got %v",
tt.angle, got)
                        }
                })
        }
```

```
}

func TestHourHandPoint(t *testing.T) {
        cases := []struct {
                time  time.Time
                point Point
        }{
                {simpleTime(6, 0, 0), Point{0, -1}},
                {simpleTime(21, 0, 0), Point{-1, 0}},
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
                        got := hourHandPoint(tt.time)
                        if !roughlyEqualPoint(got, tt.point) {
                                t.Fatalf("Wanted %v Point, but got %v",
tt.point, got)
                        }
                })
        }
}

func TestHoursInRadians(t *testing.T) {
        cases := []struct {
                time  time.Time
                angle float64
        }{
                {simpleTime(0, 0, 0), 0},
                {simpleTime(6, 0, 0), math.Pi},
                {simpleTime(21, 0, 0), math.Pi * 1.5},
                {simpleTime(0, 1, 30), math.Pi / ((6 * 60 * 60) / 90)},
        }

        for _, tt := range cases {
                t.Run(testName(tt.time), func(t *testing.T) {
                        got := hoursInRadians(tt.time)
                        if !roughlyEqualFloat64(got, tt.angle) {
                                t.Fatalf("Wanted %v radians, but got %v",
tt.angle, got)
                        }
                })
        }
}

func roughlyEqualFloat64(a, b float64) bool {
        const equalityThreshold = 1e-7
```

```
        return math.Abs(a-b) < equalityThreshold
}

func roughlyEqualPoint(a, b Point) bool {
        return roughlyEqualFloat64(a.X, b.X) &&
                roughlyEqualFloat64(a.Y, b.Y)
}

func simpleTime(hours, minutes, seconds int) time.Time {
        return time.Date(312, time.October, 28, hours, minutes, seconds, 0,
time.UTC)
}

func testName(t time.Time) string {
        return t.Format("15:04:05")
}

// 17-math/clockface/clockface.go
package clockface

import (
        "math"
        "time"
)

const (
        secondsInHalfClock = 30
        secondsInClock     = 2 * secondsInHalfClock
        minutesInHalfClock = 30
        minutesInClock     = 2 * minutesInHalfClock
        hoursInHalfClock   = 6
        hoursInClock       = 2 * hoursInHalfClock
)

// A Point represents a two-dimensional Cartesian coordinate
type Point struct {
        X float64
        Y float64
}

func secondsInRadians(t time.Time) float64 {
        return (math.Pi / (secondsInHalfClock / float64(t.Second())))
}

func secondHandPoint(t time.Time) Point {
        return angleToPoint(secondsInRadians(t))
```

```
}

func minutesInRadians(t time.Time) float64 {
        return (secondsInRadians(t) / minutesInClock) +
                (math.Pi / (minutesInHalfClock / float64(t.Minute())))
}

func minuteHandPoint(t time.Time) Point {
        return angleToPoint(minutesInRadians(t))
}

func hoursInRadians(t time.Time) float64 {
        return (minutesInRadians(t) / hoursInClock) +
                (math.Pi / (hoursInHalfClock /
float64(t.Hour()%hoursInClock)))
}

func hourHandPoint(t time.Time) Point {
        return angleToPoint(hoursInRadians(t))
}

func angleToPoint(angle float64) Point {
        x := math.Sin(angle)
        y := math.Cos(angle)

        return Point{x, y}
}

// 17-math/clockface/svg_writer.go
package clockface

import (
        "fmt"
        "io"
        "time"
)

const (
        secondHandLength = 90
        minuteHandLength = 80
        hourHandLength   = 50
        clockCenterX     = 150
        clockCenterY     = 150
)

// SVGWriter writes an SVG representation of an analogue clock, showing the
```

```
// time t, to the writer w
func SVGWriter(w io.Writer, t time.Time) {
        io.WriteString(w, svgStart)
        io.WriteString(w, bezel)
        secondHand(w, t)
        minuteHand(w, t)
        hourHand(w, t)
        io.WriteString(w, svgEnd)
}

func secondHand(w io.Writer, t time.Time) {
        p := makeHand(secondHandPoint(t), secondHandLength)
        fmt.Fprintf(w, `<line x1="150" y1="150" x2="%.3f" y2="%.3f"
style="fill:none;stroke:#f00;stroke-width:3px;"/>`, p.X, p.Y)
}

func minuteHand(w io.Writer, t time.Time) {
        p := makeHand(minuteHandPoint(t), minuteHandLength)
        fmt.Fprintf(w, `<line x1="150" y1="150" x2="%.3f" y2="%.3f"
style="fill:none;stroke:#000;stroke-width:3px;"/>`, p.X, p.Y)
}

func hourHand(w io.Writer, t time.Time) {
        p := makeHand(hourHandPoint(t), hourHandLength)
        fmt.Fprintf(w, `<line x1="150" y1="150" x2="%.3f" y2="%.3f"
style="fill:none;stroke:#000;stroke-width:3px;"/>`, p.X, p.Y)
}

func makeHand(p Point, length float64) Point {
        p = Point{p.X * length, p.Y * length}               // scale
        p = Point{p.X, -p.Y}                                // flip
        return Point{p.X + clockCenterX, p.Y + clockCenterY} // translate
}

const svgStart = `<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
"http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg xmlns="http://www.w3.org/2000/svg"
     width="100%"
     height="100%"
     viewBox="0 0 300 300"
     version="2.0">`

const bezel = `<circle cx="150" cy="150" r="100"
style="fill:#fff;stroke:#000;stroke-width:5px;"/>`
```

```
const svgEnd = `</svg>`

// 17-math/clockface/clockface/main.go
package main

import (
        "os"
        "time"

        "example.com/learn-go-with-tests/17-math/clockface"
)

func main() {
        t := time.Now()
        clockface.SVGWriter(os.Stdout, t)
}
```

Chapter 18 - Reading files

- We are going to write a program to read a folder of markdown documents and create a `Post` data structure for each, to be used as a data source by a web server. So a markdown file:

```
Title: Hello, TDD world!
Description: First post on our wonderful blog
Tags: tdd, go
---
Hello world!

The body of posts starts after the `---`
```

- Is converted into a `Post`:

```
type Post struct {
        Title, Description, Body string
        Tags                     []string
}
```

- Kent Beck says: "Optimism is an occupational hazard of programming. Feedback is the treatment." We should try to deliver real consumer value as quickly as possible instead of

delivering complicated abstractions. Our tests are our feedback. Once we have delivered a small amount of value end-to-end, further iteration of the rest of the requirements is usually straightforward.

- Thinking about the kind of test we want to see... First we think about how we would like to use the code we're going to write from a consumer's point of view. Focus on *what* and *why*, but don't get distracted by *how*. We need a function to point at a folder and return us some posts:

```
var posts []blogposts.Post
posts = blogposts.NewPostsFromFS("some-folder")
```

- To write a test around this, we'd need a test folder with some example posts in it. There's nothing terribly wrong with this, but we are making some trade-offs:
  - for each test you may need to create new files to test a particular behaviour
  - some behaviour will be challenging to test, such as failing to load files
  - the tests will run a little slower because they will need to access the file system
  - we are also unnecessarily coupling ourselves to a specific implementation of the file system
- The `io/fs` package introduced an abstraction for filesystems. It defines basic interfaces to the file system. A file system can be provided by the host operating system, but also by other packages. This lets us loosen our coupling to a specific file system. On the producer side of the interface, the new `embed.FS` type implements `fs.FS`, as does `zip.Reader`. The new `os.DirFS` function provides an implementation of `fs.FS` backed by a tree of operating system files.
- Learning to use interfaces defined in Go's standard library (such as `io/fs`, `io.Reader`, `io.Writer`) is vital to writing loosely coupled packages. These packages can then be re-used in contexts different to those you imagine with minimal fuss. Using `io/fs` in particular gives users a number of options baked into the standard library. For testing we can use `testing/fstest`, which is similar to `net/http/httptest` but for `io/fs`.
- Given this information, we decide to pass our theoretical `NewPostsFromFS` function a `someFS` argument instead of a `string` for a real file system directory.
- Instead of starting with acceptance tests this time, as we learn the library let's prove basic functionality. Our first test will just make sure we can read all the files in a directory and

check the count of `[]Post` returned is the same as the number of files in our fake file system.

- We are going to use the new practice of defining our tests in a separate package from our main code this chapter. By doing so our tests take a *consumer-driven* approach and access the code just like a real user.

```
// 18-reading-files/blogposts_test.go
package blogposts_test

import (
        "testing"
        "testing/fstest"

        blogposts "example.com/learn-go-with-tests/18-reading-files"
)

func TestNewBlogPosts(t *testing.T) {
        fs := fstest.MapFS{
                "hello world.md":  {Data: []byte("hi")},
                "hello-world2.md": {Data: []byte("hola")},
        }

        posts := blogposts.NewPostsFromFS(fs)

        if len(posts) != len(fs) {
                t.Errorf("got %d posts, wanted %d posts", len(posts),
len(fs))
        }
}
```

- We also didn't create a `blogposts` directory. This code just lives in `18-reading-files`. So you can see above a local import with an alias. The "example.com" comes from how we defined our `go.mod` in the parent directory.

- Remember you should initially fail your test. We started with `return nil` from our function to get `got 0 posts wanted 2`. We don't go so far as to slime the test and `return 2` to pass for the first time, since we have a general idea of our structure, so don't need to explore the space. Since we've limited our scope, all we need to do is read the directory and create a `Post` for each file. We can worry about opening and parsing files later. So with this simple implementation we pass. I also went with our first refactor. The book had us take an `fstest.MapFS`, but we had already planned to take the interface `fs.FS`, so I just

went with that. It was a simple refactor while the test was green, but it would've been even simpler just to take the interface from the get go. If you plan something from the start, there's no reason not to just start with it. I'm surprised we don't have the body of `Post` as well, since we already worked it out, but it isn't necessary yet. I do see the value in failing first to verify your test fails and gives the expected error.

```
// 18-reading-files/blogposts.go
package blogposts

import (
        "io/fs"
)

type Post struct {
}

func NewPostsFromFS(fileSystem fs.FS) []Post {
        dir, _ := fs.ReadDir(fileSystem, ".")
        var posts []Post
        for range dir {
                posts = append(posts, Post{})
        }
        return posts
}
```

- Our second refactoring is quite sensible. Errors can occur when working with files, so we should have the capability to return an `error`. In our sample code we also made a `TestNewBlogPostsError` with a stubbed out failing `fs.FS`, but the book points out that testing error handling is mostly worth it when we're doing something interesting with the error, rather than just propagating it as we do here. I kept mine anyway.

- Next we explore and design our `Post` type. Our initial post parser was inelegant and shoved all the handling and parsing of a `Post` into `getPost`, but as soon as we were green, we refactored it into `getPost` to open the file and `newPost` to parse a particular post file. When you factor out new functions or methods, think carefully about the arguments. We initially took an `fs.File` in our `newPost` function but refactored it to the more general `io.Reader` interface, since we were only using methods defined there and shared by multiple types. Likewise our `getPost` doesn't really need an `fs.DirEntry` when we could just pass it a `string` for `Name`, since that's all it uses. One of the nice things about the separation of concerns is that our file handling code is done! We iterate over the

directory and open all the files, and that shouldn't have to change. We can focus on the details of parsing our `Post` in `newPost`.

- We also factored out a test helper called `assertPost`.
- Let's look at where we stand now while parsing just the title. We can likely focus on `newPost` and its test in the upcoming code samples:

```go
// 18-reading-files/blogposts_test.go
func TestNewBlogPosts(t *testing.T) {
        fs := fstest.MapFS{
                "hello world.md":  {Data: []byte("Title: Post 1")},
                "hello-world2.md": {Data: []byte("Title: Post 2")},
        }

        posts, err := blogposts.NewPostsFromFS(fs)

        if err != nil {
                t.Fatal(err)
        }

        assertPost(t, posts[0], blogposts.Post{Title: "Post 1"})
}

func TestNewBlogPostsError(t *testing.T) {
        _, err := blogposts.NewPostsFromFS(StubFailingFS{})
        if err == nil {
                t.Error("expected err but got nil")
        }
}

type StubFailingFS struct {
}

func (s StubFailingFS) Open(name string) (fs.File, error) {
        return nil, errors.New("oh no, I always fail")
}

func assertPost(t testing.TB, got blogposts.Post, want blogposts.Post) {
        t.Helper()
        if !reflect.DeepEqual(got, want) {
                t.Errorf("got %+v, want %+v", got, want)
        }
}

// 18-reading-files/blogposts.go
type Post struct {
        Title string
}

func NewPostsFromFS(fileSystem fs.FS) ([]Post, error) {
        dir, err := fs.ReadDir(fileSystem, ".")
        if err != nil {
```

```
                return nil, err
        }
        var posts []Post
        for _, f := range dir {
                post, err := getPost(fileSystem, f.Name())
                if err != nil {
                        // TODO: needs clarification, should we totally fail
if one file fails? or just ignore?
                        return nil, err
                }
                posts = append(posts, post)
        }
        return posts, nil
}

func getPost(fileSystem fs.FS, name string) (Post, error) {
        postFile, err := fileSystem.Open(name)
        if err != nil {
                return Post{}, err
        }
        defer postFile.Close()
        return newPost(postFile)
}

func newPost(postFile io.Reader) (Post, error) {
        postData, err := io.ReadAll(postFile)
        if err != nil {
                return Post{}, err
        }

        post := Post{Title: string(postData)[7:]}
        return post, nil
}
```

- Now for parsing our `Post`, we can take advantage of the fact that each field is on its own line and use the `bufio.Scanner` to read our file line-by-line. It takes an `io.Reader`, so we don't even need to change our function arguments. `Scan` doesn't return an `error` (you'd check `scanner.Err()` afterwards), but we will have to deal with invalid file structures later, so we leave the `error` in the signature.

```
// 18-reading-files/blogposts.go
func newPost(postFile io.Reader) (Post, error) {
        scanner := bufio.NewScanner(postFile)

        scanner.Scan()
        titleLine := scanner.Text()

        scanner.Scan()
        descriptionLine := scanner.Text()

        return Post{
                Title:       titleLine[7:],
                Description: descriptionLine[13:],
        }, nil
}
```

- This first draft passes the tests, but it is a little repetitive and relies on magic numbers. We also want to introduce some separation between the *what* and the *how* of reading lines. So we give it a refactor:

```
// 18-reading-files/blogposts.go
const (
        titleSeparator       = "Title: "
        descriptionSeparator = "Description: "
)

func newPost(postFile io.Reader) (Post, error) {
        scanner := bufio.NewScanner(postFile)

        readLine := func() string {
                scanner.Scan()
                return scanner.Text()
        }

        title := readLine()[len(titleSeparator):]
        description := readLine()[len(descriptionSeparator):]

        return Post{
                Title:       title,
                Description: description,
        }, nil
}
```

- We even riff on this a little further with a `readMetaLine` function that automatically trims the separator. The point of refactoring is we are free to play with the internal details and the tests make sure everything still works.

```
// 18-reading-files/blogposts.go
func getPost(fileSystem fs.FS, name string) (Post, error) {
        postFile, err := fileSystem.Open(name)
        if err != nil {
                return Post{}, err
        }
        defer postFile.Close()
        return newPost(postFile)
}

func newPost(postFile io.Reader) (Post, error) {
        scanner := bufio.NewScanner(postFile)

        readMetaLine := func(tagName string) string {
                scanner.Scan()
                return strings.TrimPrefix(scanner.Text(), tagName)
        }

        return Post{
                Title:       readMetaLine(titleSeparator),
                Description: readMetaLine(descriptionSeparator),
        }, nil
}
```

- When we get to handling `Body` we find that `scanner.Scan()` returns a `bool` which indicates whether there's more data to scan, so we can use it with a `for` loop to keep reading data until the end. After each scan, we write the data into a `strings.Builder` using `fmt.Fprintln`. We use the call that adds a newline because `Scanner` trims newlines, but we want to maintain them. We trim the extra newline at the end. We also refactor `readBody` out to its own function for this, since it's different from our other parsing.

```
// 18-reading-files/blogposts.go
func newPost(postBody io.Reader) (Post, error) {
        scanner := bufio.NewScanner(postBody)

        readMetaLine := func(tagName string) string {
                scanner.Scan()
                return strings.TrimPrefix(scanner.Text(), tagName)
        }

        return Post{
                Title:       readMetaLine(titleSeparator),
                Description: readMetaLine(descriptionSeparator),
                Tags:        strings.Split(readMetaLine(tagsSeparator), ",
"),
                Body:        readBody(scanner),
        }, nil
}

func readBody(scanner *bufio.Scanner) string {
        scanner.Scan() // ignore a line
        var b strings.Builder
        for scanner.Scan() {
                fmt.Fprintln(&b, scanner.Text())
        }
        return strings.TrimSuffix(b.String(), "\n")
}
```

- So we have coded our "happy path", but there are a few things we haven't handled:
  - when the file's format is not correct
  - the file is not a `.md`
  - what if the order of the metadata fields is different? Should that be allowed? Should we be able to handle it?
- To support all three we shouldn't have to change our design but only implementation details.
- If we wanted to process a real directory of files, we need only pass `os.DirFS("directory")` to our `NewPostsFromFS` function, where `directory` is the `directory` name. This purposely looks just like the call from our tests. The idea is that a consumer could look at our tests and see how to use our code. As maintainers, we are confident our tests are useful because they're from a consumer's point of view.

- It's important to know that `fs.FS` and the interfaces that extend it are read-only. If you need to write files, you should look at what else the standard library offers. Leverage existing interfaces such as `io.Writer`.
- For more on `io/fs`, there is a good blog post at https://benjamincongdon.me/blog/2021/01/21/A-Tour-of-Go-116s-iofs-package/.
- Final code. I named the test's `fstest.MapFS` `fsys` instead of the book's `fs`, since the test file also imports the `io/fs` package for `StubFailingFS` and `fs` would shadow it. The test also checks the number of posts again and gives the second post its own data, so a bug that drops or mixes up posts would fail:

```go
// 18-reading-files/blogposts_test.go
package blogposts_test

import (
	"errors"
	"io/fs"
	"reflect"
	"testing"
	"testing/fstest"

	blogposts "example.com/learn-go-with-tests/18-reading-files"
)

func TestNewBlogPosts(t *testing.T) {
	const (
		firstBody = `Title: Post 1
Description: Description 1
Tags: tdd, go
---
Hello
World`
		secondBody = `Title: Post 2
Description: Description 2
Tags: rust, borrow-checker
---
B
L
M`
	)

	fsys := fstest.MapFS{
		"hello world.md":  {Data: []byte(firstBody)},
		"hello-world2.md": {Data: []byte(secondBody)},
	}

	posts, err := blogposts.NewPostsFromFS(fsys)

	if err != nil {
		t.Fatal(err)
	}

	if len(posts) != len(fsys) {
		t.Fatalf("got %d posts, wanted %d posts", len(posts),
len(fsys))
	}
```

```
        assertPost(t, posts[0], blogposts.Post{
                Title:       "Post 1",
                Description: "Description 1",
                Tags:        []string{"tdd", "go"},
                Body: `Hello
World`,
        })
}

func TestNewBlogPostsError(t *testing.T) {
        _, err := blogposts.NewPostsFromFS(StubFailingFS{})
        if err == nil {
                t.Error("expected err but got nil")
        }
}

type StubFailingFS struct {
}

func (s StubFailingFS) Open(name string) (fs.File, error) {
        return nil, errors.New("oh no, I always fail")
}

func assertPost(t testing.TB, got blogposts.Post, want blogposts.Post) {
        t.Helper()
        if !reflect.DeepEqual(got, want) {
                t.Errorf("got %+v, want %+v", got, want)
        }
}

// 18-reading-files/blogposts.go
package blogposts

import (
        "bufio"
        "fmt"
        "io"
        "io/fs"
        "strings"
)

const (
        titleSeparator       = "Title: "
        descriptionSeparator = "Description: "
        tagsSeparator        = "Tags: "
```

```
)

// Post represents a post on a blog.
type Post struct {
        Title       string
        Description string
        Tags        []string
        Body        string
}

// NewPostsFromFS returns a collection of blog posts from a file system. If
it
// does not conform to the format then it'll return an error.
func NewPostsFromFS(fileSystem fs.FS) ([]Post, error) {
        dir, err := fs.ReadDir(fileSystem, ".")
        if err != nil {
                return nil, err
        }
        var posts []Post
        for _, f := range dir {
                post, err := getPost(fileSystem, f.Name())
                if err != nil {
                        // TODO: needs clarification, should we totally fail
if one file fails? or just ignore?
                        return nil, err
                }
                posts = append(posts, post)
        }
        return posts, nil
}

func getPost(fileSystem fs.FS, name string) (Post, error) {
        postFile, err := fileSystem.Open(name)
        if err != nil {
                return Post{}, err
        }
        defer postFile.Close()
        return newPost(postFile)
}

func newPost(postBody io.Reader) (Post, error) {
        scanner := bufio.NewScanner(postBody)

        readMetaLine := func(tagName string) string {
                scanner.Scan()
                return strings.TrimPrefix(scanner.Text(), tagName)
```

```
        }

        return Post{
                Title:       readMetaLine(titleSeparator),
                Description: readMetaLine(descriptionSeparator),
                Tags:        strings.Split(readMetaLine(tagsSeparator), ",
"),
                Body:        readBody(scanner),
        }, nil
}

func readBody(scanner *bufio.Scanner) string {
        scanner.Scan() // ignore a line
        var b strings.Builder
        for scanner.Scan() {
                fmt.Fprintln(&b, scanner.Text())
        }
        return strings.TrimSuffix(b.String(), "\n")
}
```

Chapter 19 - HTML Templates

- Go's standard library includes HTML templates for simple and maintainable HTML generation. Many websites don't need to be a SPA, and HTML and CSS are fantastic ways of delivering content.
- We are going to use the `Post` list returned by the last chapter's application and generate HTML from it. We want to generate a `View post` page for a specific `Post` and an `Index` that lists all posts with hyperlinks to specific posts.
- We will design our code to accept an `io.Writer`. This means the caller of our code has the flexibility to:
    - Write the HTML to an `os.File` to be statically served
    - Write out the HTML directly to an `http.ResponseWriter`
    - Or write them anywhere else that implements `io.Writer`
    - Taking an `io.Writer` also makes testing simple, since `bytes.Buffer` implements the interface
- It doesn't make sense to have an Index with nothing to link to, so we will start with rendering a specific Post. Also, we'll focus on just rendering the post title.

```go
// 19-html-templates/renderer_test.go
package blogrenderer_test

import (
	"bytes"
	"testing"

	blogrenderer "example.com/learn-go-with-tests/19-html-templates"
)

func TestRender(t *testing.T) {
	var (
		aPost = blogrenderer.Post{
			Title:       "hello world",
			Body:        "This is a post",
			Description: "This is a description",
			Tags:        []string{"go", "tdd"},
		}
	)

	t.Run("it converts a single post into HTML", func(t *testing.T) {
		buf := bytes.Buffer{}
		err := blogrenderer.Render(&buf, aPost)

		if err != nil {
			t.Fatal(err)
		}

		got := buf.String()
		want := `<h1>hello world</h1>`
		if got != want {
			t.Errorf("got '%s' want '%s'", got, want)
		}
	})
}

// 19-html-templates/renderer.go
package blogrenderer

import (
	"fmt"
	"io"
)

type Post struct {
```

```
        Title, Description, Body string
        Tags                     []string
}

func Render(w io.Writer, p Post) error {
        _, err := fmt.Fprintf(w, "<h1>%s</h1>", p.Title)
        return err
}
```

- We're working in small steps to get immediate feedback. Obviously our plan is to use an HTML templating library, but just writing strings is fine for the start. Note that strings won't sustain us for long. It *feels bad* when we have to update the string in our test too, and we haven't even put the body in:

```
// 19-html-templates/renderer_test.go
                want := `<h1>hello world</h1>
<p>This is a description</p>
Tags: <ul><li>go</li><li>tdd</li></ul>`
```

- Just to understand the value of templates, consider how much code it takes to get the test to pass for this three line string:

```go
// 19-html-templates/renderer.go
func Render(w io.Writer, p Post) error {
	_, err := fmt.Fprintf(w, "<h1>%s</h1>\n<p>%s</p>\n", p.Title,
p.Description)
	if err != nil {
		return err
	}

	_, err = fmt.Fprint(w, "Tags: <ul>")
	if err != nil {
		return err
	}

	for _, tag := range p.Tags {
		_, err = fmt.Fprintf(w, "<li>%s</li>", tag)
		if err != nil {
			return err
		}
	}

	_, err = fmt.Fprint(w, "</ul>")
	if err != nil {
		return err
	}

	return err
}
```

- Granted over half of that code is checking and returning `err`, but that's Go!
- Go has two templating packages that share the same interface: `text/template` and `html/template`. Both allow you to combine a template and some data and return a string. `html/template` offers safety against code injection and should always be used when the output is HTML. Go templates are very constrained, or "logic-less". This is an important and deliberate design decision. `text/template` is useful if your project is doing complex string concatenations and manipulation.
- We will ultimately put templates in files in a templates directory, but for now, let's just define our first template as a string.

```
// 19-html-templates/renderer.go
package blogrenderer

import (
        "html/template"
        "io"
)

const (
        postTemplate = `<h1>{{.Title}}</h1><p>{{.Description}}</p>Tags: <ul>
{{range .Tags}}<li>{{.}}</li>{{end}}</ul>`
)

type Post struct {
        Title, Description, Body string
        Tags                     []string
}

func Render(w io.Writer, p Post) error {
        templ, err := template.New("blog").Parse(postTemplate)
        if err != nil {
                return err
        }

        if err := templ.Execute(w, p); err != nil {
                return err
        }

        return nil
}

// we have a line break in our template before `range`, so we need a
linebreak
// in our `want` string in the same location.
// 19-html-templates/renderer_test.go
                want := `<h1>hello world</h1><p>This is a
description</p>Tags: <ul>
<li>go</li><li>tdd</li></ul>`
```

- We create a new template with a name and then parse our template string. We can then use the `Execute` method on it, passing in our data, in this case the `Post`. Our test would pass without changes if it wasn't sensitive to whitespace. You can see we can refer to our struct fields with `{{.Description}}` and loop over our tags with `{{range .Tags}}`. Templates

also support `if`. For more features see the documentation at https://pkg.go.dev/text/template.

- As the book points out, the verbosity of Go error handling is often complained about, but as we see here, if you write your code better you may need to check for and handle fewer errors.

- The whitespace bug would've been easier to catch with `%q`, but it is likely the book uses `'%s'` for HTML because `%q` escapes everything including quotes, which show up often in HTML and tend to generate noise. The book doesn't have this problem because it puts the template all on one line, but I inserted a newline where my book wrapped assuming it was part of the example. We're about to move the template string out of our code, so this isn't worth much hair pulling.

- We create a template directory called "templates", put our template in a file called `blog.gohtml` and add the following to `embed` the file system:

```
// 19-html-templates/templates/blog.gohtml
<h1>{{.Title}}</h1>

<p>{{.Description}}</p>

Tags:
<ul>
  {{range .Tags}}
    <li>{{.}}</li>
  {{end}}
</ul>

// 19-html-templates/renderer.go
var (
        //go:embed "templates/*"
        postTemplates embed.FS
)
// ...
func Render(w io.Writer, p Post) error {
        templ, err := template.ParseFS(postTemplates, "templates/*.gohtml")
```

- The `embed` package provides access to files embedded in the running Go program. The `//go:embed` directive can be used to initialize a variable of type `string`, `[]byte`, or `embed.FS` with the contents of files read from the package directory or subdirectories at compile time. The fact that we are embedding our templates means they ship with our

executable without packaging a separate folder. You can embed individual files, or whole directories as an `embed.FS` to interact with via `io/fs`.

- Our test is failing again due to whitespace concerns. This test would be a maintenance nightmare if we had to figure out the correct multiline string every time we tweak the template. But Go ApprovalTests allow for easy testing of larger objects, strings, and anything else that can be saved to a file (images, sounds, CSV, etc). The idea is similar to "golden" files. Rather than awkwardly maintaining strings in a test file, the approval tool can compare output for you with an "approved" file you created. If the output changes, you simply copy over a new approved version. You can add a dependency on `github.com/approvals/go-approval-tests` with `go get github.com/approvals/go-approval-tests`. We also added a `TestMain` that calls `approvals.UseFolder("testdata")` so it looks for approved files in the standard `testdata` directory. Then in the test we replace the string comparison with `approvals.VerifyString(t, buf.String())` (see the full test below).

- The test will fail at first, but you will have two new files in your folder (for us `testdata` because of our change - in the book the project folder).
    - `renderer_test.TestRender.it_converts_a_single_post_into_HTML.received.txt`
    - `renderer_test.TestRender.it_converts_a_single_post_into_HTML.approved.txt`
- All you need to do is look at the `received` file and if you approve the output copy it to the `approved` file. By copying the new version you have "approved" the change, and the test now passes. Be sure to commit the approved files to source control. Go ignores the `testdata` folder when building and distributing your application.

- Obviously we can't write our approved files first. I mean we could hand edit HTML to what we think the output would be, but that would be error prone. We don't have to use TDD dogmatically for everything. The important thing is that we used TDD as a design tool to design our package's API. We can keep the discipline of making small changes and re-running our test while approving changes in output.

- Go templates support defining sections which we can then import into other templates. This is useful for headers and footers and navbars, among other things. Let's add a "top" and "bottom" to our template.

```
// 19-html-templates/templates/blog.gohtml
{{template "top" .}}
<h1>{{.Title}}</h1>

<p>{{.Description}}</p>

Tags:
<ul>
  {{range .Tags}}
    <li>{{.}}</li>
  {{end}}
</ul>
{{template "bottom" .}}

// 19-html-templates/templates/top.gohtml
{{define "top"}}
  <!DOCTYPE HTML>
  <html lang="en">
    <head>
      <title>My amazing blog!</title>
      <meta charset="UTF-8" />
      <meta name="description"
            content="Wow, like and subscribe, it really helps the channel
guys"
            lang="en" />
    </head>
    <body>
      <nav role="navigation">
        <div>
          <h1>Budding Gopher's blog</h1>
          <ul>
            <li><a href="/">home</a></li>
            <li><a href="about">about</a></li>
            <li><a href="archive">archive</a></li>
        </ul>
      </div>
    </nav>
  <main>
{{end}}

// 19-html-templates/templates/bottom.gohtml
{{define "bottom"}}
  </main>
  <footer>
    <ul>
```

```
      <li><a href="https://twitter.com/llcawthorne">Twitter</a></li>
      <li><a href="https://github.com/llcawthorne">GitHub</a></li>
    </ul>
  </footer>
  </body>
  </html>
{{end}}

// 19-html-templates/renderer_test.go
package blogrenderer_test

import (
        "bytes"
        "os"
        "testing"

        blogrenderer "example.com/learn-go-with-tests/19-html-templates"
        approvals "github.com/approvals/go-approval-tests"
)

func TestMain(m *testing.M) {
        approvals.UseFolder("testdata")
        os.Exit(m.Run())
}

func TestRender(t *testing.T) {
        var (
                aPost = blogrenderer.Post{
                        Title:       "hello world",
                        Body:        "This is a post",
                        Description: "This is a description",
                        Tags:        []string{"go", "tdd"},
                }
        )

        t.Run("it converts a single post into HTML", func(t *testing.T) {
                buf := bytes.Buffer{}

                if err := blogrenderer.Render(&buf, aPost); err != nil {
                        t.Fatal(err)
                }

                approvals.VerifyString(t, buf.String())
        })
}
```

```
// 19-html-templates/renderer.go
        if err := templ.ExecuteTemplate(w, "blog.gohtml", p); err != nil {
                return err
        }
```

- We have a minor performance problem on our hands. We are reparsing all the templates for each post. It isn't a big deal really, but it's so easy to fix we might as well; plus it cleans up the code. We can see the difference by writing (and running) a benchmark.

```
// 19-html-templates/renderer_test.go
func BenchmarkRender(b *testing.B) {
        var (
                aPost = blogrenderer.Post{
                        Title:       "hello world",
                        Body:        "This is a post",
                        Description: "This is a description",
                        Tags:        []string{"go", "tdd"},
                }
        )

        for b.Loop() {
                blogrenderer.Render(io.Discard, aPost)
        }
}
```

- And when we run this (`go test -bench=.`) the results are `BenchmarkRender-10 47896 25399 ns/op`. Now instead let's create a type that will hold the parsed template with a method to do the rendering.

```go
// 19-html-templates/renderer.go
type PostRenderer struct {
	templ *template.Template
}

func NewPostRenderer() (*PostRenderer, error) {
	templ, err := template.ParseFS(postTemplates, "templates/*.gohtml")
	if err != nil {
		return nil, err
	}
	return &PostRenderer{templ: templ}, nil
}

func (r *PostRenderer) Render(w io.Writer, p Post) error {

	if err := r.templ.ExecuteTemplate(w, "blog.gohtml", p); err != nil {
		return err
	}

	return nil
}

// The interface changed, so we need to adjust the tests including the new
bench
// 19-html-templates/renderer_test.go
func TestMain(m *testing.M) {
	approvals.UseFolder("testdata")
	os.Exit(m.Run())
}

func TestRender(t *testing.T) {
	var (
		aPost = blogrenderer.Post{
			Title:       "hello world",
			Body:        "This is a post",
			Description: "This is a description",
			Tags:        []string{"go", "tdd"},
		}
	)

	postRenderer, err := blogrenderer.NewPostRenderer()

	if err != nil {
		t.Fatal(err)
	}
```

```
        t.Run("it converts a single post into HTML", func(t *testing.T) {
                buf := bytes.Buffer{}

                if err := postRenderer.Render(&buf, aPost); err != nil {
                        t.Fatal(err)
                }

                approvals.VerifyString(t, buf.String())
        })
}

func BenchmarkRender(b *testing.B) {
        var (
                aPost = blogrenderer.Post{
                        Title:       "hello world",
                        Body:        "This is a post",
                        Description: "This is a description",
                        Tags:        []string{"go", "tdd"},
                }
        )

        postRenderer, err := blogrenderer.NewPostRenderer()

        if err != nil {
                b.Fatal(err)
        }

        for b.Loop() {
                postRenderer.Render(io.Discard, aPost)
        }
}
```

- That was a pretty big refactor that didn't break any tests! And now when we run the benchmark, we get `BenchmarkRender-10 876776 1356 ns/op`. We went down from 25399 ns/op to 1356 ns/op. The second number went up, but that is because it is the number of loops it ran which for a faster operation takes more iterations to fill a second or so.

- Now we want to render the `Body`. It should be markdown the author has written. This is left as an exercise. One warning is to be careful not to explicitly test how the 3rd party library behaves in unit tests. Writing tests against code you don't control is wasteful and adds maintenance overhead. Converting markdown to HTML is an implementation detail of rendering, and our approval test should give us enough confidence.

- Now we add an `Index` but we initially try to test it with strings:

```
// 19-html-templates/renderer_test.go
        t.Run("it renders an index of posts", func(t *testing.T) {
                buf := bytes.Buffer{}
                posts := []blogrenderer.Post{{Title: "Hello World"}, {Title:
"Hello World 2"}}

                if err := postRenderer.RenderIndex(&buf, posts); err != nil {
                        t.Fatal(err)
                }

                got := buf.String()
                want := `<ol><li><a href="/post/hello-world">Hello World</a>
</li><li><a href="/post/hello-world-2">Hello World 2</a></li></ol>`

                if got != want {
                        t.Errorf("got %q want %q", got, want)
                }
        })

// 19-html-templates/renderer.go
func (r *PostRenderer) RenderIndex(w io.Writer, posts []Post) error {
        indexTemplate := `<ol>{{range .}}<li><a href="/post/{{.Title}}">
{{.Title}}</a></li>{{end}}</ol>`

        templ, err := template.New("index").Parse(indexTemplate)
        if err != nil {
                return err
        }

        if err := templ.Execute(w, posts); err != nil {
                return err
        }

        return nil
}
```

- We do have a problem with constructing the URL from the title. We want it to be `/post/hello-world` but instead it is `/post/Hello%20World`. This allows us to demonstrate a cool feature of templates. Before you parse a template you can add a `template.FuncMap` into your template which allows you to define functions that can be called within your

template. So we are going to define a `sanitizeTitle` function and pass it in. So this passes our test:

```
// 19-html-templates/renderer.go
func (r *PostRenderer) RenderIndex(w io.Writer, posts []Post) error {
        indexTemplate := `<ol>{{range .}}<li><a href="/post/{{sanitizeTitle
.Title}}">{{.Title}}</a></li>{{end}}</ol>`

        templ, err := template.New("index").Funcs(template.FuncMap{
                "sanitizeTitle": func(title string) string {
                        return strings.ToLower(strings.Replace(title, " ", "-
", -1))
                },
        }).Parse(indexTemplate)
        if err != nil {
                return err
        }

        if err := templ.Execute(w, posts); err != nil {
                return err
        }

        return nil
}
```

- Now, we can do this, but should we? By passing in functions, we're really losing the benefit of logic-less templates. We're also limited to using approval tests to test these outputs. So in general, this is a bad idea. We could add a method to `Post` and then call it in the template. That way we can test the method on `Post` directly. The downside is that it is still view logic and can easily lead to God Objects. We could instead take a view model approach where, instead of our template being coupled to the domain object `Post`, we have a `PostViewModel` that contains all the data we need. We would encapsulate the mapping logic in a `NewPostViewModel(p Post) PostViewModel` function. This keeps our templates logic-less with the downside of a slightly more convoluted process to get our posts rendered. Here we are going to keep things simple and just add a method to `Post`.

```
// 19-html-templates/renderer.go
func (p Post) SanitizedTitle() string {
        return strings.ToLower(strings.Replace(p.Title, " ", "-", -1))
}

func (r *PostRenderer) RenderIndex(w io.Writer, posts []Post) error {
        indexTemplate := `<ol>{{range .}}<li><a
href="/post/{{.SanitizedTitle}}">{{.Title}}</a></li>{{end}}</ol>`

        templ, err := template.New("index").Parse(indexTemplate)
        if err != nil {
                return err
        }

        if err := templ.Execute(w, posts); err != nil {
                return err
        }

        return nil
}
```

- This gets our tests passing and gives us an easy to test method without really complicating our design. You see in our template we simply call `{{.SanitizedTitle}}` instead of `{{.Title}}`. Now we will factor this template out of a string and into a `templates/index.gohtml` file. We also simplified both rendering functions:

```
// 19-html-templates/renderer.go
func (r *PostRenderer) Render(w io.Writer, p Post) error {
        return r.templ.ExecuteTemplate(w, "blog.gohtml", p)
}

func (r *PostRenderer) RenderIndex(w io.Writer, posts []Post) error {
        return r.templ.ExecuteTemplate(w, "index.gohtml", posts)
}

// 19-html-templates/renderer_test.go
        t.Run("it renders an index of posts", func(t *testing.T) {
                buf := bytes.Buffer{}
                posts := []blogrenderer.Post{{Title: "Hello World"}, {Title:
"Hello World 2"}}

                if err := postRenderer.RenderIndex(&buf, posts); err != nil {
                        t.Fatal(err)
                }

                approvals.VerifyString(t, buf.String())
        })

// 19-html-templates/templates/index.gohtml
{{template "top" .}}
<ol>
  {{range .}}
    <li><a href="/post/{{.SanitizedTitle}}">{{.Title}}</a></li>
  {{end}}
</ol>
{{template "bottom" .}}
```

- The author provided a solution to parsing the markdown body into HTML with `github.com/gomarkdown/markdown`. When adding markdown parsing he chose to implement a `postViewModel` that embeds the original `Post` along with the parsed markdown as `HTMLBody`. He also used `template.HTML` to mark this as safe HTML that doesn't need to be escaped in the template.
    - The repo's code (`blogrenderer/renderer.go`) compiles and passes its tests with current gomarkdown. The catch is that a gomarkdown `parser.Parser` keeps internal state and isn't safe to reuse: parsing twice with the same parser panics. So `newPostVM` creates a fresh parser on every call instead of storing one on `PostRenderer`. My copy of the book's snippet didn't work, so the final code below uses the repo version.

    - Add the dependency with `go get github.com/gomarkdown/markdown`, add `{{.HTMLBody}}` to `blog.gohtml`, and give the test post a markdown `Body` so the approval test covers the conversion. The re-approved output has `<h1 id="first-recipe">` (from `AutoHeadingIDs`) and `<strong>amazing blog</strong>`.
- Final code and templates:

```go
// 19-html-templates/renderer_test.go
package blogrenderer_test

import (
	"bytes"
	"io"
	"os"
	"testing"

	blogrenderer "example.com/learn-go-with-tests/19-html-templates"
	approvals "github.com/approvals/go-approval-tests"
)

func TestMain(m *testing.M) {
	approvals.UseFolder("testdata")
	os.Exit(m.Run())
}

func TestRender(t *testing.T) {
	var (
		aPost = blogrenderer.Post{
			Title: "hello world",
			Body: `# First recipe!
Welcome to my **amazing blog**. I am going to write about my family recipes,
and make sure I write a long, irrelevant and boring story about my family
before you get to the actual instructions.`,
			Description: "This is a description",
			Tags:        []string{"go", "tdd"},
		}
	)

	postRenderer, err := blogrenderer.NewPostRenderer()

	if err != nil {
		t.Fatal(err)
	}

	t.Run("it converts a single post into HTML", func(t *testing.T) {
		buf := bytes.Buffer{}

		if err := postRenderer.Render(&buf, aPost); err != nil {
			t.Fatal(err)
		}

		approvals.VerifyString(t, buf.String())
```

```
	})

	t.Run("it renders an index of posts", func(t *testing.T) {
		buf := bytes.Buffer{}
		posts := []blogrenderer.Post{{Title: "Hello World"}, {Title:
"Hello World 2"}}

		if err := postRenderer.RenderIndex(&buf, posts); err != nil {
			t.Fatal(err)
		}

		approvals.VerifyString(t, buf.String())
	})
}

func BenchmarkRender(b *testing.B) {
	var (
		aPost = blogrenderer.Post{
			Title:       "hello world",
			Body:        "This is a post",
			Description: "This is a description",
			Tags:        []string{"go", "tdd"},
		}
	)

	postRenderer, err := blogrenderer.NewPostRenderer()

	if err != nil {
		b.Fatal(err)
	}

	for b.Loop() {
		postRenderer.Render(io.Discard, aPost)
	}
}

// 19-html-templates/renderer.go
package blogrenderer

import (
	"embed"
	"html/template"
	"io"
	"strings"

	"github.com/gomarkdown/markdown"
```

```
        "github.com/gomarkdown/markdown/parser"
)

var (
        //go:embed "templates/*"
        postTemplates embed.FS
)

// Post is a representation of a post.
type Post struct {
        Title, Description, Body string
        Tags                     []string
}

// SanitizedTitle returns the title of the post with spaces replaced by
dashes
// for pleasant URLs.
func (p Post) SanitizedTitle() string {
        return strings.ToLower(strings.ReplaceAll(p.Title, " ", "-"))
}

// PostRenderer renders data into HTML.
type PostRenderer struct {
        templ *template.Template
}

// NewPostRenderer creates a new PostRenderer.
func NewPostRenderer() (*PostRenderer, error) {
        templ, err := template.ParseFS(postTemplates, "templates/*.gohtml")
        if err != nil {
                return nil, err
        }
        return &PostRenderer{templ: templ}, nil
}

// Render renders post into HTML.
func (r *PostRenderer) Render(w io.Writer, p Post) error {
        return r.templ.ExecuteTemplate(w, "blog.gohtml", newPostVM(p))
}

// RenderIndex creates an HTML index page given a collection of posts.
func (r *PostRenderer) RenderIndex(w io.Writer, posts []Post) error {
        return r.templ.ExecuteTemplate(w, "index.gohtml", posts)
}

type postViewModel struct {
```

```
        Post
        HTMLBody template.HTML
}

// newPostVM converts the post's markdown Body to HTML. gomarkdown parsers
keep
// internal state and panic if reused, so each call makes a fresh one.
func newPostVM(p Post) postViewModel {
        vm := postViewModel{Post: p}
        extensions := parser.CommonExtensions | parser.AutoHeadingIDs
        mdParser := parser.NewWithExtensions(extensions)
        vm.HTMLBody = template.HTML(markdown.ToHTML([]byte(p.Body), mdParser,
nil))
        return vm
}

// 19-html-templates/templates/top.gohtml
{{define "top"}}
  <!DOCTYPE HTML>
  <html lang="en">
    <head>
      <title>My amazing blog!</title>
      <meta charset="UTF-8" />
      <meta name="description"
            content="Wow, like and subscribe, it really helps the channel
guys"
            lang="en" />
    </head>
    <body>
      <nav role="navigation">
        <div>
          <h1>Budding Gopher's blog</h1>
          <ul>
            <li><a href="/">home</a></li>
            <li><a href="about">about</a></li>
            <li><a href="archive">archive</a></li>
          </ul>
        </div>
      </nav>
      <main>
{{end}}

// 19-html-templates/templates/bottom.gohtml
{{define "bottom"}}
  </main>
  <footer>
```

```
    <ul>
      <li><a href="https://twitter.com/llcawthorne">Twitter</a></li>
      <li><a href="https://github.com/llcawthorne">GitHub</a></li>
    </ul>
  </footer>
  </body>
  </html>
{{end}}

// 19-html-templates/templates/blog.gohtml
{{template "top" .}}
<h1>{{.Title}}</h1>
<p>{{.Description}}</p>
Tags:
<ul>
  {{range .Tags}}
    <li>{{.}}</li>
  {{end}}
</ul>
{{.HTMLBody}}
{{template "bottom" .}}

// 19-html-templates/templates/index.gohtml
{{template "top" .}}
<ol>
  {{range .}}
    <li><a href="/post/{{.SanitizedTitle}}">{{.Title}}</a></li>
  {{end}}
</ol>
{{template "bottom" .}}
```

- Wrapping up
  - What we've learned
    - How to create and render HTML templates.
    - How to compose templates together and DRY up related markup and help us keep a consistent look and feel.
    - How to pass functions into templates, and why you should think twice about it.
    - How to write "Approval Tests", which help us test the big ugly output of things like template renderers.
  - On logic-less templates
    - As always, this is about **separation of concerns**. It's important that we consider what the responsibilities are of the various parts of our system. Too often people

leak important business logic into templates, mixing up concerns and making systems difficult to understand, maintain and test.

- Not just for HTML
  - Remember that Go has a `text/template` to generate other kinds of data from a template. If you find yourself needing to transform data into some kind of structured output, the techniques laid out in this chapter can be useful.
- Interactivity: Hotwire vs. htmx
  - The book recommends Hotwire (its Turbo library swaps in HTML fetched from the server) for adding interactivity to server-rendered pages. Turbo works with any backend, not just Rails.
  - In the Go world, **htmx** is much more popular for the same "HTML over the wire" idea. You add HTML attributes like `hx-get`, `hx-post`, `hx-target`, `hx-swap`, and `hx-trigger` to elements, and htmx makes the request and swaps the returned HTML fragment into the page.
  - Advantages: a single script tag with no build step or JavaScript framework, the server stays in charge of rendering (so `html/template` and approval tests still cover what users see), and handlers just return small HTML fragments.
  - The **GOTH stack** is **Go + templ + htmx**. *templ* (`github.com/a-h/templ`) is a templating language that compiles `.templ` files into type-checked Go functions, so template mistakes become compile errors. Search "GOTH stack" or "templ htmx" for tutorials.

Chapter 20 - Generics

- Generics offer us a way to make abstractions by letting us **describe our constraints**. They allow us to write functions that have a similar level of flexibility that `interface{}` offers but retain type-safety and provide a better developer experience for callers.

```go
// 20-generics/assert.go
func AssertEqual[T comparable](t testing.TB, got, want T) {
        t.Helper()
        if got != want {
                t.Errorf("got %v, want %v", got, want)
        }
}

func AssertNotEqual[T comparable](t testing.TB, got, want T) {
        t.Helper()
        if got == want {
                t.Errorf("didn't want %v", got)
        }
}
```

- Declaring `GenericFoo[T any](x, y T)` is different from `InterfaceFoo(x, y any)`, because in the first the two parameters must share a type. There is only one type `T` throughout. If your function returns a generic type, the caller can use the type as it was rather than having to make a type assertion when the function returns `interface{}` (or `any`), where the compiler can make no guarantees about type.
- Without generics, we have to define an `Assert` function for each type, and a `Stack` to hold each type or throw out type safety with `any`. Without type safety, you can push apples on a `Stack` of oranges. Plus when it returns an `interface{}`, it's horrible to work with.
- In places where the generic type cannot be inferred, you need to specify the type `T` by specifying the types inside square brackets before the arguments. Normally for a function call, it can determine the type from your arguments. But consider:

```go
// 20-generics/stack.go
func NewStack[T any]() *Stack[T] {
        return new(Stack[T])
}

// To use this constructor, you would call it like:
myStackOfInts := NewStack[int]()
myStackOfStrings := NewStack[string]()
```

- Final code:

```go
// 20-generics/generics_test.go
package generics

import "testing"

func TestAssertFunctions(t *testing.T) {
	t.Run("asserting on integers", func(t *testing.T) {
		AssertEqual(t, 1, 1)
		AssertNotEqual(t, 1, 2)
	})
	t.Run("asserting on strings", func(t *testing.T) {
		AssertEqual(t, "hello", "hello")
		AssertNotEqual(t, "hello", "Grace")
	})
}

func TestStack(t *testing.T) {
	t.Run("integer stack", func(t *testing.T) {
		myStackOfInts := NewStack[int]()

		AssertTrue(t, myStackOfInts.IsEmpty())

		myStackOfInts.Push(123)
		AssertFalse(t, myStackOfInts.IsEmpty())

		myStackOfInts.Push(456)
		value, _ := myStackOfInts.Pop()
		AssertEqual(t, value, 456)
		value, _ = myStackOfInts.Pop()
		AssertEqual(t, value, 123)
		AssertTrue(t, myStackOfInts.IsEmpty())

		myStackOfInts.Push(1)
		myStackOfInts.Push(2)
		firstNum, _ := myStackOfInts.Pop()
		secondNum, _ := myStackOfInts.Pop()
		AssertEqual(t, firstNum+secondNum, 3)
	})
}

// 20-generics/assert.go
package generics

import "testing"
```

```go
func AssertEqual[T comparable](t testing.TB, got, want T) {
	t.Helper()
	if got != want {
		t.Errorf("got %v, want %v", got, want)
	}
}

func AssertNotEqual[T comparable](t testing.TB, got, want T) {
	t.Helper()
	if got == want {
		t.Errorf("didn't want %v", got)
	}
}

func AssertTrue(t testing.TB, got bool) {
	t.Helper()
	if !got {
		t.Errorf("got %v, want true", got)
	}
}

func AssertFalse(t testing.TB, got bool) {
	t.Helper()
	if got {
		t.Errorf("got %v, want false", got)
	}
}

// 20-generics/stack.go
package generics

type Stack[T any] struct {
	values []T
}

func NewStack[T any]() *Stack[T] {
	return new(Stack[T])
}

func (s *Stack[T]) Push(value T) {
	s.values = append(s.values, value)
}

func (s *Stack[T]) IsEmpty() bool {
	return len(s.values) == 0
}
```

```
func (s *Stack[T]) Pop() (T, bool) {
        if s.IsEmpty() {
                var zero T
                return zero, false
        }

        index := len(s.values) - 1
        el := s.values[index]
        s.values = s.values[:index]
        return el, true
}
```

- Wrapping up
    - Generics are simpler than using `interface{}` in most cases
        - In particular using `interface{}` makes your code:
            - Less safe (mix apples and oranges), requires more error handling
            - Less expressive, `interface{}` tells you nothing about the data
            - More likely to rely on reflection, type-assertions, etc; which makes your code more difficult to work with and more error prone as it pushes checks from compile-time to runtime
        - Using statically typed languages is an act of describing constraints. If you do it well, you create code that is not only safe and simple to use but also simpler to write because the possible solution space is smaller. Generics give us a new way to express constraints in our code.
    - Resist making things generic until you can see that you have a useful generalization. People often advise to only generalize when you see the same code three times.

Chapter 21 - Revisiting arrays and slices with generics

- Consider these two functions:

```
// Sum calculates the total from a slice of numbers.
func Sum(numbers []int) int {
        var sum int
        for _, number := range numbers {
                sum += number
        }
        return sum
}

// SumAllTails calculates the sums of all but the first number given a
// collection of slices.
func SumAllTails(numbersToSum ...[]int) []int {
        var sums []int
        for _, numbers := range numbersToSum {
                if len(numbers) == 0 {
                        sums = append(sums, 0)
                } else {
                        tail := numbers[1:]
                        sums = append(sums, Sum(tail))
                }
        }

        return sums
}
```

- They share a pattern.
    - Create some kind of "initial" result value.
    - Iterate over the collection, applying some kind of operation (or function) to the result and the next item in the slice, setting a new value for the result.
    - Return the result.
- This is often called "reduce" or "fold" by the functional programming community. Fold/reduce sounds scary, but it is simply an abstraction over taking a collection and combining it into one item, which you probably do a lot.
- Here is a first pass of `Reduce`:

```
func Reduce[A any](collection []A, f func(A, A) A, initialValue A) A {
        var result = initialValue
        for _, x := range collection {
                result = f(result, x)
        }
        return result
}
```

- Now we can rewrite `Sum` and `SumAllTails`:

```
// 21-revisiting-arrays/sum.go
func Sum(numbers []int) int {
        add := func(acc, x int) int { return acc + x }
        return Reduce(numbers, add, 0)
}

// SumAllTails calculates the sums of all but the first number given a
collection of slices.
func SumAllTails(numbers ...[]int) []int {
        sumTail := func(acc, x []int) []int {
                if len(x) == 0 {
                        return append(acc, 0)
                } else {
                        tail := x[1:]
                        return append(acc, Sum(tail))
                }
        }

        return Reduce(numbers, sumTail, []int{})
}
```

- What if we wish to reduce into a different type from `A`? That is fine and only requires a minor adjustment to the type of `Reduce`: `func Reduce[A, B any](collection []A, f func(B, A) B, initialValue B) B`. This allows people to reduce from a collection of `A` into a `B`, but still works when `A` and `B` are the same type.
- Thanks to generics, you no longer have to write type-specific `Find` functions for each type of collection you want to search.

```go
// 21-revisiting-arrays/sum_test.go
func TestFind(t *testing.T) {
	t.Run("find first even number", func(t *testing.T) {
		numbers := []int{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

		firstEvenNumber, found := Find(numbers, func(x int) bool {
			return x%2 == 0
		})
		AssertTrue(t, found)
		AssertEqual(t, firstEvenNumber, 2)
	})
}

// 21-revisiting-arrays/collection_fun.go
func Find[A any](items []A, predicate func(A) bool) (value A, found bool) {
	for _, v := range items {
		if predicate(v) {
			return v, true
		}
	}
	return
}
```

- Final code. The chapter also reduces a `[]Transaction` into an `Account` to compute bank balances (`bad_bank.go`), an example where `A` and `B` differ. As in earlier chapters, my test helpers take `testing.TB` instead of the book's `*testing.T` :

```go
// 21-revisiting-arrays/assert.go
package main

import "testing"

func AssertEqual[T comparable](t testing.TB, got, want T) {
	t.Helper()
	if got != want {
		t.Errorf("got %v, want %v", got, want)
	}
}

func AssertNotEqual[T comparable](t testing.TB, got, want T) {
	t.Helper()
	if got == want {
		t.Errorf("didn't want %v", got)
	}
}

func AssertTrue(t testing.TB, got bool) {
	t.Helper()
	if !got {
		t.Errorf("got %v, want true", got)
	}
}

func AssertFalse(t testing.TB, got bool) {
	t.Helper()
	if got {
		t.Errorf("got %v, want false", got)
	}
}

// 21-revisiting-arrays/collection_fun.go
package main

func Find[A any](items []A, predicate func(A) bool) (value A, found bool) {
	for _, v := range items {
		if predicate(v) {
			return v, true
		}
	}
	return
}
```

```
func Reduce[A, B any](collection []A, f func(B, A) B, initialValue B) B {
        var result = initialValue
        for _, x := range collection {
                result = f(result, x)
        }
        return result
}

// 21-revisiting-arrays/sum_test.go
package main

import (
        "slices"
        "strings"
        "testing"
)

func TestSum(t *testing.T) {

        t.Run("collections of any size", func(t *testing.T) {

                numbers := []int{1, 2, 3}

                got := Sum(numbers)
                want := 6

                if got != want {
                        t.Errorf("got %d want %d given, %v", got, want,
numbers)
                }
        })

}

func TestSumAllTails(t *testing.T) {

        checkSums := func(t testing.TB, got, want []int) {
                t.Helper()
                if !slices.Equal(got, want) {
                        t.Errorf("got %v want %v", got, want)
                }
        }

        t.Run("make the sums of tails of slices", func(t *testing.T) {
                got := SumAllTails([]int{1, 2}, []int{0, 9})
                want := []int{2, 9}
```

```
                checkSums(t, got, want)
        })

        t.Run("safely sum empty slices", func(t *testing.T) {
                got := SumAllTails([]int{}, []int{3, 4, 5})
                want := []int{0, 9}
                checkSums(t, got, want)
        })

}

func TestReduce(t *testing.T) {
        t.Run("multiplication of all elements", func(t *testing.T) {
                multiply := func(x, y int) int {
                        return x * y
                }

                AssertEqual(t, Reduce([]int{1, 2, 3}, multiply, 1), 6)
        })

        t.Run("concatenate strings", func(t *testing.T) {
                concatenate := func(x, y string) string {
                        return x + y
                }

                AssertEqual(t, Reduce([]string{"a", "b", "c"}, concatenate,
""), "abc")
        })
}

func TestFind(t *testing.T) {
        t.Run("find first even number", func(t *testing.T) {
                numbers := []int{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

                firstEvenNumber, found := Find(numbers, func(x int) bool {
                        return x%2 == 0
                })
                AssertTrue(t, found)
                AssertEqual(t, firstEvenNumber, 2)
        })

        type Person struct {
                Name string
        }

        t.Run("Find the best programmer", func(t *testing.T) {
```

```
                people := []Person{
                        {Name: "Kent Beck"},
                        {Name: "Martin Fowler"},
                        {Name: "Chris James"},
                }

                king, found := Find(people, func(p Person) bool {
                        return strings.Contains(p.Name, "Chris")
                })

                AssertTrue(t, found)
                AssertEqual(t, king, Person{Name: "Chris James"})
        })
}

// 21-revisiting-arrays/sum.go
package main

// Sum calculates the total from a slice of numbers.
func Sum(numbers []int) int {
        add := func(acc, x int) int { return acc + x }
        return Reduce(numbers, add, 0)
}

// SumAllTails calculates the sums of all but the first number given a
collection of slices.
func SumAllTails(numbers ...[]int) []int {
        sumTail := func(acc, x []int) []int {
                if len(x) == 0 {
                        return append(acc, 0)
                } else {
                        tail := x[1:]
                        return append(acc, Sum(tail))
                }
        }

        return Reduce(numbers, sumTail, []int{})
}

// 21-revisiting-arrays/bad_bank_test.go
package main

import "testing"

func TestBadBank(t *testing.T) {
        var (
```

```
                riya  = Account{Name: "Riya", Balance: 100}
                chris = Account{Name: "Chris", Balance: 75}
                adil  = Account{Name: "Adil", Balance: 200}

                transactions = []Transaction{
                        NewTransaction(chris, riya, 100),
                        NewTransaction(adil, chris, 25),
                }
        )

        newBalanceFor := func(account Account) float64 {
                return NewBalanceFor(account, transactions).Balance
        }

        AssertEqual(t, newBalanceFor(riya), 200)
        AssertEqual(t, newBalanceFor(chris), 0)
        AssertEqual(t, newBalanceFor(adil), 175)
}

// 21-revisiting-arrays/bad_bank.go
package main

type Transaction struct {
        From string
        To   string
        Sum  float64
}

func NewTransaction(from, to Account, sum float64) Transaction {
        return Transaction{From: from.Name, To: to.Name, Sum: sum}
}

type Account struct {
        Name    string
        Balance float64
}

func NewBalanceFor(account Account, transactions []Transaction) Account {
        return Reduce(
                transactions,
                applyTransaction,
                account,
        )
}

func applyTransaction(a Account, transaction Transaction) Account {
```

```
        if transaction.From == a.Name {
                a.Balance -= transaction.Sum
        }
        if transaction.To == a.Name {
                a.Balance += transaction.Sum
        }
        return a
}
```

- Wrapping up
    - When done tastefully, higher-order functions like these will make your code easier to read and maintain, but remember the rule of thumb:
        - Use the TDD process to drive out real, specific behaviour that you actually need, in the refactoring stage you then might discover some useful abstractions to help tidy the code up.
    - Practice combining TDD with good source control habits. Commit your work when your test is passing before trying to refactor. This way if you make a mess, you can easily get yourself back to your working state.
    - Use commonly established names for commonly occurring patterns. Need to convert a collection of `A` to a collection of `B`? Call it `Map`, not `Convert`. Using the "proper" name for these items will reduce cognitive burden for others.

Chapter 22 - Introduction to acceptance testing